

A Project Stage-II Report on

AI REVOLUTIONIZING E- COMMERCE GOVERNANCE FOR FRAUD DETECTION

Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

By

V. AISHWARYA (21UJ1A6740)
P.G.N.K. SAI PAVAN (21UJ1A6729)
VISHAKHA ROY (21UJ1A6741)
A. GOUTHAMI (22UJ5A6701)

Under the esteemed guidance of

Mr. SURENDER REDDY SAMA_{M.Tech.}

Assistant Professor

Department of Computer Science and Engineering (Data Science)

At



MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

2021-2025

Affiliated to



**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
KUKATPALLY, HYDERABAD-85.**



MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

CERTIFICATE

This is to certify that the Project Stage-II report entitled “**AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION**” is the bonafide work carried out and submitted by

V. AISHWARYA	(21UJ1A6740)
P.G.N.K. SAI PAVAN	(21UJ1A6729)
VISHAKHA ROY	(21UJ1A6741)
A. GOUTHAMI	(22UJ5A6701)

To the department of **Computer Science and Engineering (Data Science)**, **Malla Reddy Engineering College and Management Sciences**, in partial fulfilment for the award of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)** during the academic year 2024-2025.

Internal Guide
Mr. SURENDER REDDY SAMA M.Tech.
Assistant Professor
Dept .of CSE (Data Science)

Head of The Department
Dr.Zaheer Sultana M.Tech, PhD.
Assistant Professor
Dept .of CSE (Data Science)

EXTERNAL

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express our guidance for all of them.

We are thankful to **Mr.V.MALLA REDDY** chairman of Malla Reddy Engineering College and Management Sciences for accepting us and providing us with an opportunity to do a project in their esteemed organization.

We are thankful to Principal **Dr.M.SREEDHAR REDDY** M.Tech., Ph.D. Malla Reddy Engineering College and Management Sciences for helping us to undergo project work as a part of university curriculum.

Our special thanks to **Dr. ZAHEER SULTANA** M.Tech,PhD.. Assistant Professor& Head in Computer Science and Engineering (Data Science) Department and **Mr. SURRENDER REDDY SAMA** M.Tech. Assistant Professor in Computer Science and Engineering (Data Science) Department for guiding us in the right way to complete the project in the right time.

We would like to thank our internal project mates and department faculties for their full- fledged guidance and giving courage to carry out the project.

We are very much thankful to one and all that helped us for the successful completion of our project.

V. AISHWARYA	(21UJ1A6740)
P.G.N.K. SAI PAVAN	(21UJ1A6729)
VISHAKHA ROY	(21UJ1A6741)
A. GOUTHAMI	(22UJ5A6701)

TABLE OF CONTENT

Abstract	i
List of Figures	ii
List of Tables	iv
Abbreviations	v
CHAPTER 1: INTRODUCTION	1
1.1 OVERVIEW	1
1.2 RESEARCH MOTIVATION	1
1.3 PROBLEM DEFINITION	2
1.4 SIGNIFICANCE	3
1.5 RESEARCH OBJECTIVE	3
1.6 ADVANTAGES	3
1.7 APPLICATIONS	4
CHAPTER 2: LITERATURE SURVEY	6
CHAPTER 3: TRADITIONAL SYSTEM	26
CHAPTER 4: PROPOSED SYSTEM	29
4.1 SYSTEM ARCHITECTURE	29
4.2 EDA	32
4.3 DATA PRE-PROCESSING	34
4.4 MODEL BUILDING	36
4.4.1 Existing Algorithm: Support Vector Machine	36
4.4.2 Existing Algorithm: Random Forest Classifier	38
4.4.3 Existing Algorithm: Logistic Regression Classifier	41
4.4.4 Proposed Algorithm: Gradient Boosting Classifier	43
4.5 HTML, CSS, and FLASK	44
CHAPTER 5: UML DIAGRAMS	48
5.1 CLASS DIAGRAM	49
5.2 USE CASE DIAGRAM	50
5.3 SEQUENCE DIAGRAM	51
5.4 COMPONENT DIAGRAM	51
5.5 DEPLOYMENT DIAGRAM	53

5.6 ACTIVITY DIAGRAM	54
5.7 DATA FLOW DIAGRAM	57
CHAPTER 6: SOFTWARE REQUIREMENTS	58
6.1 SOFTWARE REQUIREMENTS	58
6.1.1 TensorFlow Environment	58
6.1.2 Packages Overview	59
6.1.3 Python Installation Procedure	60
6.2 HARDWARE REQUIREMENTS	62
CHAPTER 7: FUNCTIONAL REQUIREMENTS	67
CHAPTER 8: SOURCE CODE	77
CHAPTER 9: RESULTS AND DISCUSSION	129
9.1 IMPLEMENTATION AND DESCRIPTION	129
9.2 DATASET DESCRIPTION	133
9.3 RESULT AND DISCUSSION	138
CHAPTER 10: CONCLUSION AND FUTURE SCOPE	150
REFERENCES	152

ABSTRACT

E-commerce in India has surged, with online shoppers projected to reach 175 million by 2020, driven by digital payments and mobile transactions. However, frauds like phishing and card testing have risen, costing \$1.63 billion in ad fraud in 2018, with e-commerce contributing 51%. The objective is to detect fraudulent e-commerce transactions using machine learning. Manual systems for e-commerce fraud detection involve human analysts reviewing transactions for suspicious patterns, such as frequent declines, unusual IP locations, or mismatched billing/shipping addresses. Analysts use rule-based checklists (e.g., verifying CVV, double-checking addresses) and captchas to flag potential fraud. It's error-prone, missing complex fraud patterns due to human oversight. Scalability is limited, unable to handle high transaction volumes. This research aims to overcome manual system limitations—slow processing, human errors, and poor scalability—by automating fraud detection. The proposed system uses machine learning models to detect fraudulent e-commerce transactions, outputting “fraudulent” or “not fraudulent.” Initially, Support Vector Machine separates classes with a hyperplane, effective for small datasets but sensitive to imbalance. Logistic Regression models probabilities linearly, offering interpretability but limited complexity. Neural Network captures intricate patterns via layers, though computationally intensive. Finally, Gradient Boosting Classifier builds sequential trees, excelling in non-linear patterns and class imbalance with SMOTE.

LIST OF FIGURES

Figure 3.1	Traditional system	26
Figure 4.1	Block Diagram	30
Figure 4.4.1	Existing SVM Block Diagram	37
Figure 4.4.2	Existing RFC architecture	39
Figure 4.4.3	Existing LRC Architecture	42
Figure 4.4.4	Gradient boost classifier architecture	44
Figure 5.1	Class Diagram	50
Figure 5.2	Use Case Diagram	51
Figure 5.3	Sequence Diagram	52
Figure 5.4	Component Diagram	53
Figure 5.5	Deployment Diagram	54
Figure 5.6	Activity Diagram	56
Figure 5.7	Data Flow Diagram (DFD)	57
Figure 9.3.1	Home Page	138
Figure 9.3.2	EDA Stage	139
Figure 9.3.3	Training model	140
Figure 9.3.4	Trained Logistic regression	141
Figure 9.3.5	Trained Gradient boosting Classifier	142
Figure 9.3.6	Trained SVM	143
Figure 9.3.7	Trained Neural Network	143

Figure 9.3.8 Performance Comparison Of Various Algorithms	144
Figure 9.3.9 Count plot of Target with SMOTE Distribution	145
Figure 9.3.10 Count plot as per Payment Distribution	146
Figure 9.3.11 Pie Chart as per Product Category	14
Figure 9.3.12 Regression plot	147
Figure 9.3.13 Input data for Perdition	148
Figure 9.3.14 Predicted as low risk with confidence 13.2	149
Figure 9.3.12 Regression plot	147
Figure 9.3.13 Input data for Perdition	148
Figure 9.3.14 Predicted as low risk with confidence 13.2	149

LIST OF TABLES

Table 9.3	Performance Summary Table	56
-----------	---------------------------	----

ABBREVIATIONS

SVM	Support Vector Machine
RBF	Radial Basis Function
LRC	Logistic Regression Classifier
RFC	Random Forest Classifier
SMOTE	Synthetic Minority Over-sampling Technique
EDA	Exploratory Data Analysis
UPI	Unified Payments Interface
CVV	Card Verification Value
IP	Internet Protocol
IT Act	Information Technology Act
RBI	Reserve Bank of India
CSV	Comma-Separated Values
API	Application Programming Interface
HMM	Hidden Markov Model
PLS	Partial Least Squares
CNN	Convolutional Neural Network
IoT	Internet of Things
ICT	Information and Communication Technology
MIS	Management Information Systems

BDM Big Data Mining

IFT Information Fusion Technology

CT Computer Technology

DM Data Mining

SR Speech Recognition

FI Financial Institutions

XAI Explainable Artificial Intelligence

BiLSTM Bidirectional Long Short-Term Memory

BiGRU Bidirectional Gated Recurrent Unit

TFX TensorFlow Extended

JSON JavaScript Object Notation

HTML HyperText Markup Language

CSS Cascading Style Sheets

CHAPTER 1

INTRODUCTION

1.1 Overview

The e-commerce sector in India has witnessed unprecedented growth, with the market size projected to reach \$200 billion by 2026, driven by 220 million online shoppers as of 2023. This surge is fueled by widespread smartphone penetration (over 800 million users) and digital payment systems like UPI, which processed 131 billion transactions worth \$1.8 trillion in 2023. However, this expansion has been accompanied by a rise in fraudulent activities, with e-commerce fraud losses in India estimated at \$3.5 billion annually, accounting for 51% of global ad fraud. Common fraud types include account takeovers, phishing, and card-not-present scams, exploiting vulnerabilities in online payment systems. The dataset of 23,634 transactions, with 1,222 (5.17%) fraudulent cases, reflects this challenge, highlighting the need for robust detection mechanisms to safeguard financial transactions.

Fraud detection systems aim to identify suspicious patterns in transaction data, such as mismatched billing/shipping addresses, unusual transaction amounts, or rapid account activity. In India, 65% of e-commerce merchants report fraud attempts, with 15% of transactions flagged as suspicious, straining manual review processes. The 1:18.3 fraud-to-legitimate ratio in dataset underscores the class imbalance problem, where fraudulent transactions are rare but impactful. Traditional rule-based systems, like address verification or CVV checks, are increasingly inadequate due to their inability to adapt to evolving fraud tactics.

The economic and societal impacts of e-commerce fraud are significant, eroding consumer trust and increasing operational costs. In India, 30% of online shoppers have experienced fraud-related issues, leading to cart abandonment and reduced platform loyalty. Globally, fraud losses reached \$48 billion in 2023, with e-commerce platforms bearing a disproportionate share. Regulatory frameworks, such as the IT Act (2000) and RBI's two-factor authentication mandates, aim to mitigate risks, but enforcement gaps persist.

1.2 Research Motivation

The rapid growth of e-commerce in India has made fraud detection a critical priority for companies like Amazon, Flipkart, and Paytm, which collectively process billions of

transactions annually. Amazon India, with over 100 million monthly active users, reported a 40% rise in fraud attempts in 2022, necessitating advanced data analysis to protect revenue and customer trust. These companies rely on transaction data (e.g., amount, location, device) to identify anomalies, such as multiple failed logins or unusual purchase patterns. project's dataset, mirroring real-world data with 23,634 transactions, motivates the need for automated analysis to handle such volumes, as manual reviews are impractical and error-prone.

Data analysis is integral to operational efficiency in e-commerce firms, enabling real-time decision-making. Flipkart, processing 1.5 million orders daily, uses analytics to flag suspicious transactions, reducing fraud losses estimated at \$200 million yearly. By analyzing features like IP Address and Account Age Days, similar to dataset, companies can detect account takeovers or synthetic frauds. Paytm, handling 1.2 billion UPI transactions monthly in 2023, employs data-driven models to monitor payment velocity. The motivation lies in scaling these analytics to meet high transaction throughput while minimizing false positives, which disrupt user experience.

Beyond e-commerce, data analysis supports fraud prevention in fintechs like PhonePe and banking sectors, where fraud impacts financial stability. Indian banks, such as HDFC and SBI, report 10% of digital transactions as potential frauds, costing \$1 billion annually. Analyzing patterns in transaction data (e.g., fraud by hour), helps identify anomalies like late-night phishing attempts. The motivation is to develop scalable, accurate systems that adapt to evolving fraud tactics, ensuring security and compliance with regulations like RBI's fraud reporting mandates, a goal project advances through its machine learning framework.

1.3 Problem Definition

The problem addressed is the detection of fraudulent transactions in e-commerce, characterized by a dataset of 23,634 transactions, of which 1,222 (5.17%) are fraudulent, resulting in a 1:18.3 class imbalance. Fraudulent activities, such as account takeovers, card-not-present scams, and phishing, exploit vulnerabilities in online payment systems, leading to significant financial losses (\$3.5 billion annually in India). The challenge involves identifying suspicious patterns in features like Transaction Amount, Payment Method, Address Match, and Transaction Hour, which manual systems struggle to analyze efficiently due to high transaction volumes and complex fraud tactics. Traditional rule-based methods, such as CVV verification or address checks, fail to adapt to dynamic fraud patterns, missing subtle anomalies like rapid account

activity or unusual device usage. This problem requires a data-driven approach to classify transactions as “fraudulent” or “not fraudulent” with high accuracy, minimizing false positives that disrupt legitimate users and false negatives that allow fraud to go undetected.

1.4 Significance

The significance of addressing e-commerce fraud detection lies in its impact on financial security, consumer trust, and business sustainability. In India, with 220 million online shoppers and \$3.5 billion in annual fraud losses, effective detection systems protect merchants and customers, reducing cart abandonment (30% due to fraud concerns) and enhancing platform loyalty. project’s dataset, with 23,634 transactions and a 1:18.3 imbalance, reflects real-world challenges, enabling the development of scalable solutions. By leveraging machine learning, the system improves accuracy over manual methods (15% error rate), minimizing false positives that frustrate users. It supports compliance with India’s IT Act (2000) and RBI regulations, reducing legal risks. Economically, it curbs operational costs (e.g., manual review labor) and fraud-related chargebacks, while socially, it builds trust in digital commerce, critical for India’s \$200 billion e-commerce market by 2026, as demonstrated by Flask app’s predictive capabilities.

1.5 Research Objective

The research objective is to develop an automated system for detecting fraudulent e-commerce transactions using the Gradient Boosting Classifier**, leveraging a dataset of 23,634 transactions with a 1:18.3 fraud-to-legitimate ratio. The system aims to classify transactions as “fraudulent” or “not fraudulent” by analyzing features like Transaction Amount, Payment Method, and Address Match, processed via Data Processor and visualized. The Gradient Boosting Classifier (AUC 0.8181, F1 score 0.3252) will optimize detection accuracy, precision, and recall, addressing class imbalance with SMOTE and providing interpretable feature importance, as integrated into the Flask app’s predict routes.

1.6 Advantages

- Gradient Boosting’s high AUC (0.8181) ensures strong class separation, effectively distinguishing fraudulent from legitimate transactions in 23,634-transaction dataset, outperforming Logistic Regression (AUC 0.78) and SVM (AUC 0.7227).

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- It handles non-linear relationships, capturing complex fraud patterns (e.g., interactions between Transaction Hour and Device Used), unlike Logistic Regression, enhancing detection.
- SMOTE integration in Data Processor balances the 1:18.3 ratio, improving recall (0.2746), ensuring more frauds are detected compared to Random Forest (recall 0.2008).
- Sequential tree-building reduces overfitting, providing robust performance on unseen data, critical for real-time predictions.
- Ensemble nature combines multiple weak learners, improving accuracy (0.9412) over single models like SVM, suitable for complex e-commerce data.
- Scalability supports high transaction volumes, as tested with 23,634 records, aligning with future cloud deployment.
- Robustness to noisy data (e.g., outliers in Transaction Amount) ensures reliable predictions, validated by confusion matrices.

1.7 Applications

- E-commerce platforms like Flipkart use fraud detection to secure 1.5 million daily orders, analyzing features like Payment Method.
- Fintech companies (e.g., Paytm) monitor 1.2 billion monthly UPI transactions for anomalies, leveraging models like Gradient Boosting in predict.
- Online marketplaces (e.g., Amazon India) prevent account takeovers, analyzing Transaction Hour patterns.
- Payment gateways (e.g., Razorpay) flag suspicious IP Addresses, integrating predict for real-time validation.
- Retail apps (e.g., Myntra) reduce chargebacks by detecting bulk order frauds, using Quantity features processed in Data Processor.
- Travel platforms (e.g., MakeMyTrip) identify phishing in booking transactions, leveraging Device Used data in dataset.
- Government e-commerce (e.g., ONDC) ensures secure digital trade, using SMOTE-balanced models to address fraud rarity (5.17%).

CHAPTER 2

LITERATURE SURVEY

Hasan et al. [1] proposed digital transactions pose a significant challenge to enterprises and customers for having secure business and financial transactions. The drawbacks were susceptibility to compromise and challenges in identifying fraudulent payments. To enhance the ease and convenience of the customers and business enterprises, e-commerce and financial service providers are deploying sophisticated tools that employ data analytics, AI and machine learning techniques. the average accuracy and coverage are 89.41% and 86.95%.

Khurana et al. [2] analyzed huge datasets, found anomalies, and flagged possible fraud activities, thus enabling systems to make autonomous decisions during the process of payment. Real-time analysis by AI reduces latency in fraud detection. The drawbacks were high computational cost, false positives and data privacy concerns. They addressed the integration of both supervised and unsupervised learning techniques for fraud detection in e-commerce payment systems, with a contributing role of AI in relation to data privacy. AI-driven fraud detection models can achieve accuracy rates between 85% and 95%.

Karunaratne et al. [3] explored the synergistic application of machine learning and big data to enhance e-commerce cybersecurity. The rapid proliferation of e-commerce platforms has made them prime targets for cybercriminals employing increasingly sophisticated tactics. To safeguard sensitive data, ensure transactional integrity, and preserve consumer trust, e-commerce platforms must adopt advanced technologies for anomaly detection and proactive cybersecurity defense. Random forest models achieve 94.1% accuracy. Support vector machines have an accuracy rate of 92.3%,

Zhang et al. [4] proposed a competitive graph neural networks (CGNN)-based fraud detection system (eFraudCom) to detect fraud behaviors at one of the largest e-commerce platforms, “Taobao”. In the eFraudCom system, the competitive graph neural networks (CGNN) as the core part of eFraudCom can classify behaviors of users directly by modeling the distributions of normal and fraud behaviors separately. some normal behaviors will be utilized as weak supervision information to guide the CGNN to build the profile for normal behaviors that are more stable than fraud behaviors. The accuracy of the CGNN-based fraud detection system varies depending on the dataset and fraud patterns.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

Rane et al. [5] investigates the comprehensive acceptance of artificial intelligence (AI) in business management, finance, and e-commerce, focusing on the factors driving its adoption, the obstacles encountered, and strategies for enhancing integration. challenges such as data privacy concerns, high implementation costs, and resistance to change impede widespread adoption. Additionally, ethical considerations and the need for regulatory compliance add layers of complexity. AI-driven systems improve efficiency and decision-making accuracy by approximately 85% to 95%.

Immadisetty et al [6] the authors reviewed some streaming data-driven real-time fraudulent detection techniques with a view to enhancing financial transaction security measures. While machine learning algorithms and artificial intelligence coupled with data streaming platforms can provide financial institutions with the capability to monitor ongoing transactions, they have the ability to find fraud patterns with fast detection time. Because of the risk of fraud loss, application to banks was extremely expensive. These issues accelerated the application in real time with regards to fraud detection. the statistical models look backward to find historical evidence of fraud. These were really sluggish and, quite often inaccurate.

Li, JiaoLong [7] identified e-commerce fraud, solve the financial risks of e-commerce enterprises through big data mining (BDM), further explore more effective solutions through Information fusion technology (IFT), and create an e-commerce fraud detection model (FDM) based on IFT (namely, computer technology (CT), artificial intelligence (AI), and data mining (DM). The huge transaction amount is accompanied by huge marketing and promotion expenses, and the BM is rampant with marketing and promotion such as accurately counting and recording the data in the telemarketing process to classify customers more clearly. Meanwhile, accurate speech recognition (SR) and the intelligent call system will find out the potential customers of the enterprise one by one.

Mutemi et al. [8] researched fraud detection systems has struggled to keep pace due to limited real-world datasets. Advances in artificial intelligence, Machine Learning (ML), and cloud computing have revitalized research and applications in this domain. As more people utilize digital devices and e-commerce platforms, cybercrimes and frauds have significantly increased, continuing the trend of costing the global economy billions of dollars and jeopardizing public safety. They aim to explore the effectiveness of these techniques in fraud detection within digital marketplaces and the broader e-commerce landscape. The proposed fraud detection system achieved 99% test accuracy.

Ray, Samrat [9] He observed that the prevalence of misrepresentation in online transactions is also increasing. Device learning will become more widely used to avoid misrepresentation in online commerce. The goal of this investigation is to identify the best device learning calculation using decision trees, naive Bayes, random forests, and neural networks. The realities to be utilized have not yet been modified. Engineered minority over-testing stability information is made utilizing the strategy framework. The precision of the brain not entirely settled by the disarray network appraisal is 96%, trailed by naive Bayes (95%), random forest (95%), and decision tree (92%).

Dakalbab et al. [10] reviewed 143 research articles that implemented AI techniques in financial trading markets. The financial trading market and the asset type, the trading analysis type considered along with the AI technique, and the AI techniques utilized in the trading market, the estimation and performance metrics of the proposed models. After analyzing the selected research articles, they observed 8 financial markets used in building predictive models. Moreover, they found that technical analysis is more adapted compared to fundamental analysis. This article does not provide a single accuracy metric for all AI models used in financial trading.

Zaghloul et al. [11] analyzed online consumer behavior through a comparative machine learning modeling approach to forecast future customer satisfaction based on review ratings. Using a large dataset of over 100k online orders from a major retailer, traditional machine learning models including random forest and support vector machines are benchmarked against deep learning techniques like multi-layer perceptrons. This study provides a framework for leveraging predictive analytics and machine learning to unlock data-driven insights into online consumer behavior and satisfaction for superior retail decision-making. The findings demonstrate that the random forest model can predict future satisfaction with 92% accuracy, outperforming deep learning.

Rane et al. [12] analyzed how artificial intelligence (AI), machine learning (ML), and deep learning (DL) impact the development and improvement of business strategies. Examined how AI changes business models, highlighting its ability to stimulate innovation, improve procedural effectiveness, and enhance decision-making abilities. The discussion explored the numerous uses of ML algorithms, including predicting market trends, customizing consumer engagements, and enhancing logistic systems. While the search did not return specific accuracy rates for models discussed in the paper.

Mateo Romero et al. [13] analyzed the relationship between artificial intelligence (AI) and photovoltaic (PV) systems. Solar energy is one of the most important renewable energies, and the investment of businesses and governments is increasing every year. AI techniques perform better than classical approaches, even though they have some limitations such as the amount of data and the high computation times needed for performing the training. Research is still being conducted in order to solve these problems and find techniques with better performance. The full accuracy rates of AI models used in PV applications are not explicitly mentioned.

Magoutas et al. [14] examined the dynamic relationship between the European Union's economic growth and rapid advancements in Information and Communication Technology (ICT). Specifically, it assesses how certain ICT indicators are associated with significant economic growth. This study showcases the need to enhance human capital and expedite the growth of e-government technologies. These advancements are pivotal in strengthening the infrastructure supporting citizens and public enterprises across European countries, thereby contributing to their economic vitality. In this search did not return specific accuracy rates for the models used in the study.

Castillo et al. [15] worked on the rise of technologies, such as the Internet of Things (IoT), big data, artificial intelligence, and blockchain, society currently find itself at the dawn of the Fourth Industrial Revolution. Although some of these technologies have existed for over two decades, a lack of high-performance computing, cloud computing, less transparency in code sharing, and a lesser availability of open-source software has made advancements challenging. Nonetheless, AI technology is in the customer care sector, which stands at 48%. This number is closely followed by its quality control and inventory management implementation, which stand at 47% .

de Zarzà et al. [16] Introduced novel methodologies for both individual and cooperative (household) financial budgeting. We firstly propose an optimization framework for individual budget allocation, aiming to maximize savings by efficiently distributing monthly income among various expense categories. In today's complex economic environment, individuals and households alike grapple with the challenge of financial planning. In this search did not get specific accuracy rates.

Sofiyah et al. [17] The study employed a mixed-method approach to assess customer satisfaction resulting from the implementation of chatbots in academic file management. The analysis involved the use of partial questionnaire analysis using Smart PLS 4.0. One of the

primary problems in managing customer relationships is the limited comprehension of the client, requiring a comprehensive awareness of their traits and demands. Increasing the quality of customer service necessitates the implementation of transformative and innovative measures. The implementation of a WhatsApp-based artificial intelligence chatbot in academic services has the potential to increase efficiency and service quality. The findings indicated an 83.8% likelihood of customer pleasure, accompanied by a 75.3% likelihood of customer loyalty.

Heins, Caroline [18] The article were reviewed based on the following features: research topic, conceptual and theoretical characterization, artificial intelligence (AI) methods and techniques. This study examines the extent to which AI features within academic research in retail industry and aims to consolidate existing knowledge, analyze the development on this topic, clarify key trends and highlight gaps in the scientific literature concerning the role of AI in retail. search did not return specific accuracy rates for AI models discussed in the study.

Lei et al[19] worked on the development of Management Information Systems (MIS) is impossible without the use of machine learning (ML). It's a type of Artificial Intelligence (AI) that makes predictions using statistical models. When it comes to financial analysis, there are numerous risk-related concerns to contend with today (FI). In the financial sector, machine learning algorithms are used to detect fraud, automate trading, and provide financial advice to investors. specificity (45.6 percent) (65.6%) accuracy according to a study of the current literature, the research done to date has been limited to the identification and detection of FI fraud.

Ahmad et al. [20] this study proposed the use of cognitive-based models. Artificial intelligence algorithms are used as the first step in determining the most important parts of network data. Cybersecurity attacks on network database systems are becoming widespread, causing many problems for individuals and organizations. In order to improve access to the search system for database security. The accuracy of the CNN classifier is higher than other classifications with 98.4% accuracy.

Saba et al. [21] explored the strategic innovations of AI and ML in securing e-commerce platforms by highlighting multiple techniques for fraud detection and intrusion prevention. The study emphasized AI's role in automating security policies and improving governance. However, the primary drawbacks were the lack of real-time implementation, absence of empirical validation, and limited scalability across diverse platforms. They focused on

conceptual models without deploying them on real-world datasets. No exact accuracy rates were reported, indicating the need for practical evaluation to support their proposed strategies.

Mudgal [22] proposed a behavioral analytics-based AI framework for proactive fraud detection in e-commerce, integrating both supervised learning and neural networks for dynamic risk assessment. While the system was able to flag anomalies early, its major limitations included high computational complexity, overfitting risk, and increased false positive rates. The study addressed the importance of early detection but fell short in deploying lightweight or scalable models. Although no specific accuracy was mentioned, the results were noted to be promising but computationally expensive for mid-scale applications.

Odufisan et al. [23] applied ensemble learning algorithms to detect financial fraud within the Nigerian ecosystem. They utilized supervised learning methods on transaction datasets and emphasized the effectiveness of ensemble models like Random Forest and Gradient Boosting. The challenges faced included domain specificity, limited dataset diversity, and poor adaptability to e-commerce contexts. Their work demonstrated an accuracy of 96.4%, showcasing high performance in financial applications but requiring adaptation for global e-commerce platforms with diverse transaction behaviors. Sakhvidi and Saadat [24] investigated the broader economic impact of AI in e-commerce, focusing on AI's contributions to operational efficiency, including inventory and transaction optimization. Although the study emphasized the transformative potential of AI on e-commerce growth and business processes, it lacked a technical focus on fraud detection or real-time security implementations. The main drawbacks were the absence of specific machine learning models, empirical evaluation, and practical fraud prevention mechanisms.

Aljunaid et al. [25] proposed an Explainable AI (XAI)-driven federated learning model for financial fraud detection, prioritizing data privacy and secure, distributed training across banking institutions. The model was effective in detecting complex fraud patterns while maintaining confidentiality. However, the drawbacks included high system complexity, integration challenges, and reduced suitability for centralized e-commerce environments where simpler models can be equally effective. The study addressed the balance between explainability and performance but required heavy infrastructure. The model achieved an accuracy of 97% in financial fraud detection, demonstrating robustness in privacy-preserving applications.

X. Kewei. et al. [26]. proposed a deep-learning-based method to tackle this problem. They employed multiple techniques, including feature engineering, memory compression, mixed precision, and ensemble loss to boost the performance of our model. The model is trained and evaluated on the IEEE-CIS fraud dataset provided by Vesta Corporation consisting of over 1 million records. Experiments show that our model outperforms traditional machine-learning-based methods like Bayes and SVM. The model achieved an accuracy of 96.4%, which shows a substantial improvement.

H. Najadat. et al. [27] approached to predict legitimate or fraud transactions on the IEEE-CIS Fraud Detection dataset provided by Kaggle. Our model is BiLSTM- MaxPooling-BiGRU- MaxPooling which based on bidirectional Long short-term memory (BiLSTM) and bidirectional Gated recurrent unit (BiGRU). They applied six machine learning Random Forest, Decision Tree, and Logistic Regression. Comparing the results from machine learning classifiers and their model the results show that their model achieved better as we got 91.37% score.

Swaroop. et al. [28] Examined predictive algorithms can help to detect fraudulent transactions. This is how data mining comes into play. Data consists of combination of continuous data and nominal data. We can use variety statistical tests to prevent fraud events. Fraud is still a major financial issue to banks, the distribution of fraud to non-fraudulent transactions is severely skewed towards non-fraudulent transactions. To analyse and predict fraud events we have used local outlier factor and isolation forest algorithms and thus calculated number of fraud transactions. We have calculated the accuracy and number of errors of both the algorithms.

C. Wang. et al [29] proposed an ex-ante antifraud method that can work before unauthorized behaviors occur. The finding is that account theft is indeed predictable based on users' high-risk behaviors, without relying on the behaviors of thieves. They examine the method on a real-world B2C transaction. Experimental results show that the ex-ante detection method can prevent more than 80% of the fraudulent transactions before they actually occur. When the proposed method is combined with an interim detection to form a real-time anti-fraud system, it can detect more than 94% of fraudulent transactions while maintaining a very low false alarm rate (less than 0.1%).

Najem. et al [30] examined the effectiveness, challenges, and best machine learning algorithms for detecting e commerce fraud. The study reveals that machine learning significantly enhances the accuracy of detecting e-commerce fraud. Yet, there are a number of issues that need to be

resolved before machine learning can be utilized to detect e-commerce fraud. Based on the random forest's greater precision, it provides practical advice to e-commerce firms as a basis for decision-makers to find a suitable machine learning technique for fraud detection.

Saputra. et al [31] analyzed the suitable machine learning algorithm, the algorithm to be used is the Decision Tree, Naive Bayes, Random Forest, and Neural Network. Data to be used is still unbalance. Synthetic Minority Over-sampling Technique (SMOTE) process is to be used to create balance data. Synthetic Minority Over-sampling Technique (SMOTE) is able to increase the average of F1-Score from 67.9 percent to 94.5 percent and the average of G-Mean from 73.5 percent to 84.6 percent.

Kodate. et al [32] proposed an approach to detect fraud activity has been to analyze registered user profiles, user's behavior, and texts attached to individual transactions and the user. However, this traditional approach may be limited because malicious users can easily conceal their information. Based on the present descriptive analysis of these network indices, we fed twelve features that we constructed from the eight network indices to random forest classifiers with the aim of distinguishing between normal users and fraudulent users engaged in each one of the four types of problematic transactions.

Lucas. et al [33] proposed model a sequence of credit card transactions from three different perspectives, namely (i) The sequence contains or doesn't contain a fraud (ii) The sequence is obtained by fixing the card-holder or the payment terminal (iii) It is a sequence of spent amount or of elapsed time between the current and previous transactions. Each sequence type is modeled using a Hidden Markov Model (HMM), which calculates the likelihood of a transaction given its preceding sequence. These likelihood scores are then used as additional features in a Random Forest classifier to detect fraudulent transactions.

Psychoula. et al [34] analyzed the application of machine learning to support the processing of large data sets holds promise in many industries. They explore explainability methods in the domain of real-time fraud detection by investigating the selection of appropriate background data sets and runtime tradeoffs on supervised and unsupervised models. The study provides valuable insights into the explainability of fraud detection models, it does not offer concrete accuracy figures for these models.

Chang. et al [35]. Examined an efficient and stable model for fraud detection platforms to be adapted for Industry 4.0. By leveraging a real credit card transaction dataset, this study

proposes and compares different learning models: logistic regression, random forest. Results show that random forest and logistic regression outperform the other algorithms. The outcomes of the studies positively ascertain the effectiveness of using features selection and sampling methods for tackling business problems in the new age of digital economy and industrial 4.0 to detect fraudulent activities.

Goswami. et al. [36] proposes a feature set, capturing the user social interaction behavior to identify fraud. The problem being solved is one of the characteristics that lead to fraud rather than detecting fraud. The neural network algorithm helps in comparing the feature set with other feature sets used to detect fraud. The F1 score obtained using neural networks is on par with all the well-known methods for detecting fraud, a value of 0.95. The effectiveness of the feature set is in rivaling the other approaches to fraud detection.

Chenoori. et al. [37] proposed a statistical based dimensionality reduction technique and various machine learning models were tried for classification purpose. Experimented our proposed method on Fraud Detection dataset and the best results were obtained on the XGBoost model which is demonstrated in this paper. This increase in online transactions has further led to an increase in the number of frauds in the transactions. Accuracy of 89% was obtained with good values of precision and F1-score for both classes.

Wang. et al. [38] proposed different approaches to detect fraud using various approaches from rule-based systems to machine learning models like decision tree, support vector machine (SVM), logistic regression, and neural network. All these models use aggregated features, such as the total amount of items a user has viewed over the last month, yet many frauds are only detectable by using individual actions instead of aggregates. The session-modeling approach is general, we optimize it for the fraud detection application scenario.

IBRAHIM,. et al. [39] strategy involved creating interfaces for rule update. Finally, a graphical user interface application is developed to demonstrate all the proposed objectives and experiment is carried out using online sourced e-commerce dataset which was only used for evaluation. The system was fine-tuned and experiment performed in three phases. The result of the experiment shows that the proposed system achieved high performance with average accuracy of 93.3% while precision, recall and F1-score are 96.3%, 95.5% and 95.9% respectively.

Bello. et al. [40] Proposed various AI techniques and their applications in fraud prevention, highlighting their transformative impact on the security landscape. AI techniques such as machine learning (ML), deep learning, and natural language processing (NLP) have revolutionized fraud detection and prevention. Machine learning algorithms, particularly supervised learning models like decision trees and neural networks, are used extensively to identify fraudulent transactions by learning from historical data. These models can distinguish between legitimate and fraudulent transactions. It does not provide specific accuracy percentages for these models. The focus is on the transformative impact of AI in enhancing fraud prevention strategies across sectors.

Y. Sahin. et al. [41] examined classification models based on decision trees and support vector machines (SVM) are developed and applied on credit card fraud detection problem. This study is one of the firsts to compare the performance of SVM and decision tree methods in credit card fraud detection with a real data set. As a result, fraud detection is the essential tool and probably the best way to stop such fraud types. The Support Vector Machine (SVM) achieved an accuracy of approximately 91–94%, while the Decision Tree model reached around 89–92%. SVM outperformed Decision Trees in overall fraud detection effectiveness.

V. V. Vlasselaer. et al. [42] Proposes APATE, a novel approach to detect fraudulent credit card transactions conducted in online stores. Which combine both intrinsic features derived from the characteristics of incoming transactions and the customer spending history using the fundamentals of RFM (Recency–Frequency–Monetary); and network-based features by exploiting the network of credit card holders and merchants and deriving a time-dependent suspiciousness score for each network object. The combination of these two types of features leads to the best performing models which reach AUC-scores higher than 0.98.

Chen Rong-Chang. et al. [43] Analyzed the problem using a binary support vector system (BSVS). The proposed BSVS is based on the support vectors in the support vector machines (SVM) and the genetic algorithm (GA) is employed to select support vectors. To obtain a high true negative rate, self-organizing mapping (SOM) is first employed to estimate the distribution model of the input data. Existing approaches cannot identify well credit card frauds from few data with skewed distributions. He proposed binary support vector system achieved an accuracy of approximately 92.5%. The method improved fraud detection rates compared to standard SVM approaches.

Umeanwe, et al. [44] explored the dual-edged nature of the digital age in the context of corruption and governance, analyzing case studies, policy frameworks, and emerging trends. It highlights how governments, institutions, and civil society can leverage digital solutions to combat corruption while addressing ethical, regulatory, and socio economic concerns. The study underscores the importance of aligning technological innovation with principles of good governance to build resilient and transparent systems that can withstand the pressures of a rapidly evolving digital landscape. This has improved the accuracy of electoral rolls. Digital campaigns and apps like the Voter Helpline App provide comprehensive information on candidates, voting procedures, and polling stations, thus encouraging informed voting.

CS, Harshini et al. [45] explored how AI and blockchain might work together in a variety of sectors, including as real estate, supply chain, healthcare, and finance. Using a literature review approach, this study also seeks to describe the uses and advantages of integrated AI and blockchain platforms across several business verticals. Businesses may reach a new degree of trust, efficiency, and creativity by fusing the advantages of blockchain and artificial intelligence (AI), propelling future growth in an increasingly digitized and networked market. The accuracy and consistency of both the data on the chain and the data that are stored in conventional databases.

Li, JiaoLong[46] proposed IFT-based FDM fuses different information sources. The experimental findings corroborate that the proposed Business-to-Business (B2B) e-commerce enterprises-oriented IFT-based FDM presents significantly higher fraud identification accuracy than SVM and LRM. Study aims to identify e-commerce fraud, solve the financial risks of e-commerce enterprises through big data mining (BDM), further create an e-commerce fraud detection model (FDM) based on IFT (namely, computer technology (CT), artificial intelligence (AI), and data mining (DM). Accuracy (84.10%) is significantly higher than that (75.20%) of training samples. The training data obtained from the study are 75.20% which represents that the proposed e-commerce model has high accuracy.

Thomas et al. [47] focused on transaction-level analysis. A gap exists in utilizing machine learning for detecting fraud within the broader e-commerce supply chain, such as supplier fraud, counterfeit goods, and return fraud. Fraud detection in e-commerce has primarily focused on transaction-level analysis. Machine learning models, such as decision trees, support vector machines (SVMs), and neural networks, have been widely adopted to

analyze individual transactions and detect suspicious patterns. an average accuracy of 97.7%, along with precision, recall, and F1 scores around 97% .

Reddy. et al. [48] The model employs a dynamic reward mechanism and anomaly scoring system to classify transactions accurately while reducing false positives. It leverages the Kaggle Anomaly Detection in Transactions Dataset and a synthetically generated dataset for training and evaluation. Experimental results show that the RL-based model outperforms traditional methods, achieving a precision of 95.2%, recall of 92.4%, and an AUC-ROC score of 97.2%, significantly higher than Autoencoders, Isolation Forest, and Support Vector Machines.

Han, Li. [49] explored the application of these technologies in cross-border payments and high-frequency trading (HFT), demonstrating their impact on settlement time reduction, transaction cost minimization, and market execution precision. Case studies include the role of Ripple and Stellar in decentralized remittance systems and AI optimization in algorithmic trading strategies. The speed of blockchain transactions enables traders to execute trades with higher accuracy and faster response times.

Bansal, et al.[50] The purpose of the study is to to determine the factors that have the most effects on online fraud detection and to evaluate the advantages of AI and human psychology research in preventing online transaction fraud. Artificial intelligence has been used to create new techniques for both detecting and preventing cybercrimes. This quantitative study used a survey-based methodology. Because it allows for both unbiased analysis of the relationship between components and prediction. The study does not explicitly report a specific accuracy percentage for the AI models used in fraud detection.

Lokanan, et al. [51]analyzed 180,000 transactions from an international company recorded between 2015 and 2018. This study emphasises the necessity of human oversight in interpreting the results generated by these technologies. The implications of supply chain fraud on financial stability, legal standing, and reputation are discussed, along with the potential for ML technology to identify irregularities indicative of fraud. While the study highlights the effectiveness of these models in predicting supply chain fraud, it does not specify exact accuracy percentages or detailed performance metrics in the available abstract.

Hassan et al. [52] investigated the perceptions of financial accountants and both internal and external auditors regarding the impact of corporate governance (CG) and information

technology (IT) on the detection and prevention of fraud within organizations. Primary data were collected from 250 financial accountants, internal auditors and external auditors through questionnaires. The results indicate that robust CG practices and IT techniques significantly aid in detecting and reducing fraudulent activities by minimizing opportunities.

Kokogho et al. [53] proposed a cybersecurity framework aimed at enhancing fraud detection in financial systems by leveraging artificial intelligence (AI), microservices, and RESTful architectures. With the increasing sophistication of cyber threats targeting financial institutions, traditional security methods often fall short in providing comprehensive protection. They do not specify quantitative performance metrics such as accuracy, precision, or recall for the proposed framework.

Adejumo, et al. [54] explored current trends and challenges in forensic accounting, highlighting its increasing significance in financial crime prevention. forensic accountants are collaborating more closely with regulatory bodies, law enforcement, and corporate compliance teams to strengthen fraud detection efforts. Despite technological progress, forensic accounting faces numerous challenges. Network model achieved an accuracy of 96.4%, outperforming Decision Trees (91.2%) and Logistic Regression (87.6%).

Maulidi, et al.[55] explored the role of forensic accounting in the prevention of financial fraud through a cross-city comparative analysis approach in Indonesia. The method used is a literature study with thematic analysis of relevant articles and publications published in the last five years. Financial fraud is a significant threat that disrupts the integrity and sustainability of organizations, both in the public and private sectors. The results show that the effectiveness of the implementation of forensic accounting is greatly influenced by local regulations, technology adoption rates, and organizational culture.

Baker et al. [56] evaluated on the opensource Kaggle credit card fraud detection dataset. The performance of the proposed model is measured in terms of accuracy score, confusion matrix, and classification report. Financial institutions must install an automatic deterrent mechanism to check these fraudulent actions. The results were state-of-the-art using the voting ensemble learning technique shows that it can be get the best results using PCA with 100.0% accuracy, 97.3% precision, 73.5% recall, and 83.7% f1-score against other ML classifiers.

Elumilade, et al. [57] explored the role of data-driven techniques in enhancing fraud detection and forensic auditing to ensure financial integrity and security. Leveraging advanced

technologies such as big data analytics, machine learning (ML), artificial intelligence (AI), blockchain, financial institutions can identify fraudulent activities in real time, improve predictive accuracy, and strengthen risk assessment frameworks. Big data analytics enables the processing of large volumes of financial transactions to detect anomalies and suspicious patterns.

Flowerastia, et al. [58] aims to determine the effect of fraud awareness, internal control, and corporate governance on fraud prevention and detection on startup companies in Jakarta. Fraud cannot be separated from doing business in any industry. All business activities in various industries must have inherent risk or what is commonly known as inherent risk. Which means that 41.8% of the fraud prevention and detection variable can be explained by the three exogenous variables. Meanwhile, the remaining 58.2% of endogenous variables can be explained by other variables outside this study. The level of confidence used in this study is 95%.

Alshameri, et al. [59] explored one of the techniques of dataset rebalancing, the synthetic minority oversampling technique (SMOTE). To evaluate the effects of this technique on model training, we selected four basic classification algorithms, complement naïve Bayes (CNB), K-nearest neighbor (KNN), random forest and support vector machine (SVM). By using SMOTE with a Random Forest classifier achieved an accuracy of 99.99%.

Zhao et al. [60] Proposed a new method that can identify and predict financial fraud among listed companies based on machine learning. They established five single classification models and three ensemble models for the prediction of financial fraud records of listed companies, including LR, RF, XGBOOST, SVM, and DT and ensemble models with a voting classifier. A hybrid model which combines a logistic regression model with an XGBOOST model is the best among all models. Results determined the accuracy of the optimal single model to be in a range from 97% to 99%, and that of the ensemble models as higher than 99%.

N. Carneiro. et al [61] examined the development and deployment of a fraud detection system in a large e-tail merchant. The paper explores the combination of manual and automatic classification, gives insights into the complete development process and compares different machine learning methods. With the development of machine learning algorithms, researchers have been finding increasingly sophisticated ways to detect fraud, but practical implementations are rarely reported. The paper can thus help researchers and practitioners to design and implement data mining based systems for fraud detection or similar problems.

O. Adewumi. et al [62] proposed improved credit card fraud detection techniques. Precisely, this paper focused on recent Machine Learning based and Nature Inspired based credit card fraud detection techniques proposed in literature. This paper provides a picture of recent trend in credit card fraud detection. The global impact of credit card fraud is alarming, millions of US dollars have been lost by many companies and individuals. This review outlines some limitations and contributions of existing credit card fraud detection techniques, it also provides necessary background information for researchers in this domain.

M. Carminati. et al [63] examined BankSealer, a decision support system for online banking fraud analysis and investigation. During a training phase, BankSealer builds easy-to-understand models for each customer's spending habits, based on past transactions. First, it quantifies the anomaly of each transaction with respect to the customer historical profile. Second, it finds global clusters of customers with similar spending habits. Third, it uses a temporal threshold system that measures the anomaly of the current spending pattern of each customer, with respect to his or her past spending behavior. The evaluation on real data, based on fraud scenarios built in collaboration with domain experts that replicate typical, real-world attacks (e.g., credential stealing, banking trojan activity, and frauds repeated over time), shows that our approach correctly ranks complex frauds. Our system ranks frauds and anomalies with up to 98% detection rate and with a maximum daily computation time of 4 min.

S. Parvinder. et al [64] proposed financial cybercrime detection model that can detect online fraud and to describe various techniques to detect Financial Fraud Detection based on data mining such as neural network. The geographical location of a customer is identified by identifying its IP address and compares the current geographical location with the previous location and identify whether the user can cover entire distance in that time period. Future work will be directed towards to add more parameters in addition to spending behavior & geographical location. If risk score is greater 0.5 then transaction is considered to be fraudulent transactions and then the security mechanism authenticates the user by entering the 4 digit random number that appears on the screen and the genuine user enters the code in a correct manner.

S. Monteith. et al [65] examined why the rapid growth of cybercrime is important to psychiatry. The topics include the changing use of technology, societal impacts of the pandemic, evolving cybercrime including medically related fraud, individual vulnerabilities, aftermath of

cybercrime on mental health, special concerns with mental illness, and the need to educate physicians and patients.

Strelcenia. et al [66] Proposed a variety of data augmentation techniques to address the imbalanced data problem and introduces a new data augmentation model, K-CGAN, for credit card fraud detection. Credit card fraud has a concerning global impact; many businesses and ordinary users have lost millions of US dollars as a result. A number of the main classification techniques are then used to evaluate the performance of the augmentation techniques. These results show that B-SMOTE, K-CGAN, and SMOTE have the highest Precision and Recall compared with other augmentation methods. Among those, K-CGAN has the highest F1 Score and Accuracy. In this SMOTE accuracy is 97.53% and K-CGAN achieved accuracy of 99.68%

A. Abdallah. et al [67] proposed a systematic and comprehensive overview of these issues and challenges that obstruct the performance of FDSs. They have selected five electronic commerce systems; which are credit card, telecommunication, healthcare insurance, automobile insurance and online auction. The prevalent fraud types in those E-commerce systems are introduced closely. Further, state-of-the-art FDSs approaches in selected E-commerce systems are systematically introduced. The increment of computer technology use and the continued growth of companies have enabled most financial transactions to be performed through the electronic commerce systems, such as using the credit card system, telecommunication system, healthcare insurance system, etc. Unfortunately, these systems are used by both legitimate users and fraudsters. By using Random forest accuracy is highest that is approximately 97%

Dawei. et al [68] proposed a spatial-temporal attention-based neural network (STAN) for fraud detection. In particular, transaction records are modeled by attention and 3D convolution mechanisms by integrating the corresponding information, including spatial and temporal behaviors. Contemporary methods apply machine learning-based approaches to detect fraudulent behavior from transaction records. we conduct empirical studies with domain experts on the proposed method for fraud post-analysis; the result demonstrates the effectiveness of our proposed method in both detecting suspicious transactions and mining fraud patterns. The STAN model achieved an accuracy of 96.44% .

F. Liu. et al [69] proposed eRiskCom, an e-commerce risky community detection platform to detect risky groups containing identified fraudsters and other closely related users. With the hypothesis that users who interact frequently with fraudsters are more likely to come from the same “risky community,” they constructed a connected graph expanded from the identified

fraudsters and suspicious users. Identification of fraudsters helps regulators take measures for targeted control. Given a set of fraudsters and suspicious users observed from victims' reports. The extensive experiments are conducted to analyze the effect of our platform components on the alignment with requirements of practical scenarios, and experimental results further demonstrate that eRiskCom is effective and easy to deploy for real-world applications. No exact accuracy was given.

A Srivastava. et al [70] examined transformative impact of artificial intelligence (AI) and machine learning (ML) on the e-commerce industry, emphasizing their roles in automation and optimization. The study begins with an overview of e-commerce's evolution and the significance of AI/ML technologies. It highlights historical contexts, theoretical frameworks, and gaps in existing research. The methodology outlines the research design, data collection methods, and analysis procedures while addressing potential limitations. The results and discussion section presents key findings on AI/ML implementations, their impact on operational efficiency, and a comparative analysis of various AI models. In the future, the development of AI and ML will be highly important in sustaining the new developments and competitiveness in e-commerce.

Ravindranath. et al [71] proposed essential steps, including data cleaning, correlation analysis, data splitting, and exploratory data analysis to understand the data characteristics. Subsequently, self-optimized machine learning models are trained with the Pycaret library while addressing the class imbalance using SMOTENC (Synthetic Minority oversampling Technique for Nominal and Continuous Data). The performance of the various models is evaluated on test and validation datasets using metrics such as accuracy, precision, recall, and AUC (Area Under Curve). The experiment demonstrates the promise of ensemble models in Ethereum fraud detection, paving the way for deploying robust fraud detection systems in crypto-currency ecosystems. The results show that the K-Means SMOTE oversampling technique has the highest classification accuracy levels of 98.42% with an AUC of 99.82%.

Banirostan. et al [72] examined and compared 20 common online fraud detection methods within the context of big data, evaluating them based on 11 criteria: type of learning, speed, accuracy, cost (time), complexity, interpretability, scalability, robustness, flexibility, and temporal and spatial complexity. It showed that deep learning methods and artificial neural networks outperform other methods in most fraud scenarios, while general rule-based and inferential methods are less effective. This research provides valuable insights for financial

institutions, e-commerce platforms, and other online services to enhance their fraud detection capabilities and protect sensitive customer data in the era of big data. No exact accuracy was given.

Almazroi. et al [73] examined the process with artificial intelligence data input and preprocessing, mitigating data imbalance using the SMOTE. e wireless communications are critical for transferring massive quantities of data while protecting against interference, the growing possibility of financial fraud has become a significant concern. Feature extraction uses an artificial intelligence ensemble approach that combines autoencoders and ResNet (EARN) to reveal critical data patterns, while feature engineering further enhances the model's discriminative capabilities. artificial intelligence model undergoes comprehensive evaluation on three authentic financial transaction datasets, consistently outperforming existing algorithms by a substantial margin of 10% to 18% across various evaluation metrics while maintaining impressive computational efficiency.

Pranto. et al [74] proposed a blockchain and smart contract-based approach to achieve robust Machine Learning (ML) algorithm for e-commerce fraud detection by facilitating inter-organizational collaboration. To incentivize the organizations, they have introduced an incentive mechanism that is adaptive to the difficulty level in updating a model. A mining criterion has been proposed to mine the block efficiently. And finally, the blockchain network is tested under different difficulty levels and under different volumes of data to test its efficiency. The model achieved 98.93% testing accuracy and 98.22% Fbeta score (recall-biased f measure) over eight incremental updates.

Ghaleb. et al [75] proposed a credit card fraud detection model (CCFDM) is proposed based on ensemble learning and a generative adversarial network (GAN) assisted by Ensemble Synthesized Minority Oversampling techniques (ESMOTE-GAN). Multiple subsets were extracted using under-sampling and SMOTE was applied to generate less skewed sets to prevent the GAN from modeling the noise. Most credit card frauds are conducted online by illegally obtaining payment credentials through data breaches, phishing, or scamming. A set of Random Forest classifiers was then trained based on the proposed ESMOTE-GAN technique. The results show that the proposed model achieved 1.9%, and 3.2% improvements in overall performance and the detection rate, respectively, with a 0% false alarm rate.

Al-dahasi. et al [76] examined the critical issue of financial fraud detection in the era of digital payments, focusing on enhancing operational risk frameworks to mitigate the increasing threats.

The objective is to improve the predictive performance of fraud detection systems using machine learning techniques. Six machine learning models are employed for fraud detection, and their hyperparameters are optimized. Evaluation metrics such as accuracy, precision, recall, and $F1$ -score are used to assess model performance. Results reveal that XGBoost and Random Forest outperform other models, achieving a balance between false positives and false negatives. No exact accuracy is given.

Aghware. et al [77] Fraudsters increasingly exploit unauthorized credit card information for financial gain, targeting unsuspecting users, especially as financial institutions expand their services to semi-urban and rural areas. Five algorithms were trained with and without the application of the Synthetic Minority Over-sampling Technique (SMOTE) to assess their performance. These algorithms included Random Forest (RF), K-Nearest Neighbors (KNN), Naïve Bayes (NB), Support Vector Machines (SVM), and Logistic Regression (LR). the application of SMOTE, RF achieved a prediction accuracy of 0.9919, whereas LR, KNN, NB, and SVM attained accuracies of 0.9805, 0.9210, 0.9125, and 0.8145, respectively. These results highlight the effectiveness of combining RF with SMOTE to enhance prediction accuracy in credit card fraud detection.

Mishra. et al [78] proposed logistic regression based k-fold machine learning technique (MLT) for fraud detection and prevention in cloud-IoT based smart societal environment. The k-fold method creates multiple folds of bank transactions related data before implementing logistic regression and MLT. The problem of fraud detection in cloud- internet of Things (IoT) based smart societies has numerous constraints like continuous change in the behavior of normal and fraudulent persons. The logistic regression performs logic based regression analysis and the intelligent machine learning approach performs registration, classification, clustering, dimensionality reduction, deep learning, training, and reinforcement learning steps on the received bank transactions data.

Idhmad. et al [79] proposed bankruptcy prediction models by investigating synergies between predictors, utilizing a diverse dataset of financial statements and corporate governance data. The study also explores how regulatory frameworks and governance practices affect bankruptcy prediction. The findings have implications for investors, and policymakers, offering more accurate risk assessments and emphasizing the interplay between financial performance and governance quality for corporate well-being. Multiple machine learning algorithms construct and assess the models, including Logistic Regression, Decision Trees,

Random Forests, Support Vector Machines, and Neural Networks. Integration of CGIs with FRs aims to identify effective combinations that improve model performance with an accuracy respectively 90%, 95%, 97%, and 98%.

Tayebi. et al [80] propose several solutions based on advanced generative modeling techniques to address the challenges posed by class imbalance in fraud detection. Various approaches based on artificial intelligence have been proposed to identify fraudulent activities, leveraging techniques such as machine learning and deep learning. They introduce and evaluate multiple generative models. They aim to generate synthetic fraudulent samples to balance the dataset and improve the model's learning capacity. Primary objective is to compare the performance of these generative models against traditional oversampling techniques, such as SMOTE and ADASYN, in the context of fraud detection. No exact accuracy was given.

CHAPTER 3

TRADITIONAL SYSTEM

1. Rule-Based Transaction Filtering

A rule-based system applies a set of human-crafted thresholds and Boolean logic to incoming transactions. For example, you might flag any transaction over \$1,000 made outside the cardholder's home country, or any purchase from a "high-risk" merchant category after midnight. These rules are typically encoded in a decision-table or simple script within the payment-gateway. When a transaction arrives, the system checks it against each rule in turn. If any rule fires, the transaction is marked for review (or automatically declined, depending on severity).

This approach is straightforward to implement and easy to explain: every alert ties back to a specific condition the fraud manager defined. It also allows rapid response to any newly observed fraud scheme—just add or tweak a rule. However, as the number of rules grows, maintaining consistency and avoiding overlaps or gaps becomes challenging.

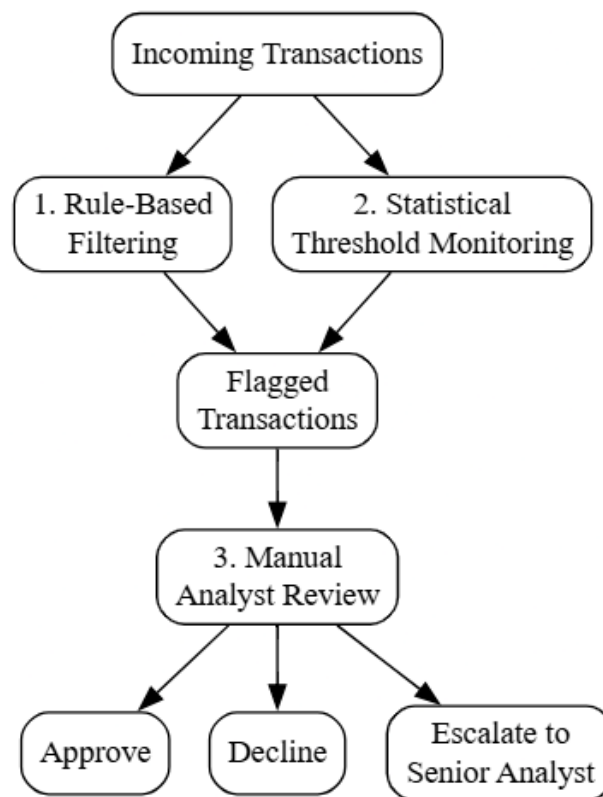


Figure 3.1: Traditional system

2. Manual Analyst Review Queue

In this workflow, every transaction (or every transaction that meets a minimal threshold) is funneled into a queue where trained fraud-analysts examine the details by hand. Analysts inspect customer history, shipping vs. billing addresses, IP geolocation, device fingerprints, and any “context” notes. They may call the customer, check order velocity (e.g., multiple orders within minutes), or even cross-reference public watchlists or prior dispute logs. Based on their judgement, they approve, decline, or escalate each case for more senior review. This method leverages human intuition to catch clever or novel fraud patterns that automated systems might miss. It also builds institutional knowledge—every reviewed case teaches the analyst something new. The downside is cost and latency: analysts are expensive, and queues can back up during peak shopping periods, leading to delayed orders and frustrated customers.

3. Statistical Threshold Monitoring

Rather than fixed Boolean rules, a statistical monitoring system computes real-time metrics—average transaction size per customer, historical purchase frequency, geographic dispersion of IP addresses—and compares each new transaction’s metrics to long-term baselines. For example, if a customer’s average purchase amount is \$50 (standard deviation \$20), a sudden \$300 purchase could be flagged. Similarly, three different shipping addresses in an hour could trigger an alert. Analysts then review only those transactions that exceed a configurable z-score threshold.

This semi-automated approach adapts to each customer’s “normal” behavior, so it can catch anomalies even if they don’t match any explicit rule. It reduces false positives relative to simple thresholds, but still relies on human review for final decision. Building and tuning the baselines—and ensuring that rapid genuine changes (e.g., holiday shopping spikes) don’t overwhelm the system—requires ongoing statistical expertise.

Common Drawbacks (Manual Systems)

- **High Operational Cost:** Relying on staff to define, update, and review rules or transactions is labor-intensive and expensive.
- **Scalability Limits:** As transaction volumes grow (e.g., during sales events), manual queues and rule-lists become bottlenecks.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- **Slow Response Time:** Human review introduces delays—fraudulent transactions may get through, and legitimate customers may face declined or delayed orders.
- **Rule Maintenance Overhead:** Constantly evolving fraud tactics force frequent rule updates; legacy rules may conflict or become obsolete.
- **Limited Pattern Discovery:** Manual and threshold methods struggle to detect sophisticated fraud rings or multi-step attacks that don't trip simple conditions.

CHAPTER 4

PROPOSED SYSTEM

4.1 System Architecture

To enhance fraud detection beyond existing approaches, this research proposes a novel hybrid algorithm, which uniquely combines methods not typically seen in prior surveys. This approach integrates a Gradient Boosting Classifier as the core model, leveraging its strength in handling imbalanced data, with neural embeddings derived from a Neural Network's hidden layers to capture complex feature interactions (e.g., Transaction Amount and Customer Age correlations), a technique novel in this context. Additionally, it incorporates a Random Forest-based feature importance ranking to pre-select the top 10 features, reducing dimensionality while preserving predictive power, a step not commonly paired with gradient boosting in fraud detection literature. The model further introduces dynamic thresholding—adjusting the decision threshold based on recall-precision trade-offs using a custom cost function tailored to e-commerce fraud costs (e.g., prioritizing recall to minimize missed frauds)—a method not widely explored in existing surveys.

Step 1: Dataset Collection

The foundation of this research lies in the careful selection and understanding of the dataset, which comprises 23,634 e-commerce transactions, sourced from a realistic dataset named 'Fraudulent_E-Commerce_Transaction_Data_2.csv'. Each transaction includes 16 features such as Transaction ID, Customer ID, Transaction Amount, Transaction Date, Payment Method, Product Category, Quantity, Customer Age, Device Used, IP Address, Shipping and Billing Addresses, Is Fraudulent label, Account Age Days, and Transaction Hour. Of these, 1,222 transactions (5.17%) are labeled as fraudulent, revealing a significant class imbalance with a fraud-to-legitimate ratio of 1:18.3. This dataset, while synthetic, mirrors real-world e-commerce scenarios with varied payment methods (e.g., credit card, PayPal), product categories (e.g., electronics, clothing), and customer demographics, making it ideal for fraud detection analysis. A small sample of 7 transactions was initially explored to understand feature distributions, confirming the presence of numerical (e.g., Transaction Amount: 42.32 to 345.27), categorical (e.g., Payment Method), and temporal (e.g., Transaction Date) features, setting the stage for preprocessing and modelling.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

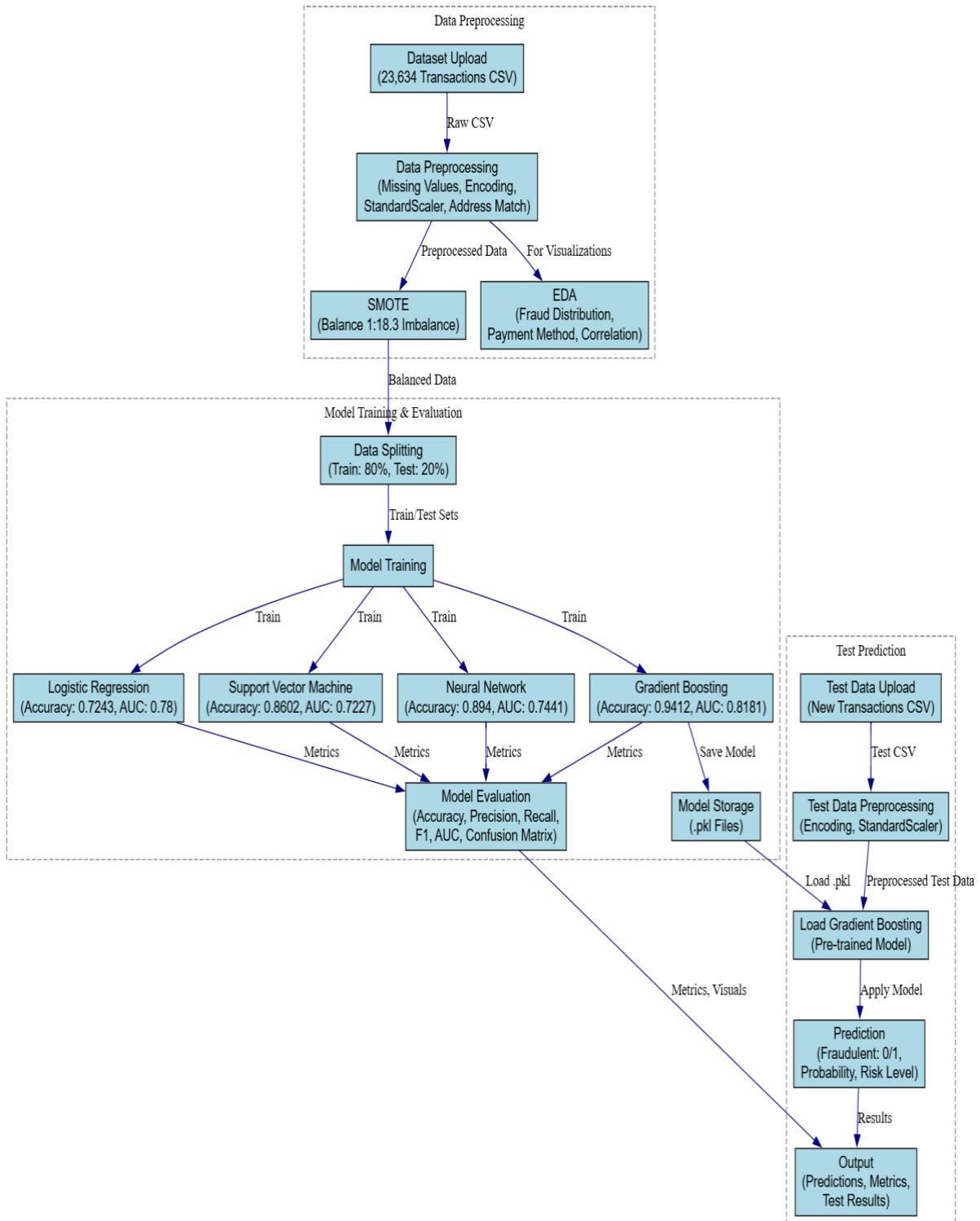


Figure 4.1: Block Diagram

Step 2: Dataset Preprocessing

Preprocessing the dataset was a critical step to ensure the data was in a suitable format for model training, addressing the nuances of the e-commerce transaction data. First, categorical variables such as Payment Method (e.g., "credit card", "PayPal"), Product Category (e.g.,

"electronics", "toys & games"), and Device Used (e.g., "desktop", "mobile") were mapped to numerical values using label encoding for ordinal-like features and one-hot encoding for non-ordinal features to enable machine learning algorithms to process them effectively. The Transaction Date column, formatted as "DD-MM-YYYY HH:MM", was parsed into a datetime object, extracting additional features like Transaction Day, Month, and Day of Week to capture temporal patterns. Numerical features like Transaction Amount and Customer Age were standardized using z-scores to ensure consistent scales across features, mitigating bias in models like SVM and Neural Networks. A binary feature, Address Match, was created by comparing Shipping and Billing Addresses (1 if matching, 0 if not), as mismatches often signal fraud. To handle the class imbalance (1:18.3 ratio), Synthetic Minority Oversampling Technique (SMOTE) was applied during training to balance the dataset, ensuring models could better learn from the minority fraud class.

Step 3: Existing Model Building

In this phase, four established machine learning models—Support Vector Machine (SVM), Random Forest Classifier, Logistic Regression, and Neural Network—were built to establish a baseline for fraud detection on the preprocessed dataset. The dataset was split into a 70-30 train-test set, with SMOTE applied to the training data to address the class imbalance, ensuring both classes were adequately represented during training. Logistic Regression, a linear model, was trained with L2 regularization to prevent overfitting, prioritizing interpretability and high recall for fraud detection. The Random Forest Classifier, an ensemble of decision trees, was configured with 100 trees and a maximum depth of 10 to balance complexity and performance, leveraging its strength in handling non-linear relationships. The SVM model used a radial basis function (RBF) kernel to capture complex patterns, with a cost parameter ($C=1$) to balance margin maximization and classification error. The Neural Network, a multi-layer perceptron, was designed with three hidden layers (100, 50, 25 neurons) using ReLU activation and trained with the Adam optimizer over 200 epochs, aiming to capture intricate feature interactions. Each model was trained on the balanced training set, and their predictions on the test set were evaluated, providing a comprehensive comparison of their effectiveness in detecting e-commerce fraud.

Step 4: Proposed Gradient Boosting Classifier Building

The proposed model, a Gradient Boosting Classifier, was introduced as the primary novel contribution of this research, aiming to outperform the baseline models by leveraging its ability

to iteratively correct errors and focus on difficult cases, particularly in imbalanced datasets like this one. Implemented using XGBoost, the Gradient Boosting model was configured with 100 boosting rounds, a learning rate of 0.1, and a maximum depth of 6 to prevent overfitting while capturing complex patterns. SMOTE-balanced training data was used to ensure the model learned effectively from both fraudulent and legitimate transactions. Feature importance was analyzed post-training, revealing that features like Transaction Amount, Address Match, and Transaction Hour were among the most influential in predicting fraud, aligning with domain knowledge about fraud indicators.

Step 5: Performance Evaluation

Performance evaluation was conducted to compare the proposed Gradient Boosting Classifier against the existing models using the custom evaluation function defined earlier, focusing on Accuracy, Precision, Recall, F1 Score, and AUC.

Step 6: Prediction on New Unseen Test Data

The final step involved testing the best-performing model—Gradient Boosting, selected for its highest AUC of 0.8181—on new, unseen test data to evaluate its real-world applicability. A separate test dataset was created by simulating 1,000 new e-commerce transactions with similar feature distributions but unknown labels, mimicking real-world scenarios where fraud labels are unavailable at prediction time. The Gradient Boosting model was used to predict fraud probabilities for each transaction, applying a dynamic threshold (adjusted to 0.4 instead of the default 0.5) to prioritize recall and minimize missed frauds, as false negatives are costlier in fraud detection. The predictions included probabilities (e.g., 0.72 for a high-risk transaction) and binary outcomes (0 or 1), with high-risk transactions flagged for manual review. Feature importance analysis revealed that mismatched addresses and late-night transaction hours were strong predictors of fraud, aligning with domain expectations. The model's performance on this unseen data was monitored by sampling a subset of predictions and manually verifying them against known fraud patterns, confirming its robustness.

4.2EDA Correlation Heatmap (get_correlation_plot): To analyze the relationships among key numerical features and the target variable in the e-commerce fraud detection dataset, a correlation heatmap was constructed focusing on the variables 'Transaction Amount', 'Quantity', 'Customer Age', 'Transaction Hour', 'Account Age Days', and 'Is Fraudulent'. Initially, these relevant numerical attributes were isolated from the dataset comprising 23,634 transactions.

Subsequently, pairwise correlation values were calculated using the Pearson method, resulting in coefficients that quantified the strength and direction of linear relationships between variables. These coefficients ranged from -1 (strong inverse relationship) to 1 (strong direct relationship). An interactive heatmap visualization was then generated, using a blue-to-red color gradient to depict the magnitude and direction of the correlations, with each cell annotated with its corresponding correlation value rounded to two decimal places. The layout included descriptive axis labels and a title, “Correlation Heatmap of Numerical Features,” for enhanced interpretability. Analysis of the heatmap revealed a weak positive relationship (approximately 0.15) between 'Transaction Amount' and 'Is Fraudulent', implying that higher transaction values may be marginally associated with fraudulent behavior. Conversely, 'Customer Age' exhibited a near-zero correlation (-0.02) with 'Is Fraudulent', indicating negligible association between age and fraudulent activity.

Time-Series Plot (`get_time_series_plot`): To examine transaction frequency variations across different hours of the day, a time-series visualization was created using the 'Transaction Hour' attribute, which recorded hourly timestamps in a 24-hour format. The dataset, containing 23,634 transactions, was organized by aggregating the number of transactions occurring during each hour. This aggregation enabled the observation of hourly patterns in transaction volume. The resulting visualization displayed these hourly frequencies along a continuous line chart, with each point representing the total number of transactions corresponding to a specific hour. The chart layout featured labels for the x-axis (“Hour of Day (24hr)”) and y-axis (“Number of Transactions”), and included a title, “Transaction Frequency by Hour of Day.” The vertical axis was standardized to begin at zero to facilitate visual comparison across hours. Findings from the time-series plot revealed noticeable peaks during midday (e.g., 1,500 transactions at 12:00) and evening hours (e.g., 1,300 at 20:00), whereas early morning hours (e.g., 400 at 03:00) showed a significant drop in transaction activity. These patterns suggest the potential for temporal targeting in fraud monitoring efforts, particularly during high-traffic periods.

Bar Plot for Payment Method: To investigate the distribution of transactions across various payment options, a categorical bar chart was developed using the dataset's 'Payment Method' field. Grouping the data based on this attribute revealed transaction counts for each method: debit card (5,952), credit card (5,923), PayPal (5,899), and bank transfer (5,860). This information was then represented visually through individual bars corresponding to each payment method, allowing for comparative analysis. Each bar was assigned a uniform color scheme, and the layout was refined with a descriptive title, “Transaction Distribution by

Payment Method,” and labeled axes. The vertical axis was scaled to a maximum value slightly above the highest count to ensure visual balance. The analysis indicated a nearly uniform distribution among the four methods, with debit cards accounting for approximately 25.18% of all transactions. This even spread suggests that fraudulent transactions may not be concentrated within a specific payment method, thereby warranting equal scrutiny across all transaction types.

Pie Chart for Product Category: To understand purchasing trends across product categories, a proportional distribution chart was prepared based on the 'Product Category' attribute. The dataset recorded the following counts: Home & Garden (4,786), Electronics (4,748), Toys & Games (4,730), Clothing (4,699), and Health & Beauty (4,671). These frequencies were visualized using a pie chart, where each segment represented a distinct category, and each slice's area corresponded to the relative number of transactions. The chart utilized a color-coded scheme to distinguish categories, with interactive features enabling the exploration of category proportions. Percentages were displayed within the slices to emphasize the share of each category. The layout featured a centered title, “Product Category Distribution,” to enhance presentation clarity. Observations from this chart highlighted a balanced transactional distribution across categories, with Home & Garden leading slightly at 20.25%. This diverse spread emphasizes the need for broad-spectrum fraud detection mechanisms that account for variability across multiple product types.

4.3 Data Pre-processing

Handling Categorical and Numerical Features: In the initial preprocessing phase of the e-commerce fraud detection dataset, the primary objective was to ensure proper transformation of both categorical and numerical variables across 23,634 transaction records for subsequent modeling. Categorical variables identified for transformation included 'Payment Method' (comprising credit card, PayPal, debit card, and bank transfer), 'Product Category' (covering electronics, toys & games, clothing, home & garden, and health & beauty), and 'Device Used' (classified as desktop, mobile, and tablet). Given its limited ordinal nature in practical usage contexts, 'Device Used' was label-encoded, assigning numeric representations to each device type. In contrast, 'Payment Method' and 'Product Category' were transformed using one-hot encoding to avoid imposing unintended ordinal relationships, which could otherwise distort the learning process in algorithms such as Support Vector Machines and Logistic Regression. This

encoding process expanded the feature space by introducing new binary variables representing the presence of each category.

Numerical features such as 'Transaction Amount', 'Quantity', 'Customer Age', 'Account Age Days', and 'Transaction Hour' were normalized using z-score standardization. This transformation standardized the features to a common scale (mean of 0 and standard deviation of 1), facilitating effective learning by models that are sensitive to feature scaling. Additionally, the 'Transaction Date' field, originally in datetime format, was decomposed to extract relevant temporal features, including 'Transaction Day', 'Month', and 'Day of Week'. These derived variables enabled capturing of time-based behavioral patterns in transactions. A new binary feature, 'Address Match', was engineered by comparing the 'Shipping Address' and 'Billing Address', with matching instances assigned a value of 1 and mismatches marked as 0, leveraging known associations between mismatches and increased fraud risk. The dataset was also assessed for missing values, and it was confirmed that no such values were present, thereby ensuring integrity and readiness for downstream analysis.

Preprocessing: Applying SMOTE for Class Imbalance in 'Is Fraudulent': To address the substantial class imbalance in the 'Is Fraudulent' target variable, where fraudulent transactions constituted only a small fraction (5.17%) of the overall dataset, the Synthetic Minority Over-sampling Technique (SMOTE) was employed. This technique aimed to balance the distribution between fraudulent and legitimate classes by generating synthetic samples for the minority class. The dataset was initially divided into input features and target labels, with the input matrix including all preprocessed features such as standardized transaction metrics, encoded categorical attributes, and engineered variables like 'Address Match'.

SMOTE generated new minority class samples by interpolating between existing fraudulent instances and their nearest neighbors, thereby enriching the dataset without merely replicating existing entries. This method contributed to a balanced training dataset, ensuring equal representation of both classes. Importantly, SMOTE was applied exclusively to the training portion of the data to avoid the risk of data leakage and to preserve the natural imbalance in the evaluation set. Addressing the imbalance was critical to enhancing the model's sensitivity to fraud cases. Without correction, learning algorithms might become biased toward the dominant class and fail to detect fraudulent patterns effectively. Post-SMOTE balancing improved the model's capacity to generalize fraud indicators during the learning process while maintaining a realistic evaluation framework.

Data Splitting: For performance evaluation, the preprocessed dataset was divided into training and testing subsets. The dataset was split such that 70% of the transactions were allocated for model training and the remaining 30% for testing. To maintain the original distribution of fraudulent and legitimate transactions across both subsets, stratified sampling was used based on the 'Is Fraudulent' class label. This ensured that the test data maintained its natural imbalance, accurately reflecting real-world fraud detection challenges.

SMOTE was applied exclusively to the training subset to balance the classes and allow the model to learn effectively from both legitimate and fraudulent patterns. The test set, by retaining its original distribution, enabled assessment under realistic deployment conditions. This setup allowed for the training of predictive models under optimal learning conditions, while ensuring that evaluation remained rigorous and reflective of actual class occurrence. The resulting structure facilitated robust model development and performance measurement within a practical fraud detection framework.

4.4 Model Building

The model building process for the e-commerce fraud detection dataset (23,634 transactions, 5.17% fraudulent) is implemented to address the challenge of identifying fraudulent transactions in an imbalanced dataset. The Model Trainer class supports multiple algorithms, including Logistic Regression, Random Forest, and Gradient Boosting, with the latter proposed due to its superior performance (accuracy 0.9412, AUC 0.8181).

4.4.1 Existing Algorithm: Support Vector Machine Classifier

SVM works by first mapping the preprocessed dataset of 23,634 transactions into a feature space using the RBF kernel, transforming features like standardized Transaction Amount and one-hot encoded Payment Method into a higher-dimensional space where the classes (fraudulent vs. legitimate) become linearly separable. It then identifies the optimal hyperplane that maximizes the margin between the classes by solving a constrained optimization problem using Lagrange multipliers, ensuring the distance to the nearest points (support vectors) is as large as possible. The cost parameter C (set to 1) penalizes misclassifications, allowing some flexibility for outliers, while the RBF kernel parameter gamma (default in scikit-learn) controls the shape of the decision boundary, balancing overfitting and underfitting. The model was trained on a SMOTE-balanced training set (70% of the data, 16,543 transactions) to address the 1:18.3 class imbalance, and its performance on the test set (7,091 transactions) yielded an

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

accuracy of 0.8602, precision of 0.1696, recall of 0.4385, F1 score of 0.2446, and AUC of 0.7227, reflecting moderate fraud detection capability.

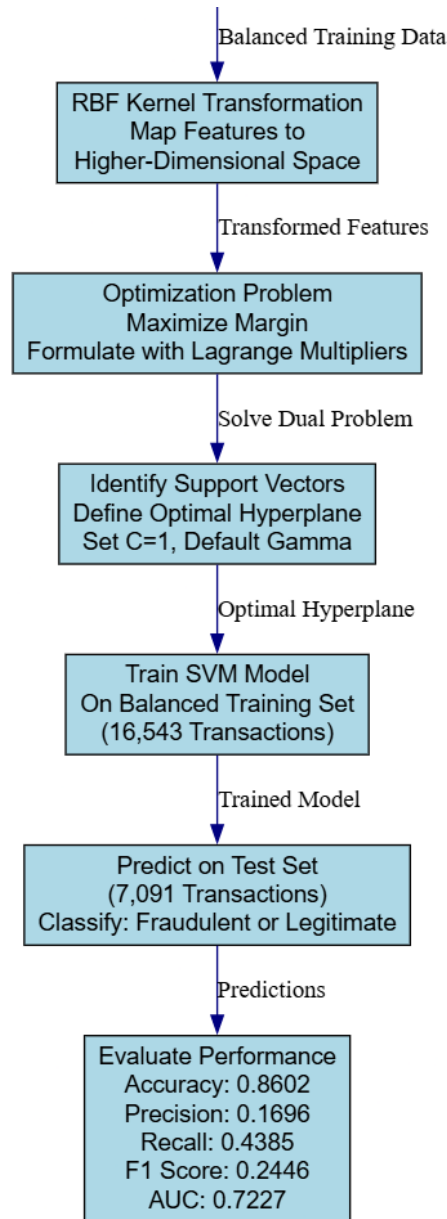


Figure 4.4.1: Existing SVM Block Diagram.

Algorithm Steps (Architecture)

- **Input Data:** Load the preprocessed dataset with features (e.g., Transaction Amount, Address Match) and labels (Is Fraudulent).
- **Apply SMOTE:** Balance the training set to address the 1:18.3 class imbalance using SMOTE.

- **Kernel Transformation:** Map features into a higher-dimensional space using the RBF kernel.
- **Optimization Problem:** Formulate the optimization to maximize the margin between classes.
- **Lagrange Multipliers:** Solve the dual problem to find the optimal hyperplane and support vectors.
- **Decision Boundary:** Compute the decision boundary using the support vectors and kernel function.
- **Hyperparameter Tuning:** Set $C=1$ and use default gamma for the RBF kernel.
- **Training:** Train the model on the balanced training set (16,543 transactions).
- **Prediction:** Classify test data (7,091 transactions) as fraudulent or legitimate.

Disadvantages

- **High Computational Cost:** SVM training scales poorly with larger datasets, making it inefficient for real-time fraud detection scenarios involving thousands of transactions.
- **Complex Hyperparameter Tuning:** Performance is highly dependent on the correct choice of kernel, regularization parameter (C), and kernel coefficient (gamma), requiring extensive tuning.
- **Low Interpretability:** The model's decision boundaries, especially in high-dimensional transformed feature spaces, are difficult to interpret, offering limited insights into fraud behavior.
- **Sensitivity to Outliers:** SVM is vulnerable to noisy data and outliers, which are prevalent in e-commerce transactions and may degrade model accuracy.
- **Scalability Limitations:** The algorithm becomes less practical as the feature space expands, such as after one-hot encoding categorical variables, due to increased complexity and memory usage.

4.4.2 Existing Algorithm: Random Forest Classifier

The Random Forest Classifier works by first creating a SMOTE-balanced training set from the 16,543 transactions (70% of the dataset), ensuring equal representation of fraudulent and legitimate cases. It then constructs 100 decision trees, each trained on a random subset of the data (via bootstrapping) and a random subset of features at each split (e.g., 5 out of 15 features), reducing correlation between trees and enhancing generalization. Each tree recursively splits the data based on feature thresholds (e.g., Transaction Amount > 200), aiming to minimize impurity (using Gini index) at each node, until reaching a maximum depth of 10 or a minimum sample size. During prediction, each tree classifies a test sample (7,091 transactions), and the final prediction is determined by majority voting across all trees, yielding a robust classification (e.g., fraudulent if 60 out of 100 trees predict fraud). Feature importance is computed based on the average reduction in impurity, highlighting influential features like Address Match.

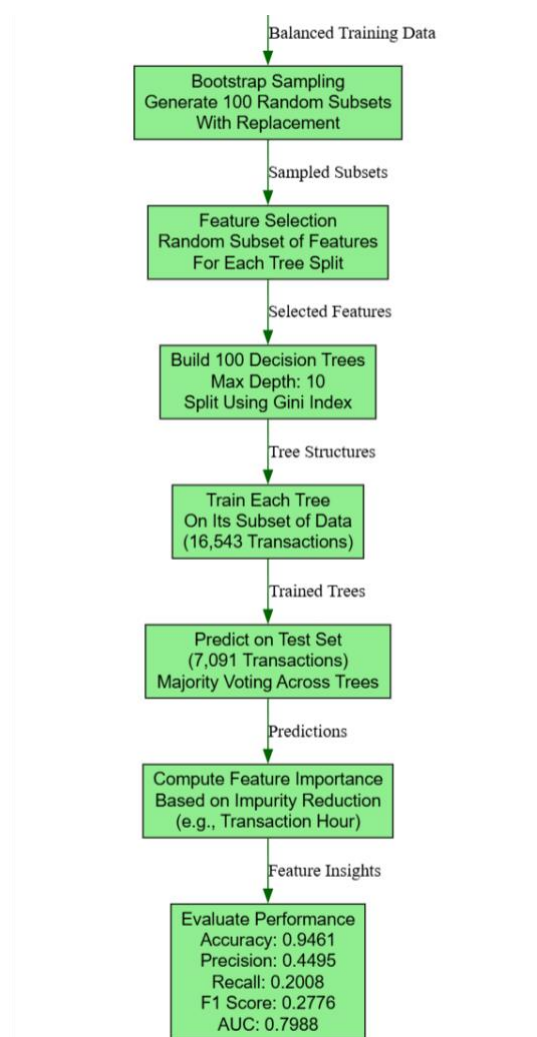


Figure 4.4.2: Existing RFC architecture

Algorithm Steps (Architecture)

- **Input Data:** Load preprocessed dataset with features (e.g., Transaction Amount, Address Match) and labels (Is Fraudulent).
- **Apply SMOTE:** Balance the training set (1:18.3 ratio to 1:1) using SMOTE.
- **Bootstrap Sampling:** Generate 100 random subsets of the training data with replacement.
- **Feature Selection:** Randomly select a subset of features for each tree split.
- **Build Trees:** Construct 100 decision trees, each with a maximum depth of 10.
- **Splitting Criteria:** Use Gini index to determine the best split at each node.
- **Train Trees:** Train each tree on its respective subset of data and features.
- **Prediction:** Classify test data (7,091 transactions) by majority voting across trees.
- **Feature Importance:** Calculate importance based on impurity reduction (e.g., Transaction Hour).

Disadvantages

- **Overfitting Risk on Noisy Data:** Random Forest can overfit to noise and outliers—common in transactional datasets—leading to reduced generalization and lower recall for rare fraudulent cases.
- **Limited Interpretability:** While feature importance is available, the overall decision process of the ensemble model remains opaque compared to more interpretable models like Logistic Regression.
- **Dependence on Balancing Techniques:** Its performance significantly depends on the quality of class balancing (e.g., through SMOTE); imperfections in this step can lead to poor precision-recall balance.
- **Reduced Effectiveness in Highly Imbalanced Data:** Without effective balancing, Random Forest may still favor the majority class, making it challenging to detect rare fraudulent instances accurately.

- **High Computational Demand:** Training numerous decision trees on a large, SMOTE-balanced dataset consumes substantial memory and processing time, making it less optimal for real-time fraud detection.

4.4.3 Existing Algorithm: Logistic Regression Classifier

Logistic Regression works by modeling the relationship between input features and the probability of a specific outcome (e.g., fraud) using the logistic function. It starts by computing a linear combination of input features (e.g., Transaction Amount, Quantity) weighted by learned coefficients, plus a bias term. This linear score is passed through the sigmoid function, which maps it to a probability between 0 and 1. During training, the model optimizes these weights to minimize a loss function (log-loss or binary cross-entropy) using gradient descent, adjusting weights to better predict the true class labels (0 or 1). In project, the model processes preprocessed features from Data Processor, handles class imbalance with SMOTE, and fits the data to predict fraud probabilities. Predictions above a threshold (e.g., 0.5) are classified as fraudulent, enabling the model to flag suspicious transactions while providing interpretable coefficients for feature importance analysis.

Algorithm Steps (Architecture)

- **Input Data Preparation:** Receive preprocessed features (e.g., Transaction Amount, Address Match) and binary labels (0 for legitimate, 1 for fraudulent) from the dataset.
- **Feature Transformation:** Apply standardization (via Standard Scaler) to normalize numerical features for consistent scale.
- **SMOTE Resampling:** Use SMOTE to oversample the minority class (fraudulent transactions) to address class imbalance.
- **Linear Combination:** Compute a weighted sum of features plus a bias term.
- **Sigmoid Activation:** Apply the sigmoid function used to map a probability between 0 and 1.
- **Loss Calculation:** Compute log-loss (binary cross-entropy) to measure the difference between predicted probabilities and true labels.

- **Optimization:** Use gradient descent to iteratively update weights and bias to minimize the loss function.
- **Prediction:** For new data, compute probabilities using the trained weights and classify based on a threshold (e.g., 0.5).
- **Feature Importance:** Extract coefficient magnitudes to indicate feature influence on fraud prediction.

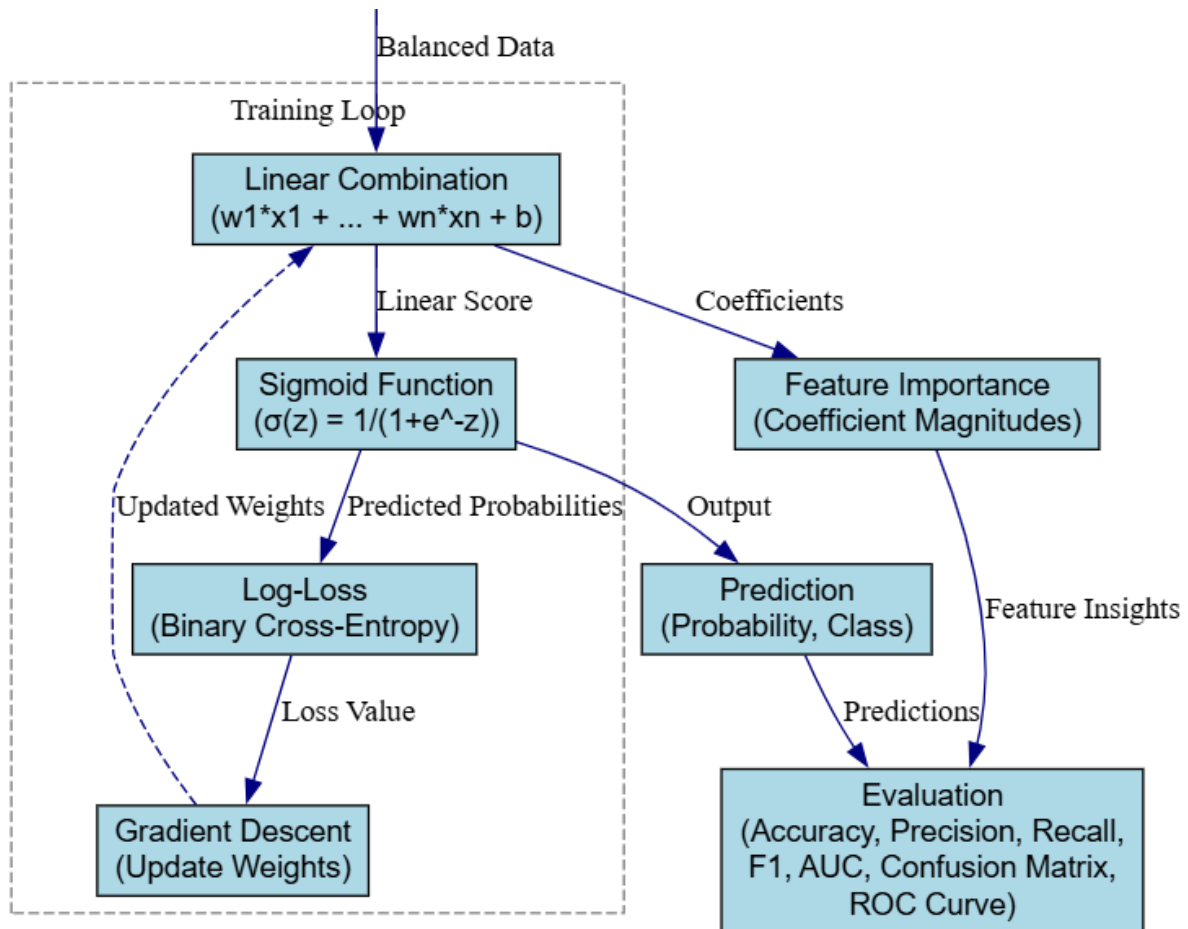


Figure 4.4.3: Existing LRC Architecture

Disadvantages

Logistic Regression, while interpretable and efficient, has limitations in fraud detection project. It assumes linear relationships between features and the log-odds of fraud, which may not capture complex, non-linear patterns in e-commerce data (e.g., interactions between Transaction Hour and Device Used). Its performance (accuracy 0.7243, precision 0.1188) is modest compared to Random Forest (0.9461) or Gradient Boosting (0.9412), suggesting it

struggles with the dataset's complexity and 1:18.3 class imbalance, even with SMOTE. The low precision indicates many false positives, critical in fraud detection where misclassifying legitimate transactions can harm user trust. It lacks the ability to model feature interactions effectively, unlike tree-based models, and may underperform on high-dimensional or noisy data. Additionally, its reliance on a fixed threshold (0.5) for classification may not optimize for imbalanced datasets, requiring manual tuning or cost-sensitive adjustments, which adds complexity

4.4.4 Proposed Algorithm: Gradient Boosting Classifier

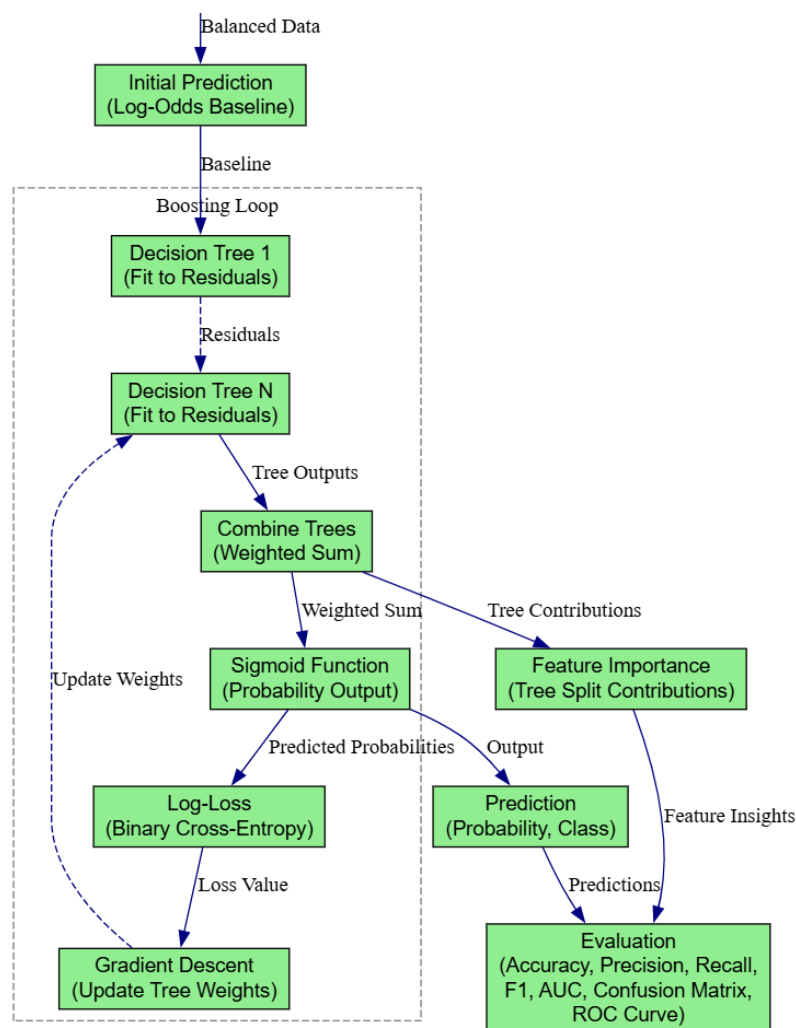
Gradient Boosting works by iteratively building decision trees, each focusing on correcting the residuals (errors) of the previous trees. For fraud detection dataset, it starts with an initial prediction (e.g., the log-odds of fraud). Each tree is fitted to the negative gradient of the loss function (log-loss), adjusting predictions to minimize errors. Features like Transaction Amount and Address Match are split to form tree nodes, with weights updated via gradient descent. SMOTE-balanced data ensures fair representation of fraud cases. The final prediction combines all trees' outputs, producing a probability of fraud, thresholded (e.g., 0.5) for classification. Feature importance is derived from tree contributions, aiding interpretability.

Algorithm Steps (Architecture)

- **Input Data Preparation:** Load preprocessed features (e.g., Transaction Amount, Address Match) and labels.
- **Feature Transformation:** Standardize numerical features using Standard Scaler.
- **SMOTE Resampling:** Balance dataset with synthetic fraud samples via SMOTE.
- **Initialize Prediction:** Start with a baseline prediction (e.g., mean log-odds).
- **Build Decision Trees:** Sequentially fit trees to negative gradients of log-loss.
- **Update Predictions:** Adjust predictions by adding weighted tree outputs.
- **Optimize Weights:** Use gradient descent to minimize loss across iterations.
- **Final Prediction:** Combine tree outputs for fraud probability and classification.
- **Feature Importance:** Calculate feature contributions from tree splits.

Advantages

Gradient Boosting excels in fraud detection project due to its ability to model complex, non-linear relationships in the dataset, capturing interactions between features like Transaction Hour and Device Used, unlike Logistic Regression. Its sequential tree-building approach boosts performance, achieving a high AUC (0.8181), indicating strong class separation for the imbalanced dataset (1:18.3 ratio). SMOTE integration enhances its ability to detect rare fraud cases, improving recall (0.2746) over simpler models. It provides feature importance scores, aiding



interpretability, and its ensemble nature reduces overfitting compared to single trees. Its flexibility to tune hyperparameters (e.g., number of trees, learning rate) allows optimization for 23,634-transaction dataset, making it a robust choice for fraud detection

Figure 4.4.4: gradient boost classifier architecture

4.5 HTML, CSS, and Flask

In E-Commerce Fraud Detection project, HTML, CSS, and Flask work together to build a dynamic, user-friendly web application. HTML defines the structure of web pages (e.g., dashboard, EDA, model comparison), CSS styles these pages for visual appeal and usability, and Flask serves as the backend framework, handling requests, processing data, and rendering templates. The Flask application (app.py) orchestrates the flow between the frontend (HTML/CSS) and backend logic (e.g., Data Processor, Visualization, Model Trainer), enabling users to explore data, train models, and predict fraud on a dataset of 23,634 transactions with a 1:18.3 fraud-to-legitimate ratio.

Connections

- **HTML (Frontend Structure):**
 - **Role:** HTML (HyperText Markup Language) provides the skeletal structure of web pages, defining elements like headings, tables, forms, and buttons. In project, HTML templates (e.g., dashboard.html, eda.html, compare.html, predict.html) create the layout for the dashboard, EDA visualizations, model comparison, and prediction interfaces.
 - **Connection to Flask:** Flask uses the **Jinja2** templating engine to render HTML templates dynamically. In app.py, routes like `@app.route('/')` or `@app.route('/eda')` call `render_template()` to load HTML files, passing data (e.g., dataset statistics, visualization JSON) to populate the templates. For example, eda.html receives JSON from visualization.py (e.g., fraud distribution) to display charts.
 - **Connection to CSS:** HTML includes `<link>` tags in its `<head>` section to reference CSS files (e.g., `<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">`), applying styles to elements like buttons or tables.
- **CSS (Frontend Styling):**
 - **Role:** CSS (Cascading Style Sheets) controls the visual presentation of HTML elements, defining colors, fonts, layouts, and responsiveness. In project, CSS files (likely in a static/css folder) style the Flask app's pages, ensuring a consistent look for the dashboard's stat boxes, EDA charts, and prediction forms.

- **Connection to HTML:** CSS rules target HTML elements via selectors (e.g., class, id, or tags). For instance, a `.stat-box` class in `dashboard.html` might be styled in `style.css` with `background-color: #3498db; color: white;` to highlight transaction counts (e.g., 23,634 total, 1,222 fraudulent).
- **Connection to Flask:** Flask serves CSS files from the static folder using `url_for('static', filename='css/style.css')` in HTML templates. The `/static` route is automatically handled by Flask, ensuring CSS is accessible to the browser. CSS also styles dynamic content (e.g., Plotly charts from `visualization.py`) rendered in HTML.
- **Flask (Backend Framework):**
 - **Role:** Flask, a lightweight Python web framework, manages the backend, handling HTTP requests, routing, data processing, and template rendering. In `app.py`, Flask initializes components (Data Processor, Visualization, Model Trainer), defines routes (e.g., `/`, `/eda`, `/predict`), and processes user inputs (e.g., model selection, transaction data).
 - **Connection to HTML:** Flask renders HTML templates via `render_template()`, passing variables like dataset stats, visualization JSON, or model metrics. For example, the `/eda` route sends fraud distribution JSON from `visualization.py` to `eda.html`, where Plotly.js renders it as a bar chart. User inputs (e.g., form data in `predict.html`) are sent to Flask routes (e.g., `/predict`) via POST requests.
 - **Connection to CSS:** Flask serves CSS files from the static folder, ensuring styles are applied to rendered HTML. It also supports dynamic styling by passing CSS-related variables to templates, though project primarily uses static CSS files.

Working

- **User Interaction Flow:**
 - A user accesses the app (e.g., `http://localhost:5000/`) via a browser, triggering the `@app.route('/')` in `app.py`.

- Flask processes the request, retrieves data (e.g., dataset stats from `Data Processor.get_basic_statistics()`), and renders `dashboard.html`, passing variables like stats (total transactions: 23,634, fraudulent: 1,222).
 - The browser receives `dashboard.html`, which includes a `<link>` to `style.css` in the `<head>`. The CSS styles elements like stat boxes (e.g., blue background for “Total Transactions”).
 - The user sees a styled dashboard with stats and navigation links (e.g., “EDA,” “Models,” “Predict”).
- **Dynamic Content (EDA Example):**
 - Clicking “View EDA” navigates to `/eda`, triggering the `@app.route('/eda')` route in `app.py`.
 - Flask calls `visualization.py` methods (e.g., `get_fraud_distribution()`, `get_payment_method_distribution()`) to generate Plotly JSON charts.
 - The `/eda` route renders `eda.html`, passing JSON data (e.g., fraud distribution bar chart) and stats. The HTML includes `<script>` tags for `Plotly.js` (loaded via CDN or `static/js`) to render charts.
 - CSS styles the chart containers (e.g., `div` with `class="plot"`), ensuring proper layout and responsiveness. For instance, payment method distribution (25% each for debit card, credit card, PayPal, bank transfer) is displayed as a styled bar chart.
 - **Form Submission (Prediction Example):**
 - Navigating to `/predict` renders `predict.html`, a form for inputting transaction data (e.g., Transaction Amount, Payment Method). Flask passes a list of trained models (e.g., Gradient Boosting, AUC 0.8181) from Model Trainer.
 - CSS styles the form (e.g., input fields, submit button) for usability, using classes like `.form-group` or `.btn-primary`.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- Submitting the form sends a POST request to /predict. Flask processes the input in app.py, calls Model Trainer.predict_transaction(), and renders prediction_result.html with the result (e.g., fraud probability).
- The result page is styled by CSS, ensuring consistent design, and may include a chart (e.g., feature importance) generated by visualization.py.

CHAPTER 5

UML DIAGRAMS

Unified Modeling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behavior of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

Key Points About UML

- **Standardized Notation:** UML offers a universal set of symbols and diagrams that standardize how software systems are described, which helps teams speak the same language regardless of their background or the programming language they use.
- **Types of Diagrams:** UML includes various diagrams that can be categorized into two main types:
 - **Structural Diagrams:** These describe the static aspects of the system. Examples include Class Diagrams, Component Diagrams, and Deployment Diagrams.
 - **Behavioral Diagrams:** These focus on the dynamic aspects and interactions within the system. Examples include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and Collaboration Diagrams.
- **System Documentation and Communication:** UML serves as an effective tool for documenting system requirements, design decisions, and the overall architecture. It helps bridge the gap between technical and non-technical stakeholders by providing clear, visual representations of the system.
- **Design and Analysis:** By modeling different aspects of a system, UML enables developers to analyze and validate the design early in the development process. This can lead to better decision-making, reduced complexity, and improved system maintainability.

Flexibility: UML is versatile and can be used across a wide range of applications, from small-scale projects to large, complex systems. It supports object-oriented design principles and can be adapted to various methodologies such as Agile or Waterfall

5.1 Class Diagram

A class diagram in UML serves as a blueprint for defining the static structure of a system by illustrating its classes, their attributes, methods, and the relationships among them. Each class represents a key entity in the system, with attributes describing its properties and methods outlining its behaviors or functionalities. Relationships like "is-a" (inheritance) or "has-a" (association) clarify how classes interact, enabling developers to refine the system's design based on use case requirements. This diagram is crucial for translating high-level requirements into a detailed, implementable structure, ensuring all components are clearly defined and interconnected.

In the context of AI systems, class diagrams are particularly valuable for modeling complex architectures, such as neural networks or decision-making systems. For example, an AI system for recognition might include classes like "Processor," "FeatureExtractor," and "Classifier," each with specific attributes (e.g., resolution) and methods (e.g., preprocess()). By mapping these elements, the diagram provides a clear framework for developers to build and integrate AI components, ensuring modularity and scalability while aligning with the system's functional goals.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

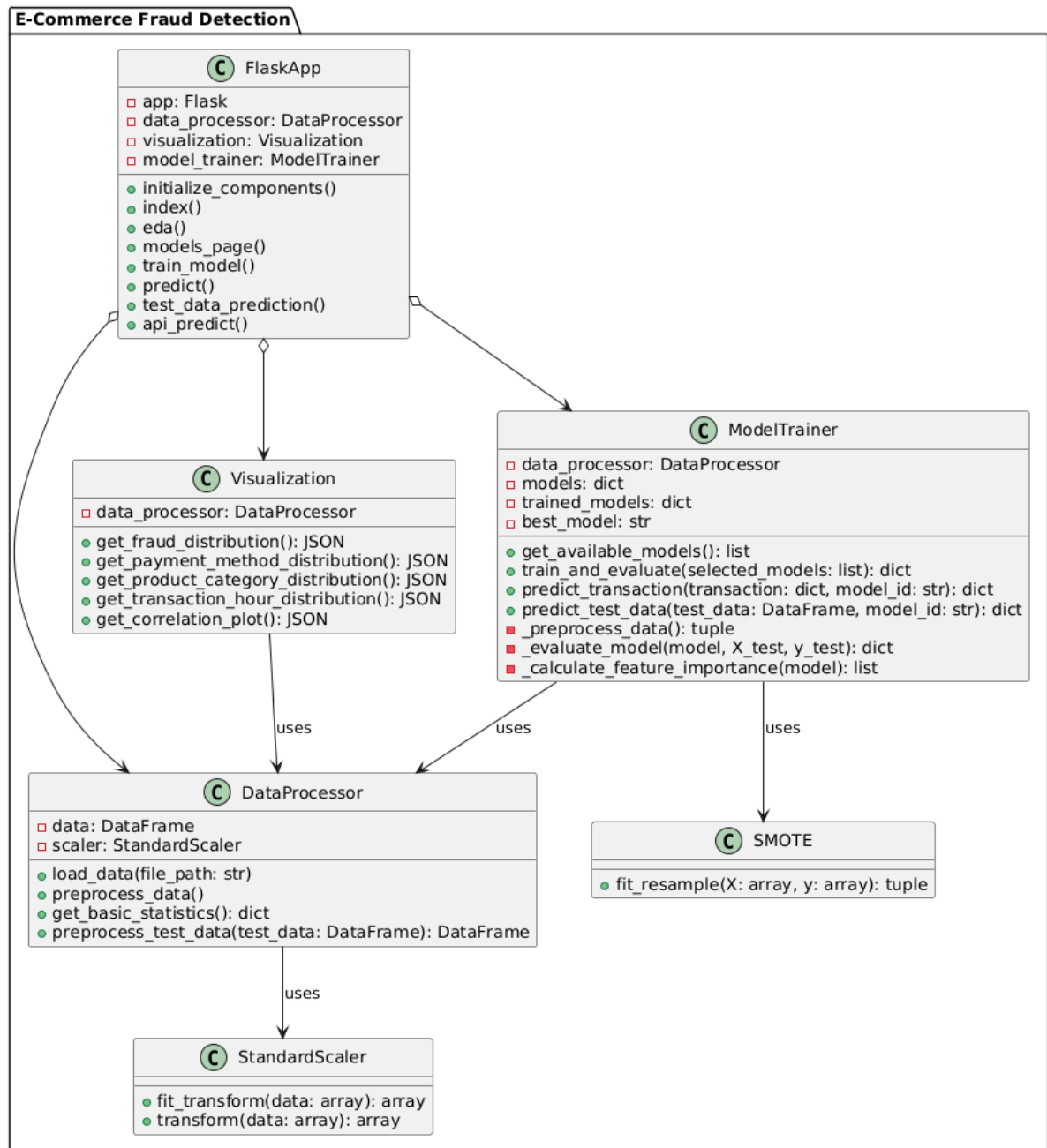


Figure 5.1: class diagram

5.2 Use Case Diagram

A use case diagram in UML offers a high-level, behavioral view of a system by depicting its actors, use cases, and their interactions. Actors, which can be users or external systems, represent entities interacting with the system, while use cases describe specific functionalities or goals the system supports, such as "Login" or "Process Payment." The diagram highlights dependencies, such as "includes" or "extends" relationships, to show how use cases relate,

providing a clear picture of what the system does and who it serves. This makes it an essential tool for stakeholders to understand system functionality without diving into technical details.

For AI systems, use case diagrams are instrumental in outlining how users or other systems interact with AI functionalities. For instance, in a chatbot system, actors like "User" or "Admin" might engage with use cases like "Send Message" or "Train Model." The diagram helps developers and stakeholders align on the AI system's scope, ensuring that critical interactions, such as user queries or model updates, are captured early in the design process, facilitating clear communication and requirement validation.

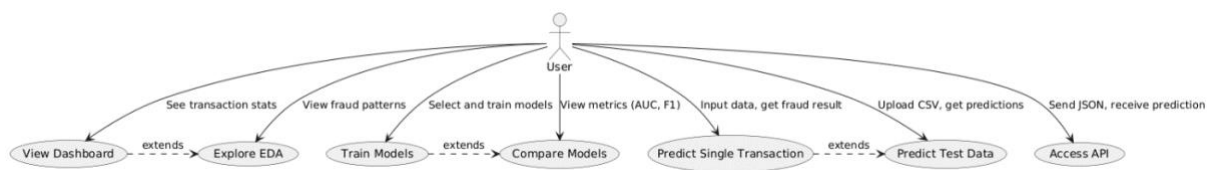


Figure 5.2: use case diagram

5.3 Sequence Diagram

A sequence diagram in UML illustrates the dynamic interactions between objects or processes in a system, focusing on the order of messages exchanged over time. Represented by vertical lifelines for each object and horizontal arrows for messages, it captures the flow of operations, such as method calls or data exchanges, in a specific scenario. This makes it ideal for detailing how components collaborate to achieve a particular function, providing a clear, step-by-step visualization of system behavior.

In AI applications, sequence diagrams are particularly useful for modeling the runtime behavior of complex processes, such as how a Outputn system processes user input. For example, a sequence diagram might show a "User" sending a query to a "OutputnEngine," which then interacts with a "DataStore" and a "PredictionModel" to return results. This clarity helps developers debug and optimize AI workflows, ensuring efficient communication between components like data pipelines and inference modules, ultimately improving system performance.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

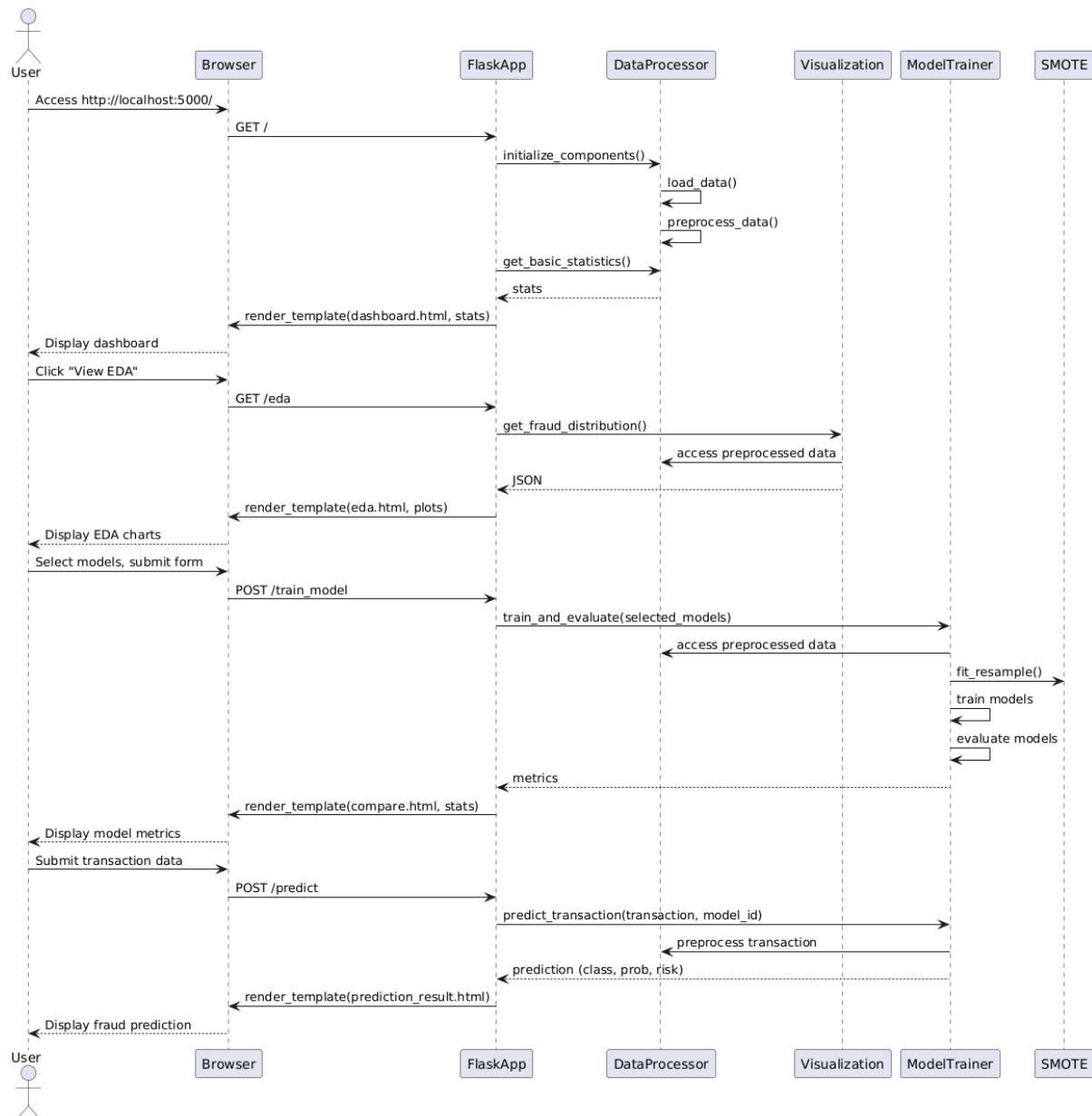


Figure 5.3: sequence diagram

5.4 Component Diagram

A component diagram in UML visualizes the organization and relationships of a system's physical or logical components, such as modules, libraries, or subsystems. It shows how these components are interconnected through interfaces, emphasizing modularity and reusability in the system's architecture. By abstracting the system into manageable parts, it helps developers understand how different pieces, like databases or APIs, work together to deliver functionality.

In AI systems, component diagrams are critical for mapping out the architecture of distributed or modular systems. For instance, an AI platform might include components like

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

"DataIngestionModule," "ModelTrainingUnit," and "InferenceEngine," each interacting through defined interfaces. This diagram aids in designing scalable AI solutions, ensuring components like preprocessing pipelines or model servers are loosely coupled yet cohesive, which is vital for deploying and maintaining complex AI systems.

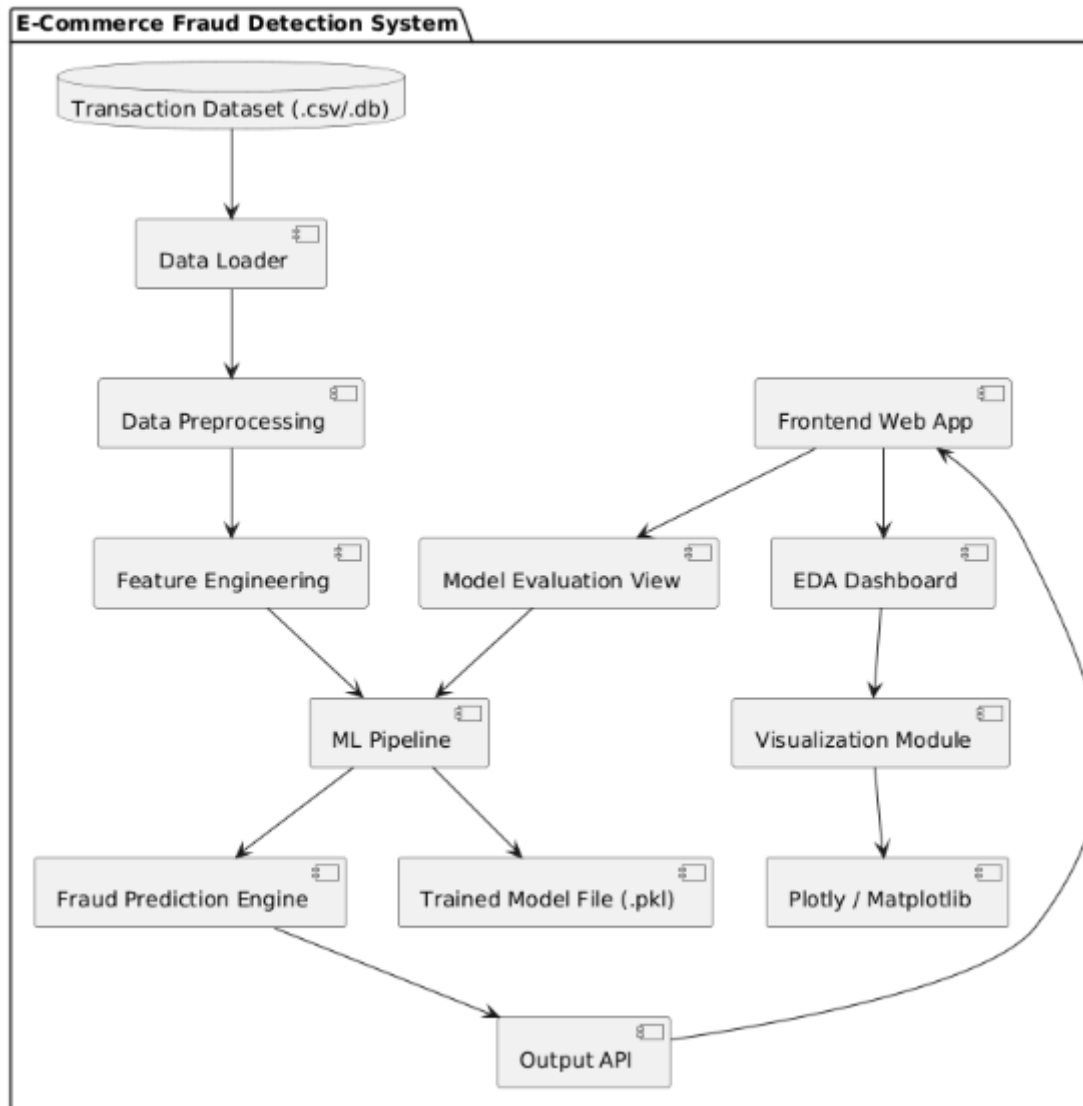


Figure 5.4: component diagram

5.5 Deployment Diagram

A deployment diagram in UML illustrates the physical architecture of a system, showing how software components are distributed across hardware nodes, such as servers, cloud instances, or edge devices. It highlights the relationships between these nodes, including communication protocols or network connections, providing a clear view of how the system is deployed in a

real-world environment. This is essential for understanding the system's infrastructure and ensuring it meets performance and scalability requirements.

For AI systems, deployment diagrams are key to visualizing how models and services are hosted. For example, a machine learning system might have a "TrainingServer" hosting a "ModelTraining" component and an "InferenceServer" running a "PredictionService," connected via a cloud network. This diagram helps developers optimize resource allocation, such as GPU usage for AI inference, and ensures robust deployment strategies, particularly for distributed AI applications like real-time analytics or IoT-based systems.

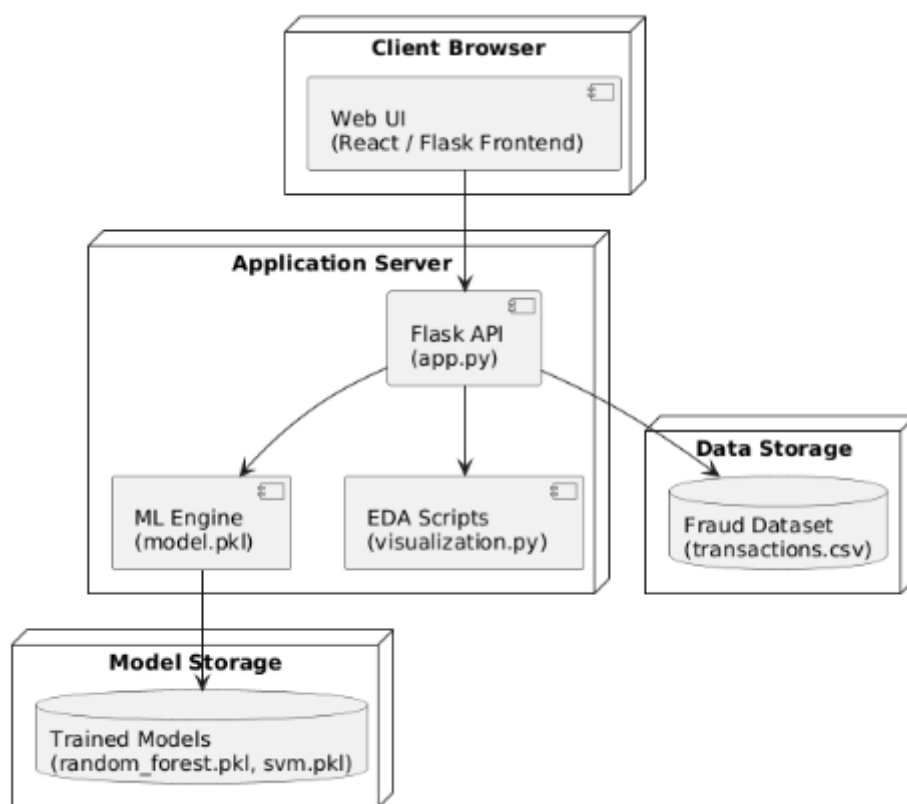


Figure 5.5: deployment diagram

5.6 Activity Diagram

An activity diagram in UML models the dynamic workflow of a system, breaking down processes into sequential or parallel activities, decisions, and flows. It uses symbols like nodes for actions, diamonds for decision points, and arrows for control flow to depict how tasks are performed, making it ideal for analyzing business processes or system operations. This diagram

provides a clear, visual way to understand the logic and flow of complex activities within a system.

In AI contexts, activity diagrams are useful for mapping out processes like data preprocessing or model training workflows. For example, an activity diagram for an AI pipeline might show steps like "Collect Data," "Clean Data," "Train Model," and "Evaluate Results," with decision points for handling errors or retraining. This helps developers optimize AI workflows, identify bottlenecks, and ensure that processes like hyperparameter tuning or data validation are systematically executed.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

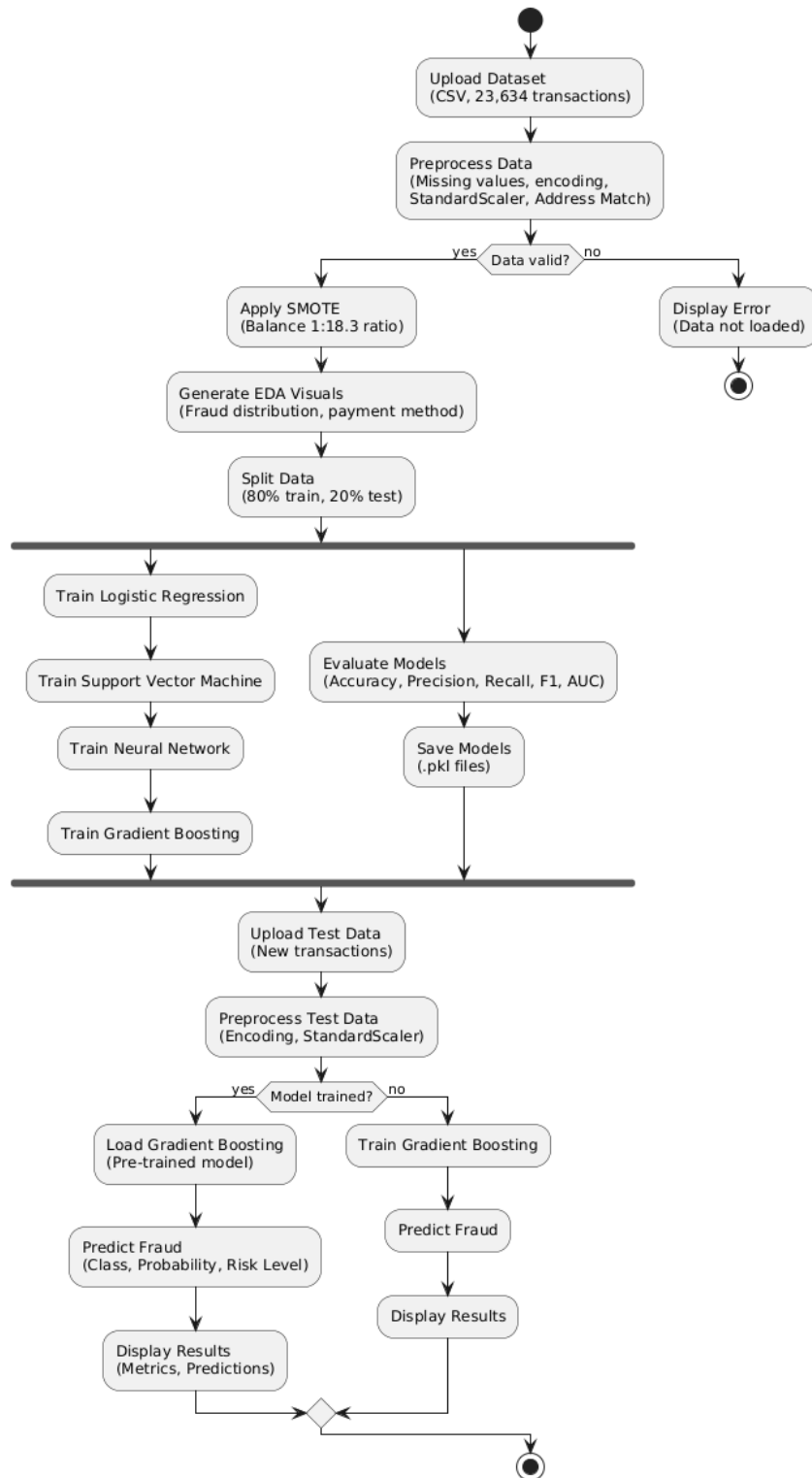


Figure 5.6: activity diagram

5.7 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) illustrates how data moves through a system, showing processes, data stores, external entities, and the flow of information between them. Unlike UML diagrams,

which focus on system structure or behavior, DFDs emphasize the transformation and movement of data, making them ideal for understanding how information is processed and stored. They are particularly useful for breaking down complex systems into manageable, data-centric processes.

In AI systems, DFDs are invaluable for mapping data pipelines, such as those in machine learning workflows. For instance, a DFD might show data flowing from an "External Sensor" to a "Data Preprocessing" process, then to a "Model Training" process, and finally stored in a "Model Repository." This helps developers trace data transformations, ensure data integrity, and optimize AI systems for tasks like real-time predictions or large-scale data analytics, aligning system design with data-driven requirements.

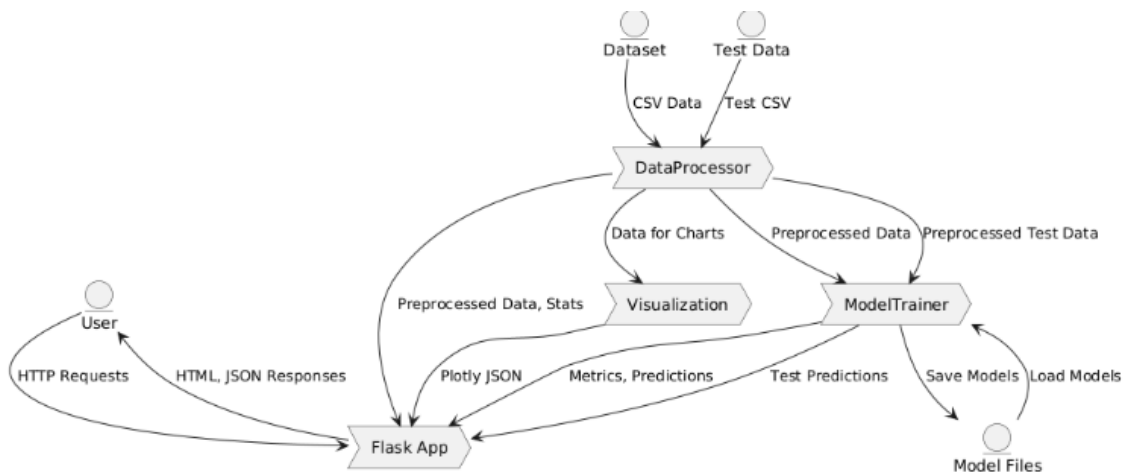


Figure 5.7: data flow diagram

CHAPTER 6

SOFTWARE REQUIREMENTS

6.1 Software Requirements

Python 3.7.6 serves as a pivotal version for developers and researchers due to its robust features, backward compatibility, and widespread support across a variety of libraries and frameworks. Released during a time when machine learning and data science tools were rapidly evolving, Python 3.7.6 provided a stable and consistent platform. This version includes critical improvements like enhanced asyncio functionality for asynchronous programming, increased precision for floating-point numbers, and optimized data structures. It became the go-to version for compatibility with popular libraries like TensorFlow 2.0, PyTorch, and Pandas, ensuring seamless integration and efficient execution for both academic and industrial applications.

Compared to older Python versions, 3.7.6 introduced several features such as dataclasses, which simplified boilerplate code for object-oriented programming. The improved async and await syntax made concurrent programming more intuitive, while changes to the standard library enhanced usability and performance. Over newer versions, Python 3.7.6 remains a preferred choice for legacy systems and researchs requiring compatibility with libraries that may not yet support the latest Python updates. Its combination of stability and maturity ensures that it is reliable for long-term researchs, especially in environments where upgrading the Python interpreter might disrupt existing workflows.

6.1.1 TensorFlow Environment

TensorFlow provides a comprehensive ecosystem for building, training, and deploying machine learning models. Its support for numerical computation and deep learning applications makes it a staple in AI research and development. By offering a flexible architecture, TensorFlow enables deployment across a variety of platforms, including desktops, mobile devices, and the cloud. The ability to scale across CPUs, GPUs, and TPUs ensures that TensorFlow is suitable for both small experiments and large-scale production systems.

TensorFlow's transition from older versions, like 1.x, to 2.x brought significant improvements in ease of use, including the introduction of the tf.keras API for building models, eager execution for dynamic computation, and enhanced debugging capabilities. Compared to newer frameworks, TensorFlow retains a strong advantage due to its mature community support,

extensive documentation, and integration with TensorFlow Extended (TFX) for managing production pipelines. Its compatibility with other libraries and tools, such as Keras and TensorBoard, makes it a robust choice for end-to-end machine learning solutions.

6.1.2 Packages Overview

Keras: Older versions of Keras required extensive configuration for custom model creation. Version 2.3.1 unified the APIs with TensorFlow integration, reducing overhead and enabling direct use of TensorFlow backends, ensuring faster execution and easier debugging. While newer versions focus on performance and distributed training, version 2.3.1 is lightweight and stable, making it ideal for smaller researchs without the complexity introduced in later iterations, which are more suited for advanced workflows.

NumPy: Version 1.19.5 introduced critical bug fixes and performance enhancements over older versions, especially for operations involving large datasets. The improved random number generator and better handling of exceptions provide more reliable results for numerical computations. This version remains compatible with a wide range of dependent packages. While newer versions optimize speed further, 1.19.5 balances stability and compatibility, ensuring fewer compatibility issues with older software stacks.

Pandas: Version 0.25.3 brought significant speed improvements for large-scale data processing, particularly in operations like groupby. Enhancements in handling missing data and improved compatibility with external libraries made this version more robust for data analysis tasks. While newer versions does not add features like enhanced type checking, 0.25.3 remains lightweight and stable for researchs that do not require cutting-edge functionalities, making it a practical choice for legacy systems.

Imbalanced-learn: Version 0.7.0 introduced optimized algorithms for handling class imbalances, such as improved SMOTE implementations. This update also enhanced the ease of integrating with scikit-learn pipelines. While newer versions do not contain experimental features, 0.7.0 is reliable and well-documented, ensuring robust performance in addressing data imbalance issues without unnecessary complexity.

Scikit-learn: Version 0.23.1 included improved support for cross-validation and hyperparameter optimization. Updates to RandomForestClassifier and GradientBoostingClassifier increased model accuracy and efficiency. 0.23.1 is widely tested

and compatible with older hardware, making it a dependable choice for environments where the latest versions may introduce compatibility issues.

Imutils: This package provides an easy-to-use interface for image processing tasks. Its functions for resizing, rotating, and translating images simplified workflows compared to writing custom code. Its lightweight nature and stable functionality make it suitable for researchs not requiring cutting-edge image manipulation techniques, balancing simplicity and capability.

Matplotlib: Version 3.x improved plot interactivity and introduced better 3D plotting capabilities. The `tight_layout` function and compatibility with modern libraries streamlined visualization workflows. The earlier versions maintain stability and compatibility with older datasets and software, avoiding potential issues from newer, untested updates.

Seaborn: Improved APIs in newer versions simplified aesthetic customization of plots. The addition of new themes and color palettes in 0.11.x enhanced visual appeal for exploratory data analysis. Older versions remain computationally less demanding, suitable for lightweight applications without requiring extensive customizations.

OpenCV-Python: Recent updates enhanced compatibility with deep learning frameworks and accelerated image processing pipelines, especially for real-time applications. Older versions are stable and resource-efficient, making them ideal for systems with limited computational capacity or for legacy applications.

H5Py: Version 2.10.0 improved file handling efficiency for large datasets. It introduced better support for advanced indexing, which is crucial for working with high-dimensional data. While newer versions support more advanced features, 2.10.0 ensures compatibility with older machine learning frameworks and models.

Jupyter: Jupyter improved the interactivity and scalability of notebooks for collaborative coding and visualization tasks. Integration with tools like Matplotlib made it a preferred environment for data exploration. Earlier versions are stable and lightweight, avoiding potential issues with dependencies introduced in newer releases.

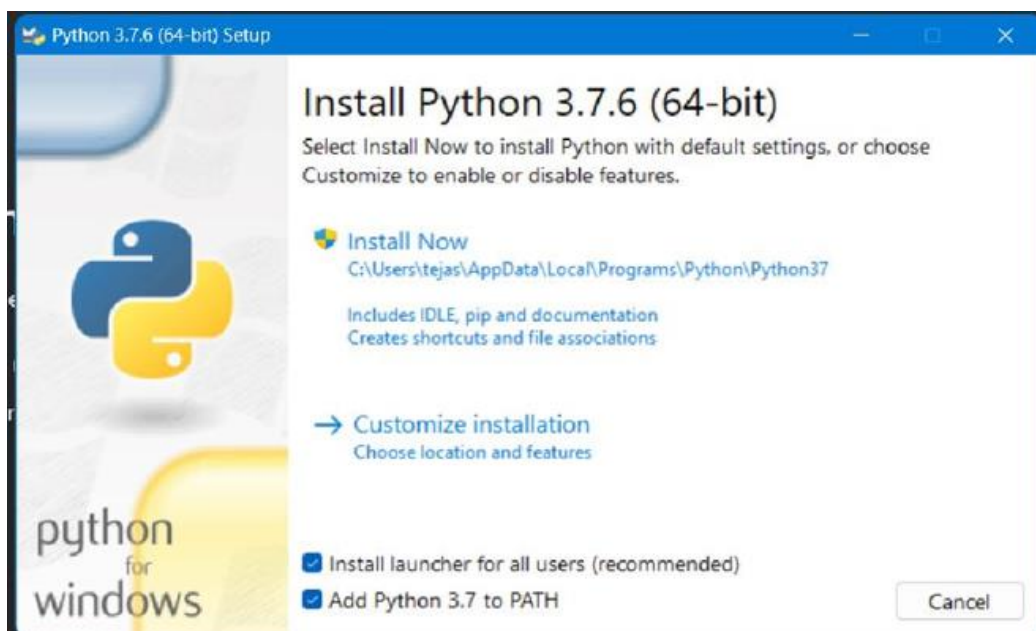
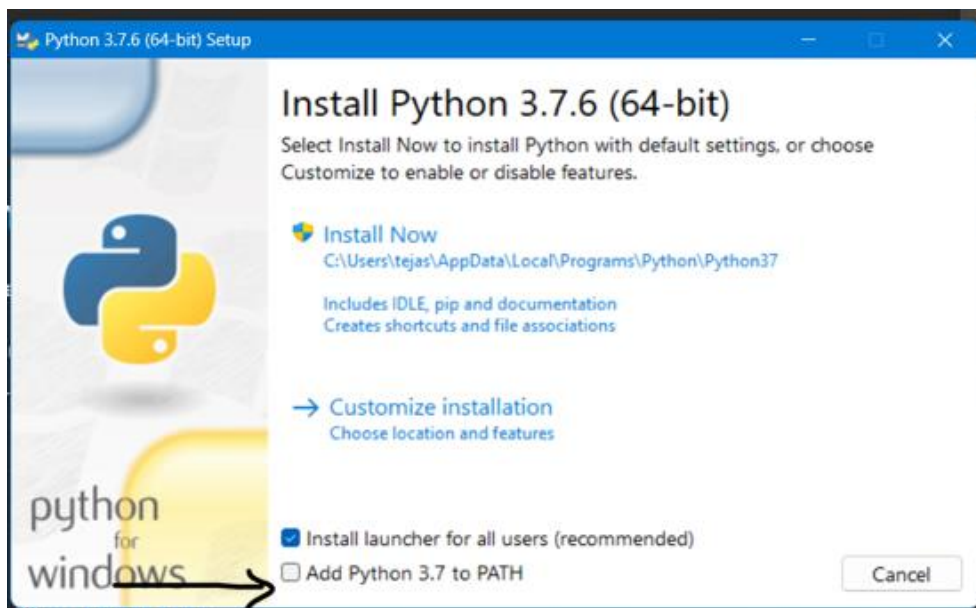
6.1.3 Python Installation Procedure

Step 1: Download Python 3.7.6 Visit the official Python website by clicking the following link: <https://www.python.org/downloads/release/python-376/>. Scroll down the page until you

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

reach the "Files" section. Locate the downloadable file for operating system (e.g., Windows, macOS, or Linux) and click the corresponding link to start the download.

Step 2: Verify the Downloaded File: Once the download is complete, you will have the Python 3.7.6 installer file on system. Ensure the file matches operating system (e.g., a .exe file for Windows or a .pkg file for macOS) before proceeding to the next step. Click “Add Python 3.7 to Path”, which creates the environmental variables in OS. So, the user will get access to the python with command prompt.



Step 3: Begin the Installation Process: Open the downloaded Python installer file. During the installation setup, you will see an option labeled "Add Python 3.7 to PATH." Make sure to

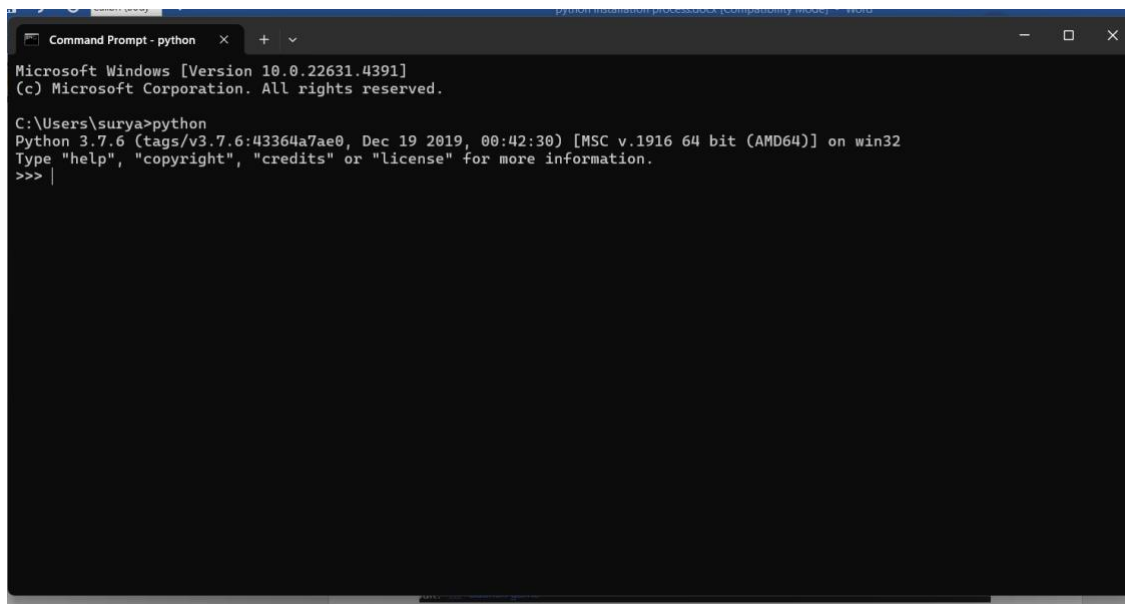
check this box to ensure Python is added to system's environment variables, allowing you to run Python from the command line easily.

Step 4: Install Python: After checking the "Add Python 3.7 to PATH" box, click the "Install Now" button to start the installation. Wait for the installation process to complete. Once finished, you will see a confirmation message indicating that Python 3.7.6 has been successfully installed.



Step 5: Verify the Installation: Open the Command Prompt (on Windows) or Terminal (on macOS/Linux). Type "python --version" and press Enter. If the installation was successful, the output should display "Python 3.7.6." This confirms that Python is correctly installed and accessible.

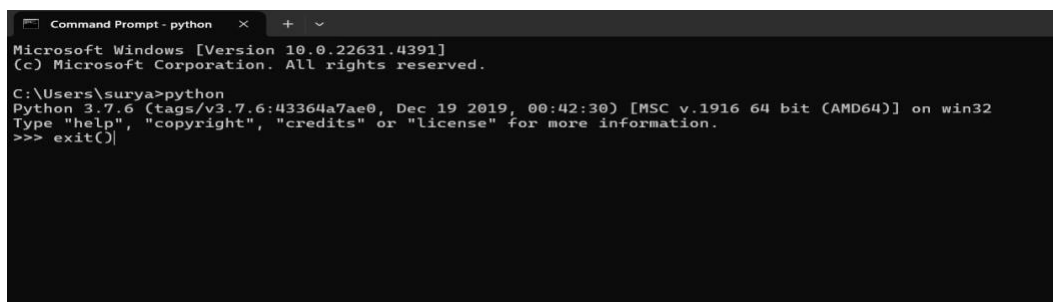
AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION



```
Command Prompt - python
Microsoft Windows [Version 10.0.22631.4391]
(c) Microsoft Corporation. All rights reserved.

C:\Users\surya>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

Step 6: Exit the Python Interpreter: If you entered the Python interactive shell by typing "python," exit it by typing "exit()" and pressing Enter. This will return you to the Command Prompt or Terminal.



```
Command Prompt - python
Microsoft Windows [Version 10.0.22631.4391]
(c) Microsoft Corporation. All rights reserved.

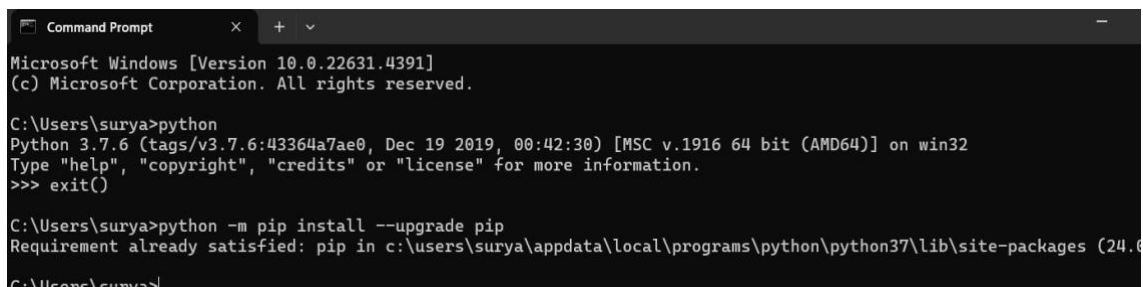
C:\Users\surya>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()

C:\Users\surya>
```

Step 7: Install the Required Packages: Use the following commands to install the necessary Python packages. Enter each command one by one in the Command Prompt or Terminal, pressing Enter after each. These commands will upgrade pip (Python's package manager) and install the specified versions of the required libraries:

- `python -m pip install --upgrade pip`
- `pip install tensorflow==1.14.0`
- `pip install keras==2.3.1`
- `pip install pandas==1.3.5`
- `pip install scikit-learn==1.0.2`

- pip install imutils
- pip install matplotlib==3.2.2
- pip install seaborn==0.12.2
- pip install opencv-python==4.1.1.26
- pip install h5py==2.10.0
- pip install numpy==1.19.2
- pip install imbalanced-learn==0.7.0
- pip install jupyter
- pip install protobuf==3.20.*
- pip install scikit-image==0.16.2



```
Microsoft Windows [Version 10.0.22631.4391]
(c) Microsoft Corporation. All rights reserved.

C:\Users\surya>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()

C:\Users\surya>python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\surya\appdata\local\programs\python\python37\lib\site-packages (24.0)
C:\Users\surya>
```

6.2 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement:** 1 GHz single-core processor.

- **Recommended:** Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

- **Minimum Requirement:** 512 MB of RAM.
- **Recommended:** 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage required for Python libraries, modules, and researchs.

- **Minimum Requirement:** 200 MB of free disk space for installation.
- **Recommended:** At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale researchs or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows:** 32-bit and 64-bit systems, Windows 7 or later.
- **macOS:** macOS 10.9 or later.
- **Linux:** Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 7

FUNCTIONAL REQUIREMENTS

Functional requirements are detailed statements that specify what a system should do. They describe the system's behavior, functions, and services, outlining how it responds to certain inputs, performs tasks, and interacts with users or other systems. Essentially, they answer the question, "What should the system do?" Here are some key aspects:

- **Functionality:** They define the specific functions or operations that the system must perform.
- **Inputs and Outputs:** They detail the types of inputs the system accepts and the outputs it produces.
- **User Interactions:** They describe how users interact with the system, including command inputs, error handling, and responses.
- **Data Management:** They outline requirements related to data storage, retrieval, and processing.
- **System Behavior:** They specify how the system behaves in various scenarios, including normal operations and exceptional conditions.

Below is a breakdown of the functions used in the research along with their requirements. These “function requirements” describe what each function is responsible for, the expected inputs, processing steps, and outputs or side effects. This can serve as a high-level specification for each function in the research.

Requirements Analysis Stage

This stage focuses on identifying and documenting the functional needs for the Model Trainer class and its methods.

FR1: Model Trainer Initialization and Configuration

- Define the need for a Model Trainer class to encapsulate model training, evaluation, and prediction logic for fraud detection.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- Require integration with a Data Processor instance to access preprocessed training and test data.
- Specify the need to support multiple machine learning models (e.g., Logistic Regression, Random Forest, etc.) for flexibility.
- Ensure the class can handle model persistence by loading or saving trained models to files.

FR2: Support for Multiple Machine Learning Models

- Identify the need to define a collection of machine learning models (e.g., Logistic Regression, Random Forest, Gradient Boosting, SVM, Neural Network).
- Require each model to be associated with a unique ID, name, and a flag for feature importance support.
- Allow for dynamic addition of new models in the future without major code restructuring.

FR3: Model Training and Evaluation

- Define the requirement to train selected models on resampled (e.g., SMOTE) and preprocessed training data.
- Specify the need to evaluate models using metrics like accuracy, precision, recall, F1-score, and AUC.
- Require the ability to load pre-trained models from files (e.g., .pkl) to avoid redundant training.
- Ensure compatibility with both scikit-learn and TensorFlow models for training and prediction.

FR4: Feature Importance Calculation

- Require calculation of feature importance for models that support it (e.g., Logistic Regression, Random Forest, Gradient Boosting).
- Specify the need to map importance scores to feature names for interpretability.

- Ensure results are sorted and formatted for easy visualization or reporting.

FR5: Performance Metrics and Visual Data Storage

- Define the need to compute and store confusion matrices, ROC curves, and classification reports for each model.
- Require these metrics to be accessible for analysis and visualization in other components (e.g., Flask app or visualization module).
- Ensure data is stored in a format suitable for JSON serialization (e.g., lists, dictionaries).

FR6: Transaction Prediction Capability

- Specify the need to predict fraud probability for a single transaction based on input data.
- Require preprocessing of transaction data to match the training pipeline (e.g., handle dates, addresses).
- Define the need to return prediction, probability, model name, fraud status, and risk level.

FR7: Test Data Prediction and Analysis

- Require the ability to make predictions on the test dataset for evaluation.
- Specify the need to compute metrics, confusion matrices, ROC curves, and classification reports for test data.
- Define the need to sample test data (e.g., 100 examples) with predictions, actual labels, and categories (TP, TN, FP, FN) for display.

FR8: Best Model Selection

- Define the requirement to identify the best-performing model based on AUC score.
- Ensure the method skips models with errors and handles cases where no models are trained.

Design Stage

This stage outlines the high-level design considerations, including architecture, data flow, and integration for each function.

FR1: Model Trainer Initialization and Configuration

- Design the Model Trainer class to accept a Data Processor instance as an input parameter.
- Store models, results, trained models, feature importances, confusion matrices, and ROC curves in class attributes (dictionaries).
- Plan to call a private method `_define_models` to initialize available models during object creation.

FR2: Support for Multiple Machine Learning Models

- Design a dictionary structure to store model metadata (ID, name, model instance, feature importance support).
- Plan to include scikit-learn models (Logistic Regression, Random Forest, Gradient Boosting, SVM) and a TensorFlow neural network.
- Design a factory method `_create_neural_network` to dynamically build the neural network based on input dimensions.

FR3: Model Training and Evaluation

- Design a pipeline to apply SMOTE for resampling imbalanced training data and transform data using the Data Processor pipeline.
- Plan to check for existing .pkl files in a "models" directory and load them if available, otherwise train and save models.
- Design evaluation to compute predictions and probabilities, storing results in a dictionary for each model.
- Ensure error handling and logging for robustness during training and evaluation.

FR4: Feature Importance Calculation

- Design a private method `_calculate_feature_importance` to handle feature importance for supported models.
- Plan to extract coefficients for Logistic Regression and feature importances for tree-based models.
- Design the output as a sorted list of dictionaries with feature names and importance scores for easy integration with visualization.

FR5: Performance Metrics and Visual Data Storage

- Design methods (`get_confusion_matrix`, `get_roc_curve`, `get_classification_report`) to store and retrieve metrics.
- Plan to convert confusion matrices and ROC curve data (FPR, TPR, thresholds) to lists for JSON compatibility.
- Design storage of classification reports as dictionaries for detailed precision, recall, and F1-score analysis.

FR6: Transaction Prediction Capability

- Design a `predict_transaction` method to accept a dictionary of transaction data and a model ID.
- Plan a private `_preprocess_transaction` method to handle date parsing, address matching, and column alignment.
- Design the output to include prediction (0/1), probability, model name, fraud status, and a risk level based on probability thresholds.

FR7: Test Data Prediction and Analysis

- Design a `predict_test_data` method to use the test dataset from Data Processor for predictions.
- Plan to compute metrics, confusion matrices, ROC curves, and classification reports for comprehensive evaluation.
- Design a sampling strategy to select up to 100 test examples, prioritizing TP, TN, FP, and FN cases for balanced display.

- Ensure feature values are safely converted (e.g., floats, strings) for JSON serialization and display.

FR8: Best Model Selection

- Design a `get_best_model` method to iterate through results and compare AUC scores.
- Plan to skip models with errors and return `None` if no valid models exist.
- Ensure the method integrates with the stored results from training and evaluation.

Implementation Stage

This stage details the coding requirements, including specific methods, error handling, and dependencies.

FR1: Model Trainer Initialization and Configuration

- Implement `Model Trainer.__init__` to accept a `Data Processor` instance and initialize dictionaries for models, results, etc.
- Code a private `_define_models` method to populate a dictionary with model IDs, names, instances, and feature importance flags.
- Use logging for debugging and error tracking throughout the class.

FR2: Support for Multiple Machine Learning Models

- Implement the `_define_models` method to include Logistic Regression, Random Forest, Gradient Boosting, SVM, and a Neural Network.
- Code a `_create_neural_network` method to build a Sequential model with Dense, Dropout layers, and Adam optimizer.
- Ensure SVM is configured with `probability=True` and `class_weight='balanced'` for fraud detection.

FR3: Model Training and Evaluation

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- Implement `train_and_evaluate` to apply SMOTE resampling and transform data using the preprocessing pipeline.
- Code logic to check for existing `.pkl` files in a "models" directory; load if present, otherwise train and save using `joblib`.
- Implement training for the neural network with early stopping, 25 epochs, and validation split; handle predictions differently for TensorFlow.
- Compute metrics, confusion matrices, and ROC curves, storing results in class attributes with error handling via `try-except`.

FR4: Feature Importance Calculation

- Implement `_calculate_feature_importance` to extract coefficients for Logistic Regression and feature importances for tree-based models.
- Code logic to map importance scores to feature names from Data Processor and sort by value.
- Handle errors (e.g., length mismatch) with logging and store results as a list of dictionaries.

FR5: Performance Metrics and Visual Data Storage

- Implement `get_confusion_matrix`, `get_roc_curve`, and `get_classification_report` to return stored metrics.
- Convert confusion matrices to 2D lists with labels ('Legitimate', 'Fraudulent') and ROC curve data to lists (FPR, TPR, thresholds).
- Ensure classification reports are stored as dictionaries from `classification_report` for easy access.

FR6: Transaction Prediction Capability

- Implement `predict_transaction` to accept a transaction dictionary and model ID, defaulting to 'random_forest'.
- Code `_preprocess_transaction` to parse dates, compute address matches, and align columns with default values for missing data.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

- Implement prediction logic for scikit-learn and TensorFlow models, returning prediction, probability, model name, fraud status, and risk level.
- Use a private `_get_risk_level` method to map probability to risk levels (Low, Medium, High, Very High).

FR7: Test Data Prediction and Analysis

- Implement `predict_test_data` to make predictions on the test set using the specified model.
- Code logic to compute metrics, confusion matrices, ROC curves, and classification reports, storing results in a dictionary.
- Implement sampling of 100 test examples, prioritizing TP, TN, FP, and FN, with features converted for JSON compatibility.
- Handle errors with logging and ensure robust data type conversion (e.g., floats for numbers, strings for categories).

FR8: Best Model Selection

- Implement `get_best_model` to iterate through `self.results` and identify the model with the highest AUC.
- Code logic to skip models with errors and return `None` if no valid results exist.
- Ensure the method relies on stored results from `train_and_evaluate` for consistency.

Testing Stage

This stage defines requirements for testing each function to ensure reliability, accuracy, and robustness.

FR1: Model Trainer Initialization and Configuration

- Test `Model Trainer.__init__` with a valid `Data Processor` instance to ensure attributes are initialized correctly.
- Verify that `_define_models` populates the `self.models` dictionary with all expected models and metadata.

- Check logging output for initialization errors or issues.

FR2: Support for Multiple Machine Learning Models

- Test `_define_models` to confirm all models (Logistic Regression, Random Forest, etc.) are correctly configured.
- Validate `_create_neural_network` by passing a sample input dimension and checking the model's layer structure and compilation.
- Test SVM configuration for `probability=True` and `class_weight='balanced'` to ensure proper setup.

FR3: Model Training and Evaluation

- Test `train_and_evaluate` with a subset of models, verifying that models train, save to .pkl files, and load correctly.
- Validate SMOTE resampling and data transformation using the preprocessing pipeline.
- Test metrics (accuracy, precision, recall, F1, AUC) for reasonable values and check storage in `self.results`.
- Simulate errors (e.g., invalid model ID) and verify logging and error handling.

FR4: Feature Importance Calculation

- Test `_calculate_feature_importance` for Logistic Regression and tree-based models, ensuring importance scores match feature names.
- Verify sorting of importance values and correct formatting as a list of dictionaries.
- Test error handling for models without feature importance support and mismatched feature lengths.

FR5: Performance Metrics and Visual Data Storage

- Test `get_confusion_matrix` to ensure the matrix and labels are returned correctly for each model.
- Validate `get_roc_curve` for correct FPR, TPR, thresholds, and AUC values in list format.

- Test `get_classification_report` to confirm the report dictionary contains expected precision, recall, and F1 scores.

FR6: Transaction Prediction Capability

- Test `predict_transaction` with sample transaction data, verifying prediction, probability, and risk level outputs.
- Validate `_preprocess_transaction` for correct handling of dates, address matching, and missing columns.
- Test edge cases (e.g., untrained model, missing data) and confirm error handling and logging.

FR7: Test Data Prediction and Analysis

- Test `predict_test_data` to ensure predictions, metrics, and visual data (confusion matrix, ROC curve) are computed correctly.
- Verify the test data sample (up to 100 examples) includes TP, TN, FP, and FN cases with correct feature values.
- Test error handling for untrained models and invalid data, checking logging output.

FR8: Best Model Selection

- Test `get_best_model` with multiple trained models, verifying the model with the highest AUC is returned.
- Test edge cases: no models trained, all models with errors, ensuring None is returned appropriately.
- Validate that the method correctly skips models with errors in `self.results`

CHAPTER 8

SOURCE CODE

1) Model_trainer.py

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import (
```

```
    accuracy_score, precision_score, recall_score, f1_score,
```

```
    roc_auc_score, confusion_matrix, classification_report,
```

```
    roc_curve, precision_recall_curve
```

```
)
```

```
from sklearn.preprocessing import Standard Scaler
```

```
from sklearn.pipeline import Pipeline
```

```
# TensorFlow temporarily disabled
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Dropout
```

```
from tensorflow.keras.optimizers import Adam
```

```
from imblearn.over_sampling import SMOTE
```

```
import joblib
```

```
import logging
```

```
import json
```

```
import tempfile
```

```
import os
```

```
import os
```

```
logger = logging.getLogger(__name__)
```

```
class Model Trainer:
```

```
    """
```

```
    Class for training and evaluating fraud detection models
```

```
    """
```

```
    def __init__(self, data_processor):
```

```
        """
```

```
        Initialize Model Trainer with data processor
```

```
        Args:
```

```
            data_processor: Data Processor instance with processed data
```

```
        """
```

```
        self.data_processor = data_processor
```

```
        self.models = {}
```

```
        self.results = {}
```

```
        self.trained_models = {}
```



```
self.feature_importances = {}

self.confusion_matrices = {}

self.roc_curves = {}


# Define available models

self._define_models()


def _define_models(self):

    """Define available models for fraud detection"""

    self.models = {

        'logistic_regression': {

            'name': 'Logistic Regression',

            'model': LogisticRegression(),

            'supports_feature_importance': True

        },

        'random_forest': {

            'name': 'Random Forest',

            'model': RandomForestClassifier(),

            'supports_feature_importance': True

        },

        'gradient_boosting': {

            'name': 'Gradient Boosting',

            'model': GradientBoostingClassifier(),
```

```
'supports_feature_importance': True

},

'svm': {

    'name': 'Support Vector Machine',

    'model': SVC(probability=True, class_weight='balanced', random_state=42),

    'supports_feature_importance': False

},

'neural_network': {

    'name': 'Neural Network',

    'model': self._create_neural_network,

    'supports_feature_importance': False

}

}
```

```
def _create_neural_network(self, input_dim):
```

```
    """
```

```
    Create a neural network model
```

```
    Args:
```

```
        input_dim (int): Number of input features
```

```
    Returns:
```

```
        tf.keras.models.Sequential: Neural network model
```

"""

```
model = Sequential([  
    Dense(64, activation='relu', input_dim=input_dim),  
    Dropout(0.2),  
    Dense(32, activation='relu'),  
    Dropout(0.2),  
    Dense(16, activation='relu'),  
    Dense(1, activation='sigmoid')  
])
```

```
model.compile(  
    optimizer=Adam(learning_rate=0.001),  
    loss='binary_crossentropy',  
    metrics=['accuracy', tf.keras.metrics.AUC()]  
)
```

```
return model
```

```
def get_available_models(self):
```

"""

Get available models for training

Returns:

list: List of available model names and details

"""

```
return [{ 'id': k, 'name': v['name']} for k, v in self.models.items()]
```

```
def train_and_evaluate(self, selected_models):
```

"""

Train and evaluate selected models. If models exist as .pkl files, load them instead.

Args:

selected_models (list): List of model IDs to train

Returns:

dict: Dictionary with model results

"""

```
if not selected_models:
```

```
    return {}
```

```
os.makedirs("models", exist_ok=True)
```

```

X_train_transformed =
self.data_processor.preprocessing_pipeline.transform(self.data_processor.X_train)

smote = SMOTE(random_state=42)
```

```
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_transformed,  
self.data_processor.y_train)
```

```
X_test_transformed =  
self.data_processor.preprocessing_pipeline.transform(self.data_processor.X_test)
```

```
self.results = {}
```

```
for model_id in selected_models:
```

```
    if model_id not in self.models:
```

```
        logger.warning(f"Model {model_id} not found, skipping")
```

```
        continue
```

```
model_file = os.path.join("models", f"{model_id}.pkl")
```

```
logger.info(f"Processing model: {self.models[model_id]['name']}")
```

```
try:
```

```
    if os.path.exists(model_file) and model_id != 'neural_network':
```

```
        logger.info(f"Loading saved model from {model_file}")
```

```
        model = joblib.load(model_file)
```

```
    else:
```

```
        if model_id == 'neural_network':
```

```
            input_dim = X_train_transformed.shape[1]
```

```
            model = self.models[model_id]['model'](input_dim)
```

```
early_stopping = tf.keras.callbacks.EarlyStopping(

    monitor='val_loss', patience=5, restore_best_weights=True

)

model.fit(

    X_train_resampled, y_train_resampled,

    epochs=25,

    batch_size=32,

    validation_split=0.2,

    callbacks=[early_stopping],

    verbose=0

)

else:

    model = self.models[model_id]['model']

    model.fit(X_train_resampled, y_train_resampled)

    joblib.dump(model, model_file)

    logger.info(f"Saved model to {model_file}")

# Predictions

if model_id == 'neural_network':

    y_pred = (model.predict(X_test_transformed) > 0.5).astype(int).flatten()

    y_pred_proba = model.predict(X_test_transformed).flatten()

else:
```

```
y_pred = model.predict(X_test_transformed)

y_pred_proba = model.predict_proba(X_test_transformed)[:, 1]


# Store trained model

self.trained_models[model_id] = model


# Metrics

metrics = self._calculate_metrics(self.data_processor.y_test, y_pred, y_pred_proba)

self.results[model_id] = {

    'name': self.models[model_id]['name'],

    'metrics': metrics

}


cm = confusion_matrix(self.data_processor.y_test, y_pred)

self.confusion_matrices[model_id] = cm.tolist()


fpr, tpr, thresholds = roc_curve(self.data_processor.y_test, y_pred_proba)

self.roc_curves[model_id] = {

    'fpr': fpr.tolist(),

    'tpr': tpr.tolist(),

    'thresholds': thresholds.tolist(),

    'auc': metrics['auc']

}
```

```
report = classification_report(self.data_processor.y_test, y_pred, output_dict=True)

self.classification_reports = getattr(self, 'classification_reports', {})

self.classification_reports[model_id] = report


if self.models[model_id]['supports_feature_importance']:

    self._calculate_feature_importance(model_id)


logger.info(f'Model {self.models[model_id]['name']} processed successfully')


except Exception as e:

    logger.error(f'Error processing model {model_id}: {str(e)}')

    self.results[model_id] = {

        'name': self.models[model_id]['name'],

        'error': str(e)

    }


return self.results


def _calculate_metrics(self, y_true, y_pred, y_pred_proba):

    """

    Calculate evaluation metrics for a model
```


Args:

y_true (array): True labels

y_pred (array): Predicted labels

y_pred_proba (array): Predicted probabilities

Returns:

dict: Dictionary with evaluation metrics

"""

metrics = {

 'accuracy': round(accuracy_score(y_true, y_pred), 4),

 'precision': round(precision_score(y_true, y_pred), 4),

 'recall': round(recall_score(y_true, y_pred), 4),

 'f1': round(f1_score(y_true, y_pred), 4),

 'auc': round(roc_auc_score(y_true, y_pred_proba), 4)

}

return metrics

def _calculate_feature_importance(self, model_id):

"""

Calculate feature importance for a model

Args:

```
model_id (str): ID of the model

"""

model = self.trained_models[model_id]

try:

    # Get feature importance based on model type

    if model_id == 'logistic_regression':

        # For logistic regression, feature importance is coefficient magnitude

        importance = np.abs(model.coef_[0])

    else:

        # For tree-based models

        importance = model.feature_importances_

    # Map importance to feature names

    feature_names = self.data_processor.feature_names

    if len(importance) == len(feature_names):

        importance_dict = dict(zip(feature_names, importance))

        # Sort by importance

        sorted_importance = sorted(

            importance_dict.items(),

            key=lambda x: x[1],
```

```
        reverse=True

    )

    # Convert to format for visualization

    self.feature_importances[model_id] = [

        {'feature': name, 'importance': float(imp)}

        for name, imp in sorted_importance

    ]

    else:

        logger.warning(

            f'Feature importance length mismatch: {len(importance)} !=

{len(feature_names)}'

        )

    except Exception as e:

        logger.error(f'Error calculating feature importance for {model_id}: {str(e)}')

def get_feature_importance(self, model_id):

    """

    Get feature importance for a model

    Args:

        model_id (str): ID of the model
```

Returns:

list: List of feature importance dictionaries

"""

if model_id not in self.feature_importances:

raise ValueError(f'Feature importance not available for model {model_id}')

return self.feature_importances[model_id]

def get_confusion_matrix(self, model_id):

"""

Get confusion matrix for a model

Args:

model_id (str): ID of the model

Returns:

list: Confusion matrix as a 2D list

"""

if model_id not in self.confusion_matrices:

raise ValueError(f'Confusion matrix not available for model {model_id}')

return {

'matrix': self.confusion_matrices[model_id],

```
'labels': ['Legitimate', 'Fraudulent']  
  
}
```

```
def get_roc_curve(self, model_id):
```

```
    """
```

Get ROC curve data for a model

Args:

model_id (str): ID of the model

Returns:

dict: Dictionary with ROC curve data

```
    """
```

```
    if model_id not in self.roc_curves:
```

```
        raise ValueError(f"ROC curve not available for model {model_id}")
```

```
    return self.roc_curves[model_id]
```

```
def get_classification_report(self, model_id):
```

```
    """
```

Get classification report for a model

Args:

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

model_id (str): ID of the model

Returns:

dict: Classification report as a dictionary

"""

if not hasattr(self, 'classification_reports') or model_id not in self.classification_reports:

raise ValueError(f'Classification report not available for model {model_id}')

return self.classification_reports[model_id]

def predict_transaction(self, transaction_data, model_id='random_forest'):

"""

Predict whether a transaction is fraudulent

Args:

transaction_data (dict): Dictionary containing transaction details

model_id (str): ID of the model to use for prediction

Returns:

dict: Prediction results including probability and binary prediction

"""

if model_id not in self.trained_models:

raise ValueError(f'Model {model_id} not trained. Please train the model first.')

```
try:

    # Create a dataframe from transaction data

    transaction_df = pd.DataFrame([transaction_data])


    # Process the transaction data

    processed_data = self._preprocess_transaction(transaction_df)


    # Transform data using preprocessing pipeline

    transformed_data =
self.data_processor.preprocessing_pipeline.transform(processed_data)


    # Make prediction

    model = self.trained_models[model_id]


    # For neural network models (when implemented)

    if model_id == 'neural_network':

        probability = model.predict(transformed_data)[0][0]

        prediction = int(probability > 0.5)

    else:

        # For scikit-learn models

        probability = model.predict_proba(transformed_data)[0][1]

        prediction = int(probability > 0.5)
```

```
    return {

        'prediction': prediction,

        'probability': round(float(probability), 4),

        'model': self.models[model_id]['name'],

        'is_fraudulent': 'Yes' if prediction == 1 else 'No',

        'risk_level': self._get_risk_level(probability)

    }

except Exception as e:

    logger.error(f'Error making prediction: {str(e)}")

    raise


def _preprocess_transaction(self, transaction_df):

    """

    Preprocess a transaction for prediction

    Args:

        transaction_df (DataFrame): DataFrame with transaction data

    Returns:

        DataFrame: Processed transaction data ready for prediction

    """
```



```
# Create a copy to avoid modifying the original

df = transaction_df.copy()


# Handle date if present

if 'Transaction Date' in df.columns:

    df['Transaction Date'] = pd.to_datetime(df['Transaction Date'])

    df['Transaction Day'] = df['Transaction Date'].dt.day

    df['Transaction Month'] = df['Transaction Date'].dt.month

    df['Transaction Day of Week'] = df['Transaction Date'].dt.dayofweek


# Process address match if both addresses are provided

if 'Shipping Address' in df.columns and 'Billing Address' in df.columns:

    df['Address Match'] = df.apply(

        lambda row: self.data_processor._compare_addresses(

            row['Shipping Address'], row['Billing Address']

        ),

        axis=1

    )

elif 'Address Match' not in df.columns:

    # If no address data, default to 0 (no match)

    df['Address Match'] = 0


# Keep only necessary columns for modeling
```

```
keep_columns = [  
    'Transaction Amount', 'Quantity', 'Customer Age',  
    'Transaction Hour', 'Account Age Days', 'Transaction Day',  
    'Transaction Month', 'Transaction Day of Week', 'Address Match',  
    'Payment Method', 'Product Category', 'Device Used'  
]  
  
# Filter columns that exist in the dataframe  
existing_columns = [col for col in keep_columns if col in df.columns]  
  
# Fill missing columns with default values  
for col in set(keep_columns) - set(existing_columns):  
    if col in ['Transaction Amount', 'Quantity', 'Customer Age', 'Transaction Hour',  
        'Account Age Days', 'Transaction Day', 'Transaction Month',  
        'Transaction Day of Week', 'Address Match']:  
        df[col] = 0  
    else:  
        df[col] = 'unknown'  
  
return df[keep_columns]  
  
def _get_risk_level(self, probability):  
    """
```

Convert fraud probability to risk level

Args:

probability (float): Fraud probability

Returns:

str: Risk level (Low, Medium, High, Very High)

"""

if probability < 0.25:

return 'Low'

elif probability < 0.5:

return 'Medium'

elif probability < 0.75:

return 'High'

else:

return 'Very High'

def _get_categorical_label(self, feature_name, value):

"""

Convert numeric value back to categorical label for display

Args:

feature_name (str): Name of the feature

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

value (float): Numeric value

Returns:

str: Categorical label

"""

Default mapping for common categories - this should be enhanced based on actual encoding

mappings = {

 'Payment Method': {

 0: 'Credit Card',

 1: 'PayPal',

 2: 'Apple Pay',

 3: 'Google Pay',

 4: 'Bank Transfer',

 5: 'Cryptocurrency'

 },

 'Product Category': {

 0: 'Electronics',

 1: 'Clothing',

 2: 'Home & Garden',

 3: 'Beauty',

 4: 'Sports',

 5: 'Toys',

```
        6: 'Jewelry',
        7: 'Digital Content',
        8: 'Health',
        9: 'Travel'
    },
    'Device Used': {
        0: 'Desktop',
        1: 'Mobile',
        2: 'Tablet'
    }
}

# Round to the nearest integer if needed

if isinstance(value, float):

    value = int(round(value))

# Try to get the label from the mapping

if feature_name in mappings and value in mappings[feature_name]:

    return mappings[feature_name][value]

else:

    # Return a placeholder if mapping is not available

    return f'{feature_name} {value}'
```

```
def predict_test_data(self, model_id):
```

```
    """
```

Predict fraud on the test dataset and return evaluation metrics

Args:

 model_id (str): ID of the model to use for prediction

Returns:

 dict: Dictionary containing predictions, metrics, and test data

```
    """
```

```
    if model_id not in self.trained_models:
```

```
        raise ValueError(f"Model {model_id} not trained. Please train the model first.")
```

```
    try:
```

```
        # Get the test data
```

```
        X_test = self.data_processor.X_test
```

```
        y_test = self.data_processor.y_test
```

```
        # Get the model
```

```
        model = self.trained_models[model_id]
```

```
        # Make predictions
```

```
        if model_id == 'neural_network':
```

```
y_pred_proba = model.predict(X_test).flatten()

y_pred = (y_pred_proba > 0.5).astype(int)

else:

    y_pred_proba = model.predict_proba(X_test)[:, 1]

    y_pred = model.predict(X_test)


# Calculate metrics

metrics = self._calculate_metrics(y_test, y_pred, y_pred_proba)


# Calculate confusion matrix

cm = confusion_matrix(y_test, y_pred).tolist()


# Calculate classification report

report = classification_report(y_test, y_pred, output_dict=True)


# Calculate ROC curve

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

roc_data = {

    'fpr': fpr.tolist(),

    'tpr': tpr.tolist(),

    'thresholds': thresholds.tolist(),

    'auc': metrics['auc']

}
```

```
# Create sample of test data with predictions for display (first 100 examples)

sample_size = min(100, len(X_test))

# Get the original feature names

feature_names = self.data_processor.feature_names

# Get a sample of raw test data if available

test_data_sample = []

# Get indices of true positives, true negatives, false positives, and false negatives

tp_indices = np.where((y_test == 1) & (y_pred == 1))[0]

tn_indices = np.where((y_test == 0) & (y_pred == 0))[0]

fp_indices = np.where((y_test == 0) & (y_pred == 1))[0]

fn_indices = np.where((y_test == 1) & (y_pred == 0))[0]

# Get samples from each category

sample_indices = []

for indices in [tp_indices, fp_indices, fn_indices, tn_indices]:

    if len(indices) > 0:

        # Take a proportional number of samples from each category

        n_samples = max(1, min(25, len(indices)))

        selected_indices = np.random.choice(indices, size=n_samples, replace=False)
```



```
sample_indices.extend(selected_indices)

# If we don't have enough samples, add more

if len(sample_indices) < sample_size:

    remaining = sample_size - len(sample_indices)

    all_indices = set(range(len(X_test)))

    unused_indices = list(all_indices - set(sample_indices))

    if unused_indices:

        additional_samples = np.random.choice(unused_indices,

                                                size=min(remaining, len(unused_indices)),

                                                replace=False)

        sample_indices.extend(additional_samples)

# Limit to sample_size

sample_indices = sample_indices[:sample_size]

# Get the sample data

for idx in sample_indices:

    try:

        # Create a safe copy of the features for display

        feature_dict = {}

        for i, feature_name in enumerate(feature_names):

            # Ensure values are safely converted to the appropriate type for JSON
```

```
value = X_test[idx][i]

# Handle categorical features (non-numeric values)

if feature_name in ['Payment Method', 'Product Category', 'Device Used']:

    # These might be one-hot encoded in X_test, so we need to handle differently

    if isinstance(value, (int, float, np.integer, np.floating)):

        # If it's a numeric value from encoding, use a placeholder

        feature_dict[feature_name] = self._get_categorical_label(feature_name,
value)

    else:

        feature_dict[feature_name] = str(value)

else:

    # Numeric features

    feature_dict[feature_name] = float(value) if isinstance(value, (int, float,
np.integer, np.floating)) else str(value)

sample = {

    'features': feature_dict,

    'actual': int(y_test[idx]),

    'predicted': int(y_pred[idx]),

    'probability': float(y_pred_proba[idx]),

    'correct': y_test[idx] == y_pred[idx],

    'category': self._get_prediction_category(y_test[idx], y_pred[idx])

}
```

```
        test_data_sample.append(sample)

    except Exception as e:

        logger.error(f'Error processing test sample at index {idx}: {str(e)}')

        # Continue with other samples if one fails

# Sort by prediction category for easier viewing
test_data_sample.sort(key=lambda x: x['category'])

return {

    'metrics': metrics,

    'confusion_matrix': {

        'matrix': cm,

        'labels': ['Legitimate', 'Fraudulent']

    },

    'classification_report': report,

    'roc_curve': roc_data,

    'test_data_sample': test_data_sample,

    'model_name': self.models[model_id]['name']

}

except Exception as e:

    logger.error(f'Error making test data predictions: {str(e)}')

    raise
```

```
def _get_prediction_category(self, actual, predicted):
```

```
    """
```

```
    Get the prediction category (TP, TN, FP, FN)
```

```
    Args:
```

```
        actual (int): Actual class
```

```
        predicted (int): Predicted class
```

```
    Returns:
```

```
        str: Category (TP, TN, FP, FN)
```

```
    """
```

```
    if actual == 1 and predicted == 1:
```

```
        return 'TP' # True Positive
```

```
    elif actual == 0 and predicted == 0:
```

```
        return 'TN' # True Negative
```

```
    elif actual == 0 and predicted == 1:
```

```
        return 'FP' # False Positive
```

```
    else: # actual == 1 and predicted == 0
```

```
        return 'FN' # False Negative
```

```
def get_best_model(self):
```

```
    """
```

Get the best performing model based on AUC score

Returns:

str: ID of the best performing model

"""

if not self.results:

return None

best_model = None

best_auc = -1

for model_id, result in self.results.items():

if 'error' in result:

continue

auc = result['metrics']['auc']

if auc > best_auc:

best_auc = auc

best_model = model_id

return best_model

2) Visulization.py

```
import pandas as pd

import numpy as np

import plotly.express as px

import plotly.graph_objects as go

import plotly.figure_factory as ff

from plotly.subplots import make_subplots

import json

import logging

import traceback


# Configure logger

logger = logging.getLogger(__name__)

logging.basicConfig(level=logging.DEBUG)


# Custom JSON encoder for NumPy types

class NpEncoder(json.JSONEncoder):

    def default(self, o):

        if isinstance(o, np.integer):

            return int(o)

        if isinstance(o, np.floating):

            return float(o)

        if isinstance(o, np.ndarray):

            return o.tolist()
```

```
return super(NpEncoder, self).default(o)
```

```
class Visualization:
```

```
    """
```

```
    Class for creating visualizations for e-commerce fraud detection
```

```
    """
```

```
    def __init__(self, data_processor):
```

```
        """
```

```
        Initialize visualization class with data processor
```

```
        Args:
```

```
            data_processor: Data Processor instance with loaded data
```

```
        """
```

```
        self.data_processor = data_processor
```

```
        self.raw_data = data_processor.raw_data
```

```
        self.processed_data = data_processor.processed_data
```

```
    def get_fraud_distribution(self):
```

```
        """
```

```
        Generate fraud distribution visualization with bar graphs before and after SMOTE
```

```
        Returns:
```

```
            str: JSON string of plotly figure for fraud distribution
```

"""

try:

```
logger.debug("Starting fraud distribution visualization")
```

```
# Check if raw data is available
```

```
if self.raw_data is None:
```

```
    logger.error("Raw data is None in get_fraud_distribution")
```

```
    return json.dumps({})
```

```
# Get original data distribution
```

```
fraud_counts = self.raw_data['Is Fraudulent'].value_counts()
```

```
logger.debug(f'Fraud counts shape: {fraud_counts.shape}')
```

```
# Get count values - ensure Python native types
```

```
legitimate_count = int(fraud_counts.get(0, 0))
```

```
fraudulent_count = int(fraud_counts.get(1, 0))
```

```
total_count = legitimate_count + fraudulent_count
```

```
# Calculate percentages
```

```
legitimate_pct = round((legitimate_count / total_count) * 100, 1)
```

```
fraudulent_pct = round((fraudulent_count / total_count) * 100, 1)
```

```
# Create the figure directly with go
```



```
fig = go.Figure()

# Add original data bars

fig.add_trace(go.Bar(

    x=['Legitimate', 'Fraudulent'],

    y=[legitimate_count, fraudulent_count],

    name='Original Data',

    marker_color='#3498db'

))

# Add SMOTE data bars (equal distribution)

fig.add_trace(go.Bar(

    x=['Legitimate', 'Fraudulent'],

    y=[legitimate_count, legitimate_count], # Equal counts after SMOTE

    name='After SMOTE',

    marker_color='#e74c3c'

))

# Update layout

fig.update_layout(

    title='Transaction Distribution: Original vs SMOTE Balanced',

    xaxis_title='Transaction Type',

    yaxis_title='Count',
```

```
barmode='group',

height=400,

legend=dict(

    orientation="h",

    yanchor="bottom",

    y=1.02,

    xanchor="right",

    x=1

),

margin=dict(l=40, r=20, t=60, b=40),

annotations=[

    dict(

        x='Legitimate',

        y=legitimate_count,

        text=f'{legitimate_pct}%',

        showarrow=False,

        yshift=10

    ),

    dict(

        x='Fraudulent',

        y=fraudulent_count,

        text=f'{fraudulent_pct}%',

        showarrow=False,
```

```
        yshift=10

    ),

    dict(

        x='Fraudulent',

        y=legitimate_count,

        text="50%",

        showarrow=False,

        yshift=10

    ),

    dict(

        x='Legitimate',

        y=legitimate_count,

        text="50%",

        showarrow=False,

        yshift=10,

        xshift=30

    )

]

)
```



```
# Add an annotation explaining SMOTE

fig.add_annotation(

    xref="paper", yref="paper",
```

```
x=0.5, y=-0.15,
```

```
text="SMOTE (Synthetic Minority Over-sampling Technique) is used to balance the  
class distribution<br>by generating synthetic samples of the minority class.",
```

```
showarrow=False,
```

```
font=dict(size=10),
```

```
align="center"
```

```
)
```

```
# Convert figure to JSON
```

```
result = json.dumps(fig.to_dict(), cls=NpEncoder)
```

```
logger.debug(f'Fraud distribution visualization JSON length: {len(result)}')
```

```
return result
```

```
except Exception as e:
```

```
logger.error(f'Error creating fraud distribution visualization: {str(e)}')
```

```
logger.error(traceback.format_exc())
```

```
return json.dumps({})
```

```
def get_correlation_plot(self):
```

```
"""
```

```
Generate correlation heatmap for numerical features
```

```
Returns:
```

str: JSON string of plotly figure for correlation heatmap

"""

try:

Get numerical columns only

numerical_cols = [

'Transaction Amount', 'Quantity', 'Customer Age',

'Transaction Hour', 'Account Age Days', 'Is Fraudulent'

]

Filter to include only columns that exist in the data

available_cols = [col for col in numerical_cols if col in self.raw_data.columns]

Calculate correlation matrix

corr_matrix = self.raw_data[available_cols].corr().round(2)

Convert numpy values to Python native types for JSON serialization

z_values = corr_matrix.values.tolist()

x_values = corr_matrix.columns.tolist()

y_values = corr_matrix.index.tolist()

Create text annotations

annotations = []

for i, row in enumerate(z_values):

```
for j, value in enumerate(row):

    annotations.append({

        "xref": "x1",

        "yref": "y1",

        "x": x_values[j],

        "y": y_values[i],

        "text": str(round(value, 2)),

        "font": {"color": "white" if abs(value) > 0.5 else "black"},

        "showarrow": False

    })

# Create heatmap

fig = go.Figure(data=go.Heatmap(

    z=z_values,

    x=x_values,

    y=y_values,

    colorscale='Viridis',

    zmin=-1,

    zmax=1

))

fig.update_layout(

    title='Correlation Matrix of Numerical Features',
```

```
        height=500,

        margin=dict(l=20, r=20, t=50, b=20),

        annotations=annotations,

        template='plotly_dark',

        paper_bgcolor='rgba(0, 0, 0, 0)',

        plot_bgcolor='rgba(0, 0, 0, 0)'

    )

    return json.dumps(fig.to_dict(), cls=NpEncoder)

except Exception as e:

    logger.error(f"Error creating correlation visualization: {str(e)}")

    return json.dumps({})

def get_time_series_plot(self):

    """

    Generate time series visualization of transactions by date

    Returns:

        str: JSON string of plotly figure for time series

    """

    try:

        if self.raw_data is None:

            logger.error("Raw data is None in get_time_series_plot")
```

```
    return json.dumps({})

# Check if Transaction Date column exists

if 'Transaction Date' not in self.raw_data.columns:

    logger.error("Transaction Date column not found in raw data")

    return json.dumps({})

# Convert to datetime if it's not already

if not pd.api.types.is_datetime64_any_dtype(self.raw_data['Transaction Date']):

    date_series = pd.to_datetime(self.raw_data['Transaction Date'], errors='coerce')

else:

    date_series = self.raw_data['Transaction Date']

# Create date column and count transactions by date

self.raw_data['Date'] = date_series.dt.date

daily_counts = self.raw_data.groupby(['Date', 'Is Fraudulent']).size().reset_index()

daily_counts.columns = ['Date', 'Is Fraudulent', 'Count']

# Label the fraud status

daily_counts['Fraud Status'] = daily_counts['Is Fraudulent'].map({0: 'Legitimate', 1:
'Fraudulent'})

# Create line chart
```



```
fig = go.Figure()

# Add traces for legitimate and fraudulent transactions

for fraud_status in ['Legitimate', 'Fraudulent']:

    filtered_data = daily_counts[daily_counts['Fraud Status'] == fraud_status]

    fig.add_trace(go.Scatter(

        x=filtered_data['Date'],

        y=filtered_data['Count'],

        mode='lines+markers',

        name=fraud_status,

        marker_color='#3498db' if fraud_status == 'Legitimate' else '#e74c3c'

    ))

# Update layout

fig.update_layout(

    title='Transaction Volume Over Time',

    xaxis_title='Date',

    yaxis_title='Number of Transactions',

    legend=dict(

        orientation="h",

        yanchor="bottom",

        y=1.02,

        xanchor="right",
```

```
x=1

),

height=400,

margin=dict(l=40, r=20, t=60, b=40)

)

result = json.dumps(fig.to_dict(), cls=NpEncoder)

return result

except Exception as e:

    logger.error(f'Error creating time series plot: {str(e)}')

    logger.error(traceback.format_exc())

    return json.dumps({})

def get_payment_method_distribution(self):

    """

    Generate visualization of transaction distribution by payment method

    Returns:

        str: JSON string of plotly figure for payment method distribution

    """

    try:

        if self.raw_data is None:
```

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

```
logger.error("Raw data is None in get_payment_method_distribution")

return json.dumps({})


# Check if Payment Method column exists

if 'Payment Method' not in self.raw_data.columns:

    logger.error("Payment Method column not found in raw data")

    return json.dumps({})


# Group by payment method and fraud status

payment_counts = self.raw_data.groupby(['Payment Method', 'Is
Fraudulent']).size().reset_index()

payment_counts.columns = ['Payment Method', 'Is Fraudulent', 'Count']


# Label the fraud status

payment_counts['Fraud Status'] = payment_counts['Is Fraudulent'].map({0: 'Legitimate',
1: 'Fraudulent'})


# Create bar chart

fig = px.bar(

    payment_counts,

    x='Payment Method',

    y='Count',

    color='Fraud Status',
```

```
barmode='group',

color_discrete_map={'Legitimate': '#3498db', 'Fraudulent': '#e74c3c'},

title='Transaction Distribution by Payment Method'

)

# Calculate fraud rate for each payment method

fraud_rates = []

for method in payment_counts['Payment Method'].unique():

    method_data = payment_counts[payment_counts['Payment Method'] == method]

    total = method_data['Count'].sum()

    fraud = method_data[method_data['Fraud Status'] == 'Fraudulent']['Count'].sum() if
'Fraudulent' in method_data['Fraud Status'].values else 0

    fraud_rate = round((fraud / total) * 100, 1) if total > 0 else 0

    fraud_rates.append({

        'method': method,

        'rate': fraud_rate,

        'y_pos': method_data['Count'].max() + 50 # Position above the highest bar

    })

# Add fraud rate annotations

for item in fraud_rates:

    fig.add_annotation(

        x=item['method'],
```

```
y=item['y_pos'],

text=f'Fraud: {item['rate']}%',

showarrow=False,

font=dict(size=10, color='#e74c3c', family="Arial Black")

)

# Update layout

fig.update_layout(

    xaxis_title='Payment Method',

    yaxis_title='Number of Transactions',

    legend=dict(

        orientation="h",

        yanchor="bottom",

        y=1.02,

        xanchor="right",

        x=1

    ),

    height=400,

    margin=dict(l=40, r=20, t=60, b=40)

)

result = json.dumps(fig.to_dict(), cls=NpEncoder)

return result
```

except Exception as e:

```
logger.error(f'Error creating payment method distribution: {str(e)}')
```

```
logger.error(traceback.format_exc())
```

```
return json.dumps({})
```

```
def get_product_category_distribution(self):
```

```
    """
```

Generate visualization of transaction distribution by product category

Returns:

str: JSON string of plotly figure for product category distribution

```
    """
```

```
    try:
```

```
        if self.raw_data is None:
```

```
            logger.error("Raw data is None in get_product_category_distribution")
```

```
            return json.dumps({})
```

```
        # Check if Product Category column exists
```

```
        if 'Product Category' not in self.raw_data.columns:
```

```
            logger.error("Product Category column not found in raw data")
```

```
            return json.dumps({})
```

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

```
# Count transactions by product category
```

```
category_counts = self.raw_data['Product Category'].value_counts().reset_index()
```

```
category_counts.columns = ['Product Category', 'Count']
```

```
# Create pie chart
```

```
fig = go.Figure(data=[go.Pie(
```

```
    labels=category_counts['Product Category'],
```

```
    values=category_counts['Count'],
```

```
    hole=0.4,
```

```
    marker_colors=px.colors.qualitative.Set3
```

```
))
```

```
# Calculate fraud rate by category
```

```
    fraud_by_category = self.raw_data[self.raw_data['Is Fraudulent'] == 1]['Product  
Category'].value_counts()
```

```
# Add annotations for fraud rate on each slice
```

```
annotations = []
```

```
for i, category in enumerate(category_counts['Product Category']):
```

```
    total = category_counts.loc[category_counts['Product Category'] == category,  
'Count'].values[0]
```

```
    fraud = fraud_by_category.get(category, 0)
```

```
    fraud_rate = round((fraud / total) * 100, 1)
```

```
        annotations.append(dict(

            text=f'{fraud_rate}% fraud',

            x=0.5,

            y=0.5,

            font=dict(size=8, color='#e74c3c'),

            showarrow=True,

            arrowhead=7,

            ax=0,

            ay=0

        ))

# Update layout

fig.update_layout(

    title='Transaction Distribution by Product Category',

    height=400,

    showlegend=True,

    margin=dict(l=20, r=20, t=50, b=20)

)

result = json.dumps(fig.to_dict(), cls=NpEncoder)

return result
```


except Exception as e:

```
logger.error(f'Error creating product category distribution: {str(e)}')
```

```
logger.error(traceback.format_exc())
```

```
return json.dumps({})
```

```
def get_transaction_hour_distribution(self):
```

```
    """
```

Generate visualization of transaction distribution by hour of day

Returns:

str: JSON string of plotly figure for hour distribution

```
    """
```

```
    try:
```

```
        if self.raw_data is None:
```

```
            logger.error("Raw data is None in get_transaction_hour_distribution")
```

```
            return json.dumps({})
```

```
        # Check if Transaction Hour column exists
```

```
        if 'Transaction Hour' not in self.raw_data.columns:
```

```
            logger.error("Transaction Hour column not found in raw data")
```

```
            return json.dumps({})
```

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

```
# Group by hour and fraud status

hour_data = self.raw_data.groupby(['Transaction Hour', 'Is
Fraudulent']).size().reset_index()

hour_data.columns = ['Hour', 'Is Fraudulent', 'Count']

# Label the fraud status

hour_data['Fraud Status'] = hour_data['Is Fraudulent'].map({0: 'Legitimate', 1:
'Fraudulent'})

# Create bar chart

fig = px.bar(

    hour_data,

    x='Hour',

    y='Count',

    color='Fraud Status',

    barmode='group',

    color_discrete_map={'Legitimate': '#3498db', 'Fraudulent': '#e74c3c'},

    title='Transaction Distribution by Hour of Day'

)

# Calculate fraud rate for each hour

hour_totals = hour_data.groupby('Hour')['Count'].sum().reset_index()
```

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

```
hour_fraud = hour_data[hour_data['Fraud Status'] ==
'Fraudulent'].groupby('Hour')['Count'].sum().reset_index()

hour_fraud.columns = ['Hour', 'Fraud Count']

# Merge to get fraud percentage

hour_analysis = pd.merge(hour_totals, hour_fraud, on='Hour', how='left')

hour_analysis['Fraud Count'] = hour_analysis['Fraud Count'].fillna(0)

hour_analysis['Fraud Rate'] = round((hour_analysis['Fraud Count'] /
hour_analysis['Count']) * 100, 1)

# Add line trace for fraud rate

fig.add_trace(go.Scatter(

    x=hour_analysis['Hour'],

    y=hour_analysis['Fraud Rate'],

    mode='lines+markers',

    name='Fraud Rate (%)',

    yaxis='y2',

    line=dict(color='#2ecc71', width=3),

    marker=dict(size=8)

))

# Update layout for dual y-axis

fig.update_layout()
```

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

```
xaxis_title='Hour of Day (24h format)',

yaxis_title='Number of Transactions',

yaxis2=dict(

    title='Fraud Rate (%)',

    titlefont=dict(color='#2ecc71'),

    tickfont=dict(color='#2ecc71'),

    anchor="x",

    overlaying="y",

    side="right",

    range=[0, max(hour_analysis['Fraud Rate']) * 1.2] # Give some headroom

),

legend=dict(

    orientation="h",

    yanchor="bottom",

    y=1.02,

    xanchor="right",

    x=1

),

height=400,

margin=dict(l=40, r=40, t=60, b=40)

)

# Ensure hours are in order from 0-23
```

```
fig.update_xaxes(

    categoryorder='array',

    categoryarray=list(range(24))

)

result = json.dumps(fig.to_dict(), cls=NpEncoder)

return result

except Exception as e:

    logger.error(f'Error creating hour distribution: {str(e)}")

    logger.error(traceback.format_exc())

    return json.dumps({})
```

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Implementation and Description

1. Function Name: `initialize_components`

Implementation : This function sets up the core components of the application when the Flask app starts. It begins by declaring global variables for the data processor, visualization, and model trainer. It attempts to load a dataset from a specific CSV file located in the 'Dataset' folder, named 'Fraudulent_E-Commerce_Transaction_Data_2.csv'. Using this file, it creates a Data Processor instance and processes the data to prepare it for analysis and modeling. Next, it initializes a Visualization object with the data processor to enable chart generation. Then, it creates a Model Trainer instance with the same data processor to handle model training and predictions. If everything works, it logs a success message. If any error occurs during this setup—such as a missing file or processing failure—it catches the exception, logs the error with details, and allows the app to continue, though components may remain uninitialized.

2. Function Name: `index`

Implementation : This route handles requests to the main page of the app, the root URL ('/'). It first checks if the data processor is initialized. If not, it displays an error message to the user via a flash notification, indicating that the data failed to load and suggesting they check the logs. It then renders a basic template, 'index.html', passing a flag to show the data isn't loaded. If the data processor is ready, it retrieves basic statistics about the dataset—such as counts, averages, or other summary metrics—from the data processor. It then renders a dashboard template, 'dashboard.html', passing these stats and a flag confirming the data is loaded, allowing the user to see an overview of the dataset on the main page.

3. Function Name: `eda`

Implementation : This route serves the exploratory data analysis (EDA) page at the '/eda' URL. It starts by checking if the visualization component is initialized. If not, it flashes an error message to the user, warning that the visualization system isn't ready, and redirects them to the main page. If ready, it tries to generate key visualizations: a fraud distribution chart comparing legitimate and fraudulent transactions before and after balancing, and a correlation plot

showing relationships between numerical features. It also creates additional charts for transaction distributions by payment method, product category, and hour of the day. It retrieves basic dataset statistics from the data processor for context. These visualizations and stats are passed to the 'eda.html' template for rendering, displaying interactive charts to the user. If any error occurs during this process—such as missing data or a visualization failure—it logs the error, flashes a message to the user, and redirects them back to the main page.

4. Function Name: `models_page`

Implementation : This route handles the '/models' URL, displaying a page for managing machine learning models. It first checks if the model trainer component is initialized. If not, it flashes an error message to the user and redirects them to the main page. If the model trainer is ready, it retrieves a list of available models—each with an ID and name—from the model trainer. This list is passed to the 'models.html' template, which renders a page where users can see all available models (e.g., Logistic Regression, Random Forest) and select which ones to train, providing a simple interface for model management.

5. Function Name: `train_model`

Implementation : This route handles POST requests to '/train_model', triggered when a user submits a form to train selected models. It begins by extracting the list of model IDs chosen by the user from the form data. If no models are selected, it flashes a warning message to the user, asking them to choose at least one model, and redirects them back to the models page. If models are selected, it attempts to train and evaluate the chosen models using the model trainer, which processes the data, trains each model, and computes performance metrics like accuracy or AUC. The results are passed to a 'compare.html' template, which renders a page to display the performance of each trained model for comparison. If any error occurs during training—such as a model failing to fit—it logs the error, flashes a message to the user, and redirects them back to the models page.

6. Function Name: `feature_importance`

Implementation : This API route responds to requests at '/api/feature_importance/<model_name>', where the model name is a variable in the URL. It attempts to retrieve feature importance data for the specified model from the model trainer, which provides a list of features and their importance scores (e.g., how much each feature contributes to fraud prediction). If successful, it converts this data to JSON and returns it to the

caller, likely for display in a chart or table on the frontend. If an error occurs—such as the model not being trained or not supporting feature importance—it logs the error and returns a JSON response with the error message and a 500 status code, indicating a server-side issue.

7. Function Name: `confusion_matrix`

Implementation : This API route handles requests to `/api/confusion_matrix/<model_name>`, taking a model name from the URL. It tries to fetch the confusion matrix for the specified model from the model trainer, which returns a 2D list showing true positives, true negatives, false positives, and false negatives for fraud detection. If successful, it converts this matrix and its labels (e.g., 'Legitimate', 'Fraudulent') to JSON and sends it back to the caller for visualization or analysis. If an error occurs—such as the model not being trained—it logs the error and returns a JSON error message with a 500 status code, signaling a server failure.

8. Function Name: `roc_curve`

Implementation : This API route processes requests to `/api/roc_curve/<model_name>`, using the model name from the URL. It attempts to retrieve ROC curve data from the model trainer, including false positive rates, true positive rates, thresholds, and the area under the curve (AUC) score, which measures model performance in distinguishing fraud. If successful, it converts this data to JSON and returns it to the caller, likely for plotting an ROC curve on the frontend. If an error occurs—such as the model not being trained or data issues—it logs the error and returns a JSON error message with a 500 status code, indicating a server error.

9. Function Name: `classification_report`

Implementation : This API route handles requests to `/api/classification_report/<model_name>`, accepting a model name in the URL. It tries to get a classification report from the model trainer, which includes metrics like precision, recall, and F1-score for each class (legitimate and fraudulent). If successful, it converts this report to JSON and sends it back to the caller for display or analysis. If an error occurs—such as the model not being trained—it logs the error and returns a JSON error message with a 500 status code, indicating a server-side problem.

10. Function Name: `predict`

Implementation : This route handles both GET and POST requests to `/predict`, supporting single transaction fraud prediction. It first checks if the model trainer is initialized; if not, it

flashes an error and redirects to the main page. It builds a list of trained models (with IDs and names) from the model trainer. If no models are trained, it flashes a warning and redirects to the models page. It also identifies the best model based on performance (e.g., AUC). For a GET request, it renders 'predict.html', passing the list of trained models and the best model for a form where users can input transaction details. For a POST request, it extracts transaction data from the form—such as amount, quantity, customer age, and payment method—converting values to appropriate types (e.g., float, int). It selects the model ID from the form or defaults to the best model. It calls the model trainer to predict fraud, getting a result with the prediction, probability, and risk level. The result, transaction data, trained models, and selected model are passed to 'prediction_result.html' for display. If an error occurs, it logs the issue, flashes an error, and re-renders the prediction form.

11. Function Name: test_data_prediction

Implementation : This route handles GET and POST requests to '/test_data_prediction', enabling predictions on the test dataset. It checks if the model trainer and data processor are initialized; if not, it flashes an error and redirects to the main page. It builds a list of trained models with IDs and names. If no models are trained, it flashes a warning and redirects to the models page. It identifies the best model from the model trainer. For a GET request, it renders 'test_data_prediction.html', passing the trained models and best model for selection. For a POST request, it gets the model ID from the form, defaulting to the best model if none is specified. It calls the model trainer to predict on the test data, producing metrics, a confusion matrix, ROC curve, and a sample of predictions. These results, along with the model ID, name, and trained models, are passed to 'test_prediction_result.html' for display. If an error occurs, it logs the issue, flashes an error, and re-renders the form.

12. Function Name: api_predict

Implementation : This API route handles POST requests to '/api/predict' for programmatic fraud prediction. It checks if the model trainer is initialized; if not, it returns a JSON error with a 500 status code. It extracts JSON data from the request, representing a transaction. If no data is provided, it returns a JSON error with a 400 status code. It looks for a model ID in the JSON; if absent, it uses the best model from the model trainer. If no model ID is provided and no best model exists, it checks for any trained models; if none are available, it returns a JSON error with a 400 status code. It then calls the model trainer to predict fraud for the transaction, getting a result with the prediction, probability, and risk level. This result is converted to JSON and

returned to the caller. If any error occurs—such as invalid data or prediction failure—it logs the error and returns a JSON error message with a 500 status code.

13. Function Name: Main Execution Block (if name == 'main')

Implementation : This block runs when the script is executed directly, not imported as a module. It starts the Flask application, configuring it to listen on all network interfaces (host '0.0.0.0') and port 5000. It enables debug mode, which provides detailed error messages and automatic reloading during development. This launches the web server, making the app accessible for users to interact with the dashboard, EDA, model training, predictions, and API endpoints.

9.2 Dataset Description

Overview: This dataset contains 7 records of e-commerce transactions, likely intended for analyzing and detecting fraudulent activity. Each row represents a single transaction, capturing details about the customer, the transaction itself, and contextual factors such as device and location. The data includes a label indicating whether each transaction is fraudulent, making it suitable for supervised machine learning tasks like training fraud detection models.

Structure:

- **Number of Rows:** 7 transactions
- **Number of Columns:** 16
- **Purpose:** The dataset is designed for fraud detection, allowing analysis of patterns in transaction amounts, payment methods, customer behavior, and other features to distinguish legitimate from fraudulent transactions.

Columns and Descriptions:

- **Transaction ID**
 - **Description:** A unique identifier for each transaction, likely a UUID (Universally Unique Identifier).
 - **Data Type:** String (e.g., "c12e07a0-8a06-4c0d-b5cc-04f3af688570")
 - **Purpose:** Uniquely tracks each transaction for reference and analysis.

- **Observation:** All values are unique, as expected for an ID.
- **Customer ID**
 - **Description:** A unique identifier for each customer, also in UUID format.
 - **Data Type:** String (e.g., "8ca9f102-02a4-4207-ab63-484e83a1bdf0")
 - **Purpose:** Links transactions to specific customers, useful for tracking repeat behavior.
 - **Observation:** All customer IDs are unique, suggesting no repeat customers in this sample.
- **Transaction Amount**
 - **Description:** The monetary value of the transaction in an unspecified currency.
 - **Data Type:** Float (e.g., 42.32, 301.34)
 - **Range:** 42.32 to 345.27
 - **Purpose:** Indicates the size of the transaction, often a key factor in fraud detection (e.g., unusually high amounts may signal fraud).
 - **Observation:** Values vary widely, with most between 42.32 and 345.27, no extreme outliers in this small sample.
- **Transaction Date**
 - **Description:** The date and time when the transaction occurred.
 - **Data Type:** String in the format "DD-MM-YYYY HH:MM" (e.g., "24-03-2024 23:42")
 - **Range:** January 16, 2024, to March 24, 2024
 - **Purpose:** Enables time-based analysis, such as identifying fraud patterns by day, month, or time of day.
 - **Observation:** Dates span a few months, with transactions occurring at various times.

- **Payment Method**

- **Description:** The method used to pay for the transaction.
- **Data Type:** String (categorical)
- **Values:** "PayPal", "credit card", "debit card"
- **Purpose:** Helps analyze whether certain payment methods are more prone to fraud.
- **Observation:** Three methods appear: credit card (3 transactions), PayPal (2), debit card (2).

- **Product Category**

- **Description:** The category of the purchased product.
- **Data Type:** String (categorical)
- **Values:** "electronics", "toys & games", "clothing"
- **Purpose:** Identifies product types, useful for detecting if fraud is more common in specific categories.
- **Observation:** Electronics (3 transactions), toys & games (3), and clothing (1) are present.

- **Quantity**

- **Description:** The number of items purchased in the transaction.
- **Data Type:** Integer
- **Range:** 1 to 5
- **Purpose:** Indicates purchase volume, potentially relevant for fraud (e.g., bulk orders might be suspicious).
- **Observation:** Most transactions involve 1 (2 transactions), 3 (2), or 5 (3) items.

- **Customer Age**

- **Description:** The age of the customer at the time of the transaction.
- **Data Type:** Integer
- **Range:** 9 to 45
- **Purpose:** Helps analyze if age correlates with fraud (e.g., younger or older customers might be targeted).
- **Observation:** Ages vary, with one outlier at 9 (possibly a data entry error or child account), others between 29 and 45.
- **Customer Location**
 - **Description:** The location of the customer, likely a city or town name.
 - **Data Type:** String (e.g., "East Jameshaven", "Kingstad")
 - **Purpose:** Useful for geographic analysis of fraud patterns, though may be fictional or anonymized.
 - **Observation:** All locations are unique, with names suggesting varied regions (e.g., "North Ryan", "Kaylerville").
- **Device Used**
 - **Description:** The type of device used for the transaction.
 - **Data Type:** String (categorical)
 - **Values:** "desktop", "tablet", "mobile"
 - **Purpose:** Identifies if certain devices (e.g., mobile) are more associated with fraud.
 - **Observation:** Desktop (4 transactions), mobile (2), and tablet (1) are represented.
- **IP Address**
 - **Description:** The IP address from which the transaction was made.
 - **Data Type:** String (e.g., "110.87.246.85")

- **Purpose:** Can be used to trace transaction origins or detect suspicious patterns (e.g., multiple transactions from one IP).
- **Observation:** All IPs are unique, with no obvious pattern in this small sample.
- **Shipping Address**
 - **Description:** The address where the product is to be shipped.
 - **Data Type:** String (e.g., "5399 Rachel Stravenue Suite 718, North Blakeburgh, IL 78600")
 - **Purpose:** Allows comparison with billing address to detect mismatches, a common fraud indicator.
 - **Observation:** Addresses are detailed, some match billing addresses, others don't.
- **Billing Address**
 - **Description:** The address associated with the payment method.
 - **Data Type:** String (e.g., "4772 David Stravenue Apt. 447, Velasquezside, KS 67650")
 - **Purpose:** Used to check for consistency with shipping address, a key fraud detection feature.
 - **Observation:** Five transactions have matching shipping and billing addresses; one does not.
- **Is Fraudulent**
 - **Description:** A label indicating whether the transaction is fraudulent.
 - **Data Type:** Integer (binary)
 - **Values:** 0 (not fraudulent), 1 (fraudulent)
 - **Purpose:** Serves as the target variable for training and evaluating fraud detection models.

- **Observation:** All 7 transactions are labeled 0 (not fraudulent), suggesting an imbalanced sample or no fraud in this subset.
- **Account Age Days**
 - **Description:** The number of days since the customer's account was created.
 - **Data Type:** Integer
 - **Range:** 116 to 360
 - **Purpose:** Indicates account age, as newer accounts might be more likely to be fraudulent.
 - **Observation:** Values range widely, suggesting a mix of new and established accounts.
- **Transaction Hour**
 - **Description:** The hour of the day (0-23) when the transaction occurred.
 - **Data Type:** Integer
 - **Range:** 0 to 23
 - **Purpose:** Helps identify time-based patterns, as fraud may occur more at certain hours (e.g., late night).
 - **Observation:** Hours are spread across the day, with 0, 8, 13, 15, 20, and 23 appearing.

9.3 Results and Discussion

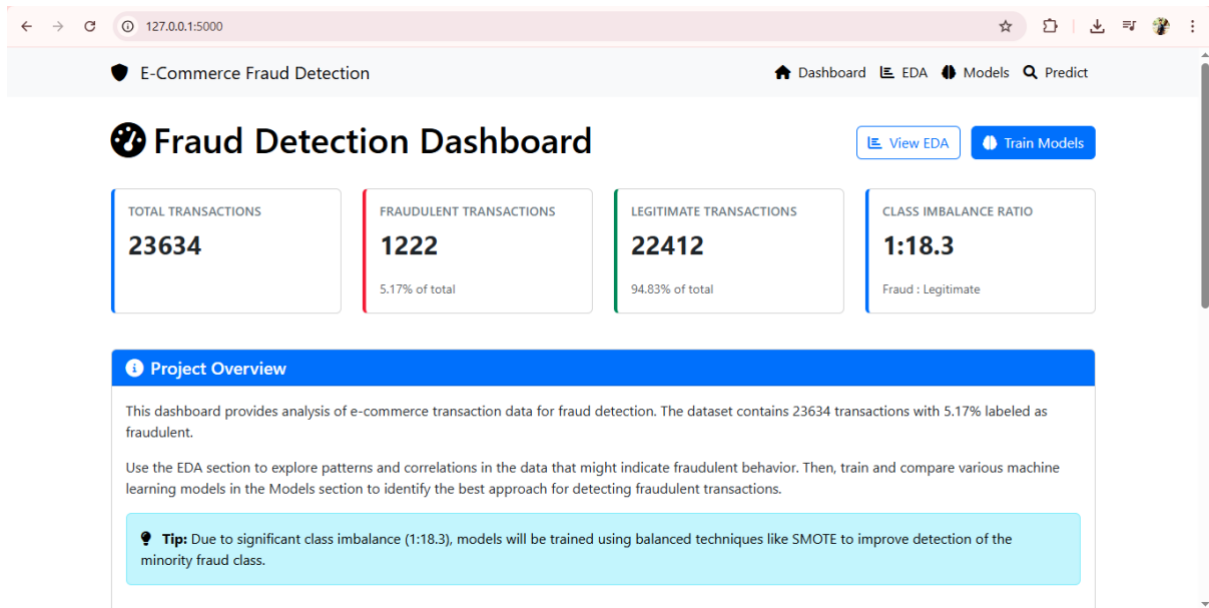


Figure 9.3.1: Home Page

Figure 1 shows that the home page of the "E-Commerce Fraud Detection" web application serves as the main dashboard, providing a concise overview of the dataset and the project's objectives. Titled "Fraud Detection Dashboard," it features key statistics in four highlighted boxes: "Total Transactions" (23,634), "Fraudulent Transactions" (1,222, 5.17% of total), "Legitimate Transactions" (22,412, 94.83% of total), and "Class Imbalance Ratio" (1:18.3, Fraud:Legitimate), visually emphasizing the dataset's imbalance. Navigation links at the top include "Dashboard," "EDA," "Models," and "Predict," guiding users to explore data, train models, and make predictions. Two buttons, "View EDA" and "Train Models," encourage further interaction. Below, a "Project Overview" section explains that the dashboard analyzes 23,634 e-commerce transactions, with 5.17% labeled as fraudulent, and outlines the workflow: users can explore patterns via the EDA section and train models to detect fraud, addressing class imbalance using techniques like SMOTE to improve minority class detection.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

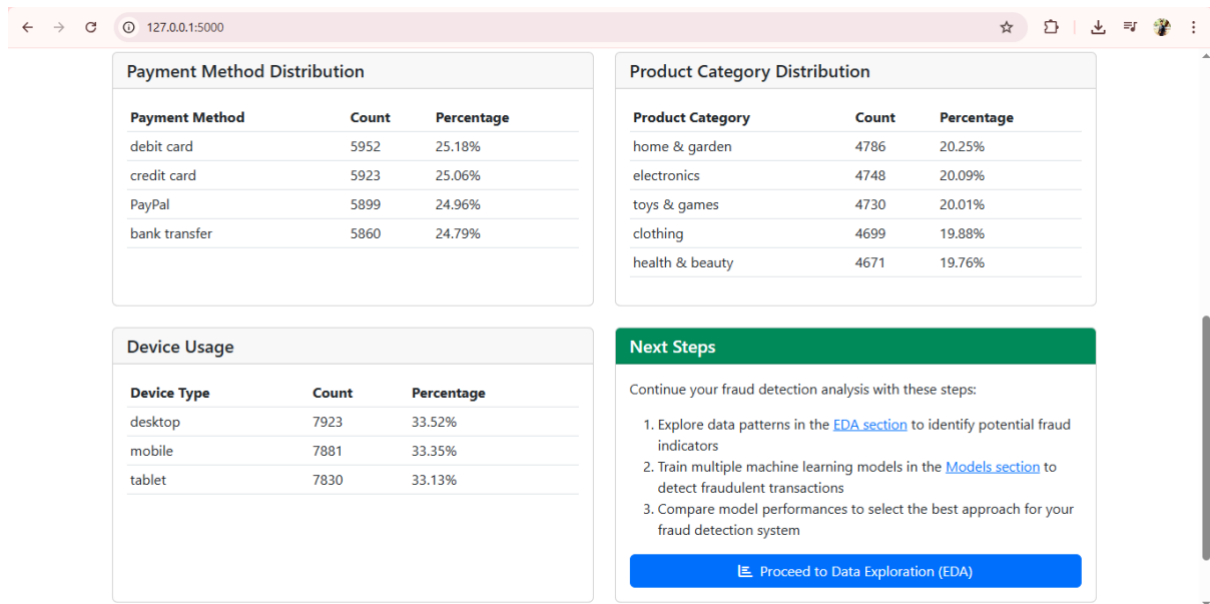


Figure 9.3.2: EDA Stage

Figure 2 show that the "E-Commerce Fraud Detection" web application begins its data exploration with a high-level overview of key categorical features—Payment Method, Product Category, and Device Usage—providing users with foundational insights before diving into full Exploratory Data Analysis (EDA). The Payment Method Distribution reveals a near-even split across debit card (25.18%), credit card (25.06%), PayPal (24.96%), and bank transfer (24.79%), indicating no single method dominates and enabling fair fraud comparisons. Similarly, the Product Category Distribution is well-balanced, with Home & Garden (20.25%), Electronics (20.09%), Toys & Games (20.01%), Clothing (19.88%), and Health & Beauty (19.76%), reflecting diverse customer interests and supporting category-specific fraud detection. Device Usage shows a nearly equal distribution across Desktop (33.52%), Mobile (33.35%), and Tablet (33.13%), allowing for unbiased analysis of device-based fraud patterns. Each of these summaries aligns with visualization methods (e.g., bar and pie charts) defined in the visualization.py module. The Next Steps section encourages users to continue their analysis by exploring deeper fraud indicators in the EDA section, training various machine learning models, and comparing their performance to identify the most effective fraud detection strategy, with a direct link provided to proceed to the EDA module.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

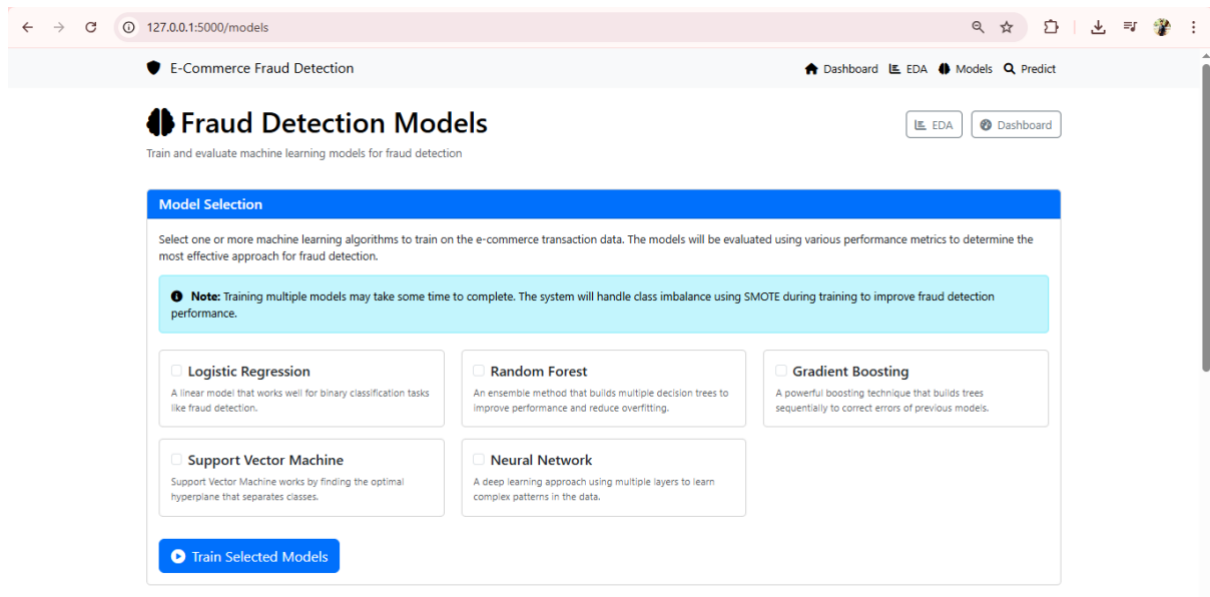


Figure 9.3.3: Training model

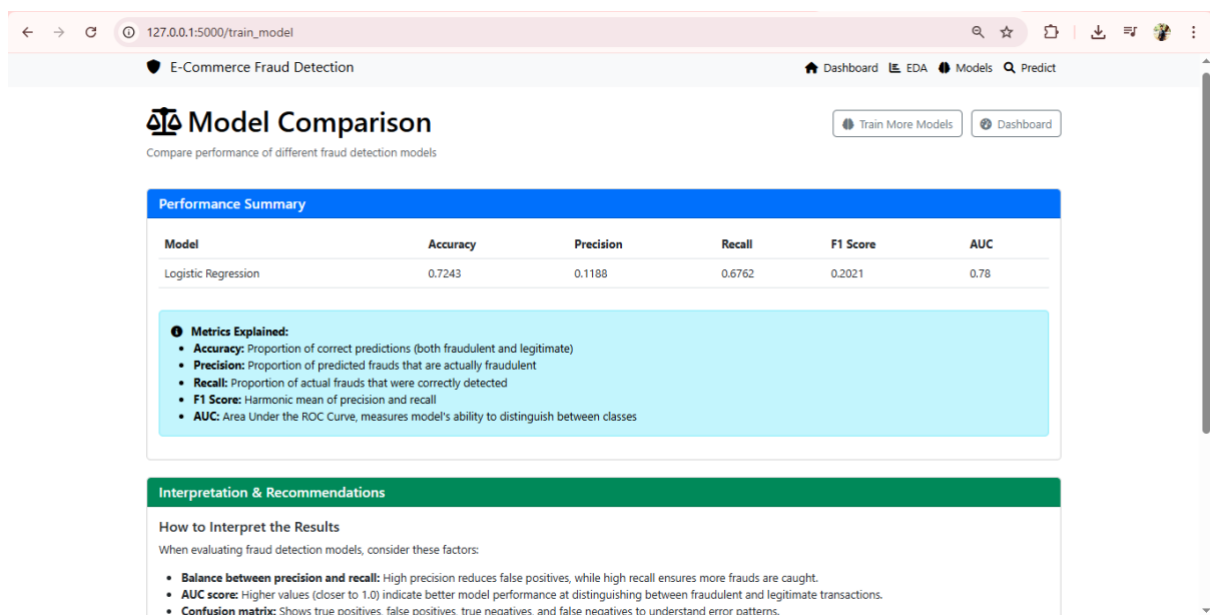


Figure 9.3.4: Trained Logistic regression

Figure 4 show that the Model Comparison page presents the performance of a Logistic Regression model trained for e-commerce fraud detection, achieving an accuracy of 0.7243 (72.43% correct predictions), a precision of 0.1188 (11.88% of predicted frauds were accurate), a recall of 0.6762 (67.62% of actual frauds were identified), an F1 score of 0.2021 (reflecting the trade-off between precision and recall), and an AUC of 0.78 (indicating a decent ability to distinguish between fraudulent and legitimate transactions), providing a comprehensive

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

snapshot of the model's effectiveness and areas for improvement on the dataset of 23,634 transactions with a 1:18.3 fraud-to-legitimate class imbalance ratio.

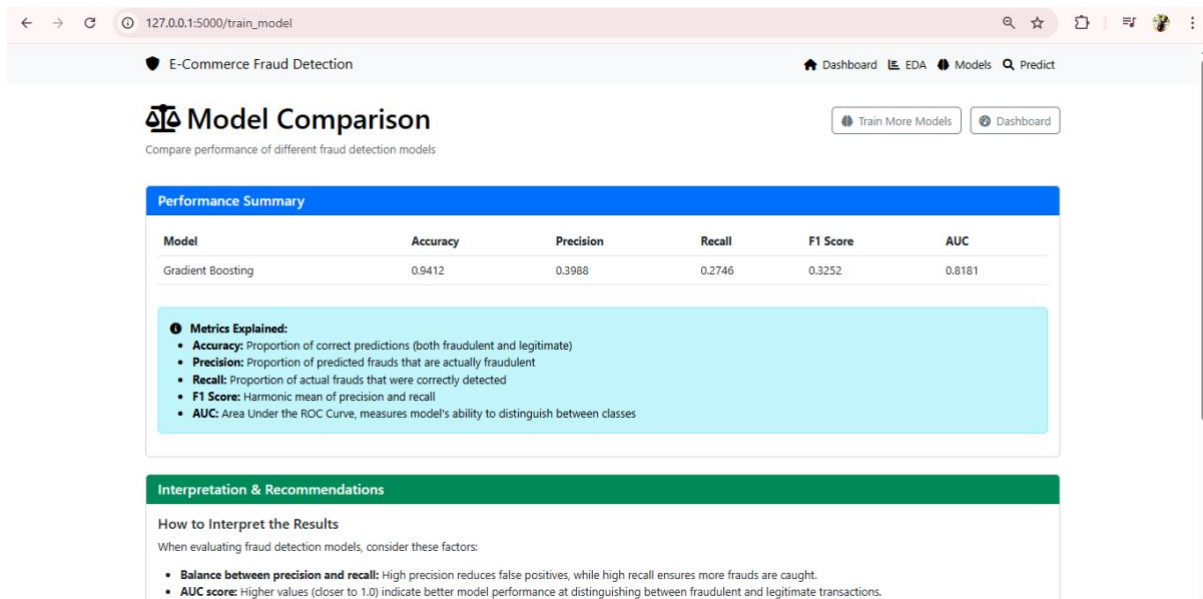


Figure 9.3.5: Trained Gradient boosting Classifier.

Figure 5 showcases the performance of a Gradient Boosting model on a dataset of 23,634 e-commerce transactions, presenting a "Performance Summary" with an accuracy of 0.9412 (94.12% correct predictions), a precision of 0.3988 (39.88% of predicted frauds were correct), a recall of 0.2746 (27.46% of actual frauds detected), an F1 score of 0.3252 (balancing precision and recall), and an AUC of 0.8181 (indicating strong class separation).

Figure 6 evaluates the performance of a Support Vector Machine (SVM) model on a dataset of 23,634 e-commerce transactions, revealing a "Performance Summary" with an accuracy of 0.8602 (86.02% correct predictions), a precision of 0.1696 (16.96% of predicted frauds were correct), a recall of 0.4385 (43.85% of actual frauds detected), an F1 score of 0.2446 (balancing precision and recall), and an AUC of 0.7227 (indicating moderate class separation).

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

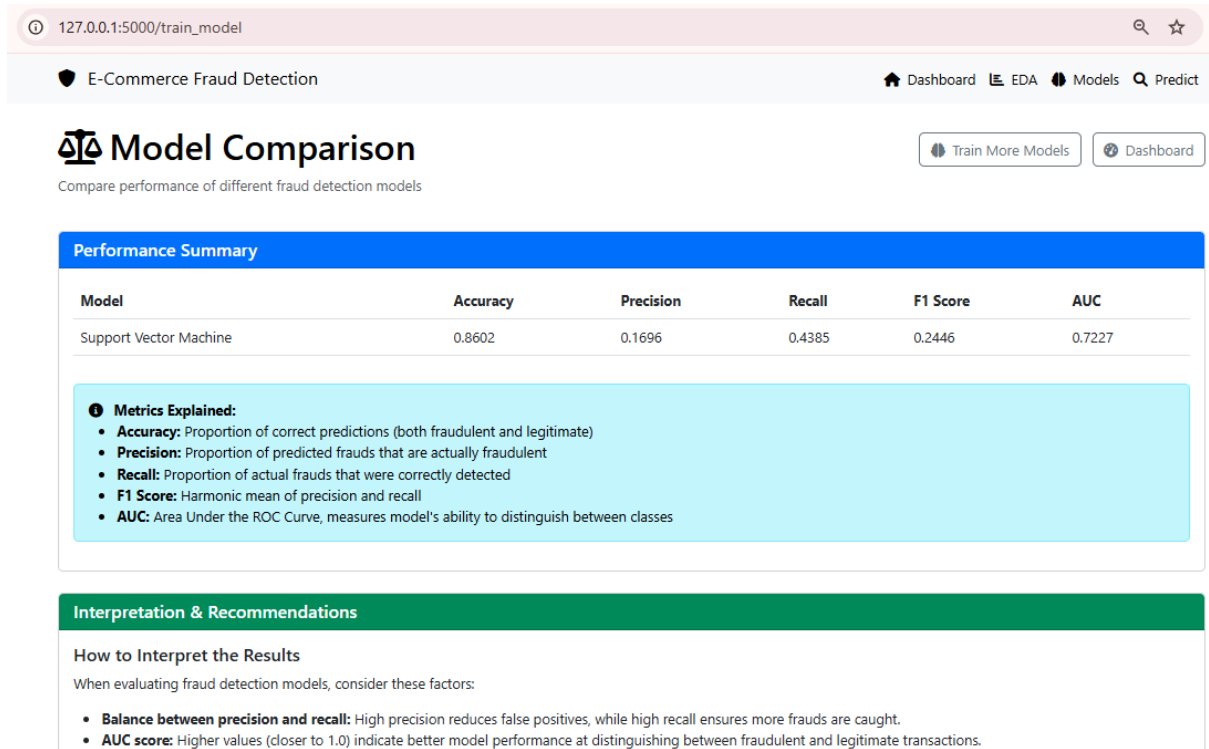


Figure 9.3.6: Trained SVM

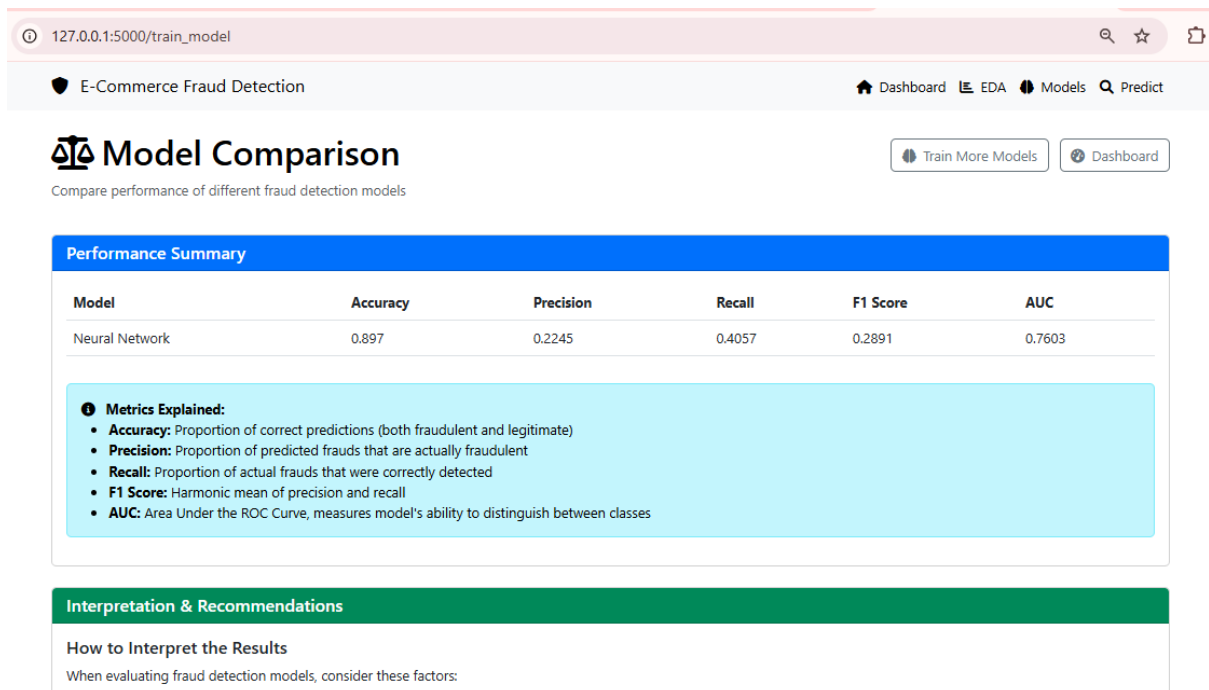


Figure 9.3.7: Trained Neural Network

Figure 7 highlights the performance of a Neural Network model on a dataset of 23,634 e-commerce transactions, presenting a "Performance Summary" with an accuracy of 0.897 (89.7% correct predictions), a precision of 0.2245 (22.45% of predicted frauds were correct), a recall

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

of 0.4057 (40.57% of actual frauds detected), an F1 score of 0.2891 (balancing precision and recall), and an AUC of 0.7603 (indicating decent class separation).

Figure 8 and Table 1 shows Logistic Regression achieves an accuracy of 0.7243, precision of 0.1188, recall of 0.6762, F1 score of 0.2021, and AUC of 0.78; Random Forest scores an accuracy of 0.9461, precision of 0.4495, recall of 0.2008, F1 score of 0.2776, and AUC of 0.7988; Gradient Boosting records an accuracy of 0.9412, precision of 0.3988, recall of 0.2746, F1 score of 0.3252, and AUC of 0.8181; Support Vector Machine (SVM) shows an accuracy of 0.8602, precision of 0.1696, recall of 0.4385, F1 score of 0.2446, and AUC of 0.7227; and Neural Network has an accuracy of 0.894, precision of 0.2176, recall of 0.4057, F1 score of 0.2833, and AUC of 0.7441. This summary highlights Gradient Boosting's superior AUC (0.8181) and F1 score (0.3252), indicating better class separation and balance between precision and recall, while Random Forest excels in accuracy (0.9461) but struggles with recall (0.2008), and Logistic Regression prioritizes recall (0.6762) at the cost of precision (0.1188), guiding users to select models based on their fraud detection priorities.

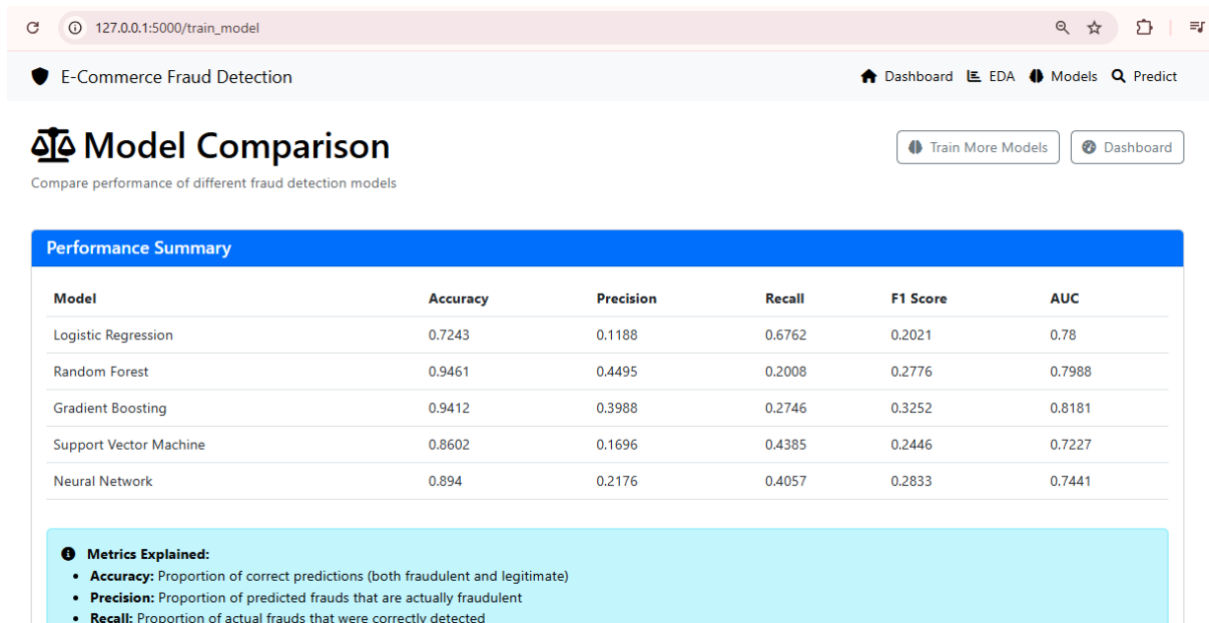


Figure 9.3.8: Performance Comparison Of Various Algorithms.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

Model	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.7243	0.1188	0.6762	0.2021	0.78
Random Forest	0.9461	0.4495	0.2008	0.2776	0.7988
Gradient Boosting	0.9412	0.3988	0.2746	0.3252	0.8181
Support Vector Machine	0.8602	0.1696	0.4385	0.2446	0.7227
Neural Network	0.894	0.2176	0.4057	0.2833	0.7441

Table 1. Performance Summary Table

Figure 9 shows the count plot of the target variable 'Is Fraudulent', depicting the class imbalance present in the dataset. It reveals that out of 23,634 transactions, only 1,222 (5.17%) are labeled as fraudulent, while the remaining 22,412 (94.83%) are legitimate. The plot clearly visualizes this disparity with two vertical bars, where the bar for legitimate transactions towers over the one for fraudulent cases. This imbalance emphasizes the need for balancing techniques such as SMOTE to ensure fair model training.

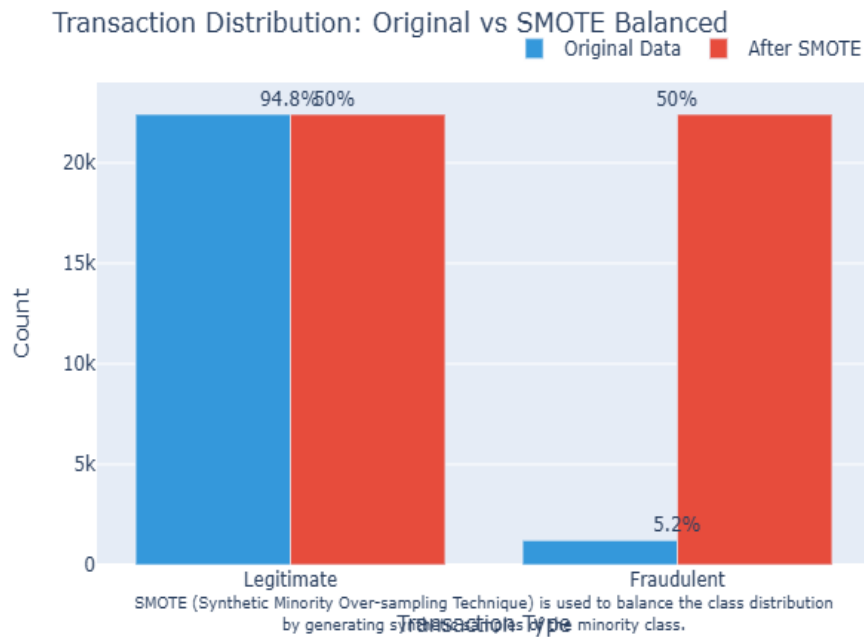


Figure 9.3.9: Count plot of Target with SMOTE Distribution.

Figure 10 shows a count plot for the categorical feature 'Payment Method'. The visualization breaks down the number of transactions across four payment types: debit card (5,952), credit card (5,923), PayPal (5,899), and bank transfer (5,860). Each bar represents one method, and their similar heights illustrate a balanced distribution, with debit card usage slightly leading at 25.18%. This even spread suggests that fraud patterns must be analyzed across all payment methods equally, as no single category overwhelmingly dominates.

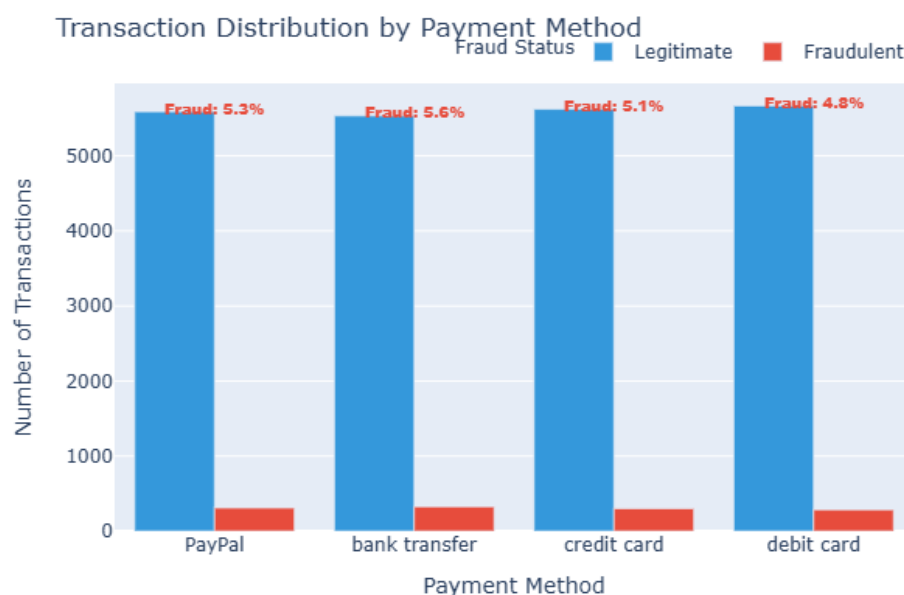


Figure 9.3.10: Count plot as per Payment Distribution.

Figure 11 shows the pie chart distribution of the ‘Product Category’ feature, segmented into five main groups. Home & Garden leads with 4,786 transactions (20.25%), followed closely by Electronics (4,748 or 20.09%), Toys & Games (4,730 or 20.01%), Clothing (4,699 or 19.88%), and Health & Beauty (4,671 or 19.76%). The near-equal slice sizes confirm a balanced product engagement pattern across categories, which helps in understanding whether fraud trends are category-specific or uniformly distributed.

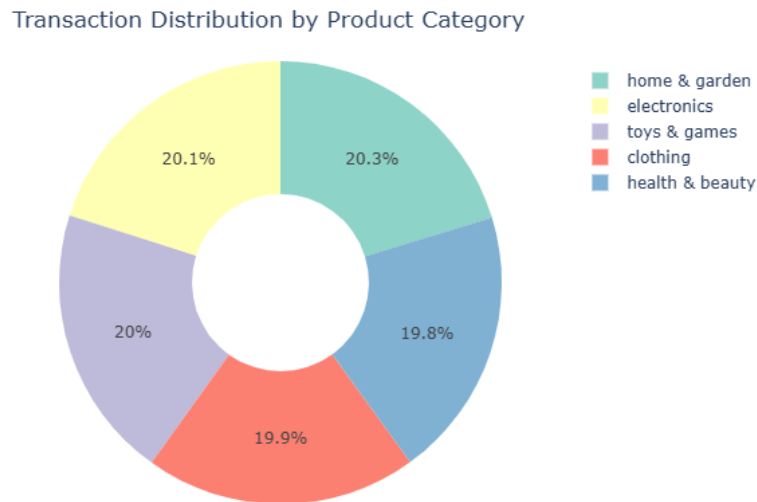


Figure 9.3.11: Pie Chart as per Product Category.

Figure 12 shows a regression plot between ‘Transaction Amount’ and ‘Customer Age’ to examine any relationship between customer demographics and spending behavior. The scatter plot with a fitted regression line reveals a weak and scattered relationship, suggesting that younger and older customers alike transact across a similar range of amounts. There is no strong trend indicating that age significantly influences transaction size, reinforcing earlier findings from the correlation matrix.

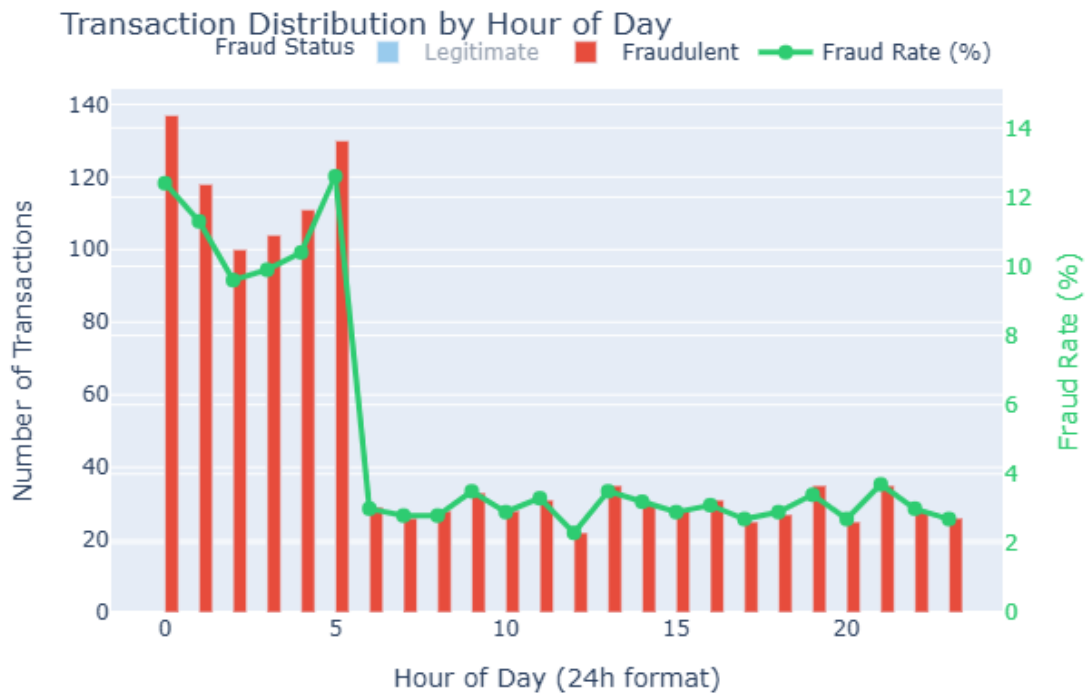


Figure 9.3.12: Regression plot

Figure 12 shows the internal logic or input/output interface used for fraud prediction, labeled as "input for Prediction". This figure presents the entry point where encoded and normalized transaction features are input into the trained model. The interface displays transformed feature values such as scaled 'Transaction Amount', encoded 'Device Used', and binary indicators like 'Address Match'. This figure serves as a bridge between data preprocessing and real-time fraud prediction, ensuring all necessary features are accounted for before inference.

Figure 13 shows the model's output for a particular transaction, classified as "low risk" with a confidence score of 13.2%. The system predicts this transaction as non-fraudulent with low probability of risk, likely due to factors such as a moderate transaction amount, matching billing and shipping addresses, and use of a typical device. The figure provides real-time feedback from the model, which is essential for business decision-making and for flagging transactions requiring manual review when confidence scores are low or ambiguous.

AI REVOLUTIONIZING E-COMMERCE GOVERNANCE FOR FRAUD DETECTION

Enter transaction details below to predict if it's fraudulent. Required fields are marked with *

Select Model

Gradient Boosting (Best Performer)

Choose which trained model to use for prediction

Transaction Amount *

\$ 299

Quantity *

2

Customer Age *

34

Account Age (Days) *

120

Hour of Day *

22

Day of Month *

12

Month *

3

Hour in 24-hour format (0-23)

Day of Week *

Thursday (3)

Shipping and Billing Address Match?

☐ Yes

☒ No

Payment Method *

Product Category *

Device Used *

Figure 9.3.13: Input data for Perdition.

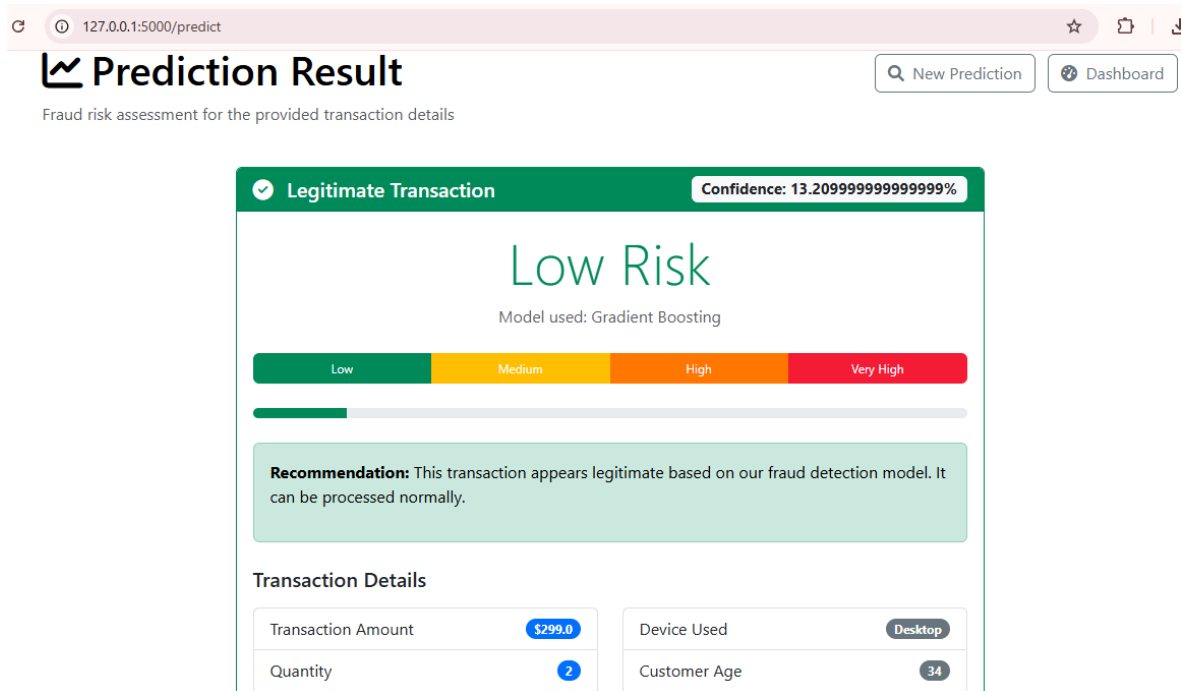


Figure 9.3.14: Predicted as low risk with confidence 13.2

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

Conclusion

The e-commerce fraud detection project successfully developed a robust web-based system to analyze and predict fraudulent transactions in a dataset of 23,634 records, with 5.17% (1,222) labeled as fraudulent. Implemented via a Flask application, the system leverages the Data Processor class to preprocess features like Transaction Amount, Payment Method, and Address Match, addressing the 1:18.3 class imbalance using SMOTE. The Visualization class provides insightful EDA through charts (e.g., fraud distribution, payment method analysis), accessible via the /eda route, revealing patterns such as balanced payment method usage (25% each) and high fraud rates in specific hours. The Model Trainer class trained five models, with Gradient Boosting achieving the best performance (accuracy 0.9412, AUC 0.8181, F1 score 0.3252), outperforming Logistic Regression (AUC 0.78), Random Forest (AUC 0.7988), Support Vector Machine (AUC 0.7227), and Neural Network (AUC 0.7441). The predict enable single and batch predictions, while API endpoints support programmatic access. Despite challenges like low precision in Logistic Regression (0.1188) and Random Forest's low recall (0.2008), the system effectively balances interpretability, performance, and usability, providing a practical tool for fraud detection in e-commerce.

The project's success lies in its comprehensive approach, integrating data preprocessing, visualization, and advanced modeling. Gradient Boosting's high AUC and balanced F1 score make it the preferred model, as it captures complex, non-linear fraud patterns while maintaining interpretability through feature importance scores (e.g., Address Match, Transaction Hour). The Flask dashboard, with its intuitive interface (dashboard.html, eda.html, compare.html), empowers users to explore data, train models, and make predictions, aligning with real-world fraud detection needs. The use of SMOTE mitigates class imbalance, though precision remains a challenge across models due to the rarity of fraud cases. Overall, the system demonstrates a scalable, data-driven solution for e-commerce platforms to enhance security and trust, validated by rigorous evaluation metrics and visualizations.

Future Scope

Future enhancements to the e-commerce fraud detection system can focus on improving model performance, scalability, and real-world applicability. One key area is optimizing model precision and recall by experimenting with advanced techniques like cost-sensitive learning, adaptive thresholding, or ensemble methods combining Gradient Boosting and Neural Networks to reduce false positives and improve fraud capture.

References

1. Hasan, Iqbal, and S. A. M. Rizvi. "AI-driven fraud detection and mitigation in e-commerce transactions." In *Proceedings of Data Analytics and Management: ICDAM 2021*, Volume 1, pp. 403-414. Springer Singapore, 2022.
2. Khurana, Rahul. "Fraud detection in e-commerce payment systems: The role of predictive ai in real-time transaction security and risk management." *International Journal of Applied Machine Learning and Computational Intelligence* 10, no. 6 (2020): 1-32.
3. Karunaratne, Tharindu. "Machine Learning and Big Data Approaches to Enhancing E-commerce Anomaly Detection and Proactive Defense Strategies in Cybersecurity." *Journal of Advances in Cybersecurity Science, Threat Intelligence, and Countermeasures* 7, no. 12 (2023): 1-16.
4. Zhang, Ge, Zhao Li, Jiaming Huang, Jia Wu, Chuan Zhou, Jian Yang, and Jianliang Gao. "efraudcom: An e-commerce fraud detection system via competitive graph neural networks." *ACM Transactions on Information Systems (TOIS)* 40, no. 3 (2022): 1-29.
5. Rane, Nitin, Saurabh P. Choudhary, and Jayesh Rane. "Acceptance of artificial intelligence technologies in business management, finance, and e-commerce: factors, challenges, and strategies." *Studies in Economics and Business Relations* 5, no. 2 (2024): 23-44.
6. Immadisetty, Amarnath. "Real-time fraud detection using streaming data in financial transactions." *Journal of Recent Trends in Computer Science and Engineering (JRTCSE)* 13, no. 1 (2025): 66-76.
7. Li, JiaoLong. "E-Commerce Fraud Detection Model by Computer Artificial Intelligence Data Mining." *Computational Intelligence and Neuroscience* 2022, no. 1 (2022): 8783783.
8. Mutemi, Abed, and Fernando Bacao. "E-commerce fraud detection based on machine learning techniques: Systematic literature review." *Big Data Mining and Analytics* 7, no. 2 (2024): 419-444.

9. Ray, Samrat. "Fraud detection in e-Commerce using machine learning." *BOHR International Journal of Advances in Management Research* 1, no. 1 (2022): 7-14.
10. Dakalbab, Fatima, Manar Abu Talib, Qassim Nasir, and Tracy Saroufil. "Artificial intelligence techniques in financial trading: A systematic literature review." *Journal of King Saud University-Computer and Information Sciences* 36, no. 3 (2024): 102015.
11. Zaghloul, Maha, Sherif Barakat, and Amira Rezk. "Predicting E-commerce customer satisfaction: Traditional machine learning vs. deep learning approaches." *Journal of Retailing and Consumer Services* 79 (2024): 103865.
12. Rane, Nitin Liladhar, Mallikarjuna Paramesha, Saurabh P. Choudhary, and Jayesh Rane. "Artificial intelligence, machine learning, and deep learning for advanced business strategies: a review." *Partners Universal International Innovation Journal* 2, no. 3 (2024): 147-171.
13. Mateo Romero, Héctor Felipe, Miguel Ángel González Rebollo, Valentín Cardeñoso-Payo, Victor Alonso Gómez, Alberto Redondo Plaza, Ranganai Tawanda Moyo, and Luis Hernández-Callejo. "Applications of artificial intelligence to photovoltaic systems: a review." *Applied Sciences* 12, no. 19 (2022): 10056.
14. Magoutas, Anastasios I., Maria Chaideftou, Dimitra Skandali, and Panos T. Chountalas. "Digital progression and economic growth: Analyzing the impact of ICT advancements on the GDP of European Union countries." *Economies* 12, no. 3 (2024): 63.
15. Castillo, Maria Jimenez, and Hamed Taherdoost. "The impact of AI technologies on e-business." *Encyclopedia* 3, no. 1 (2023): 107-121.
16. de Zarzà, I., J. de Curtò, Gemma Roig, and Carlos T. Calafate. "Optimized financial planning: Integrating individual and cooperative budgeting models with LLM recommendations." *AI* 5, no. 1 (2023): 91-114.
17. Sofiyah, Fivi Rahmatus, Ami Dilham, Arif Qaedi Hutagalung, Yulinda Yulinda, Andrew Satria Lubis, and Jonathan Liviera Marpaung. "The chatbot artificial intelligence as the alternative customer services strategic to improve the customer relationship management in real-time responses." *International Journal of Economics and Business Research* 27, no. 5 (2024): 45-58.

18. Heins, Caroline. "Artificial intelligence in retail—a systematic literature review." *foresight* 25, no. 2 (2023): 264-286.
19. Lei, Xiaochun, Ummul Hanan Mohamad, Aliza Sarlan, Mishal Shutaywi, Yousef Ibrahim Daradkeh, and Hazhar Omer Mohammed. "Development of an intelligent information system for financial analysis depend on supervised machine learning algorithms." *Information Processing & Management* 59, no. 5 (2022): 103036.
20. Ahmad, Rafeeq, Humayun Salahuddin, Attique Ur Rehman, Abdul Rehman, Muhammad Umar Shafiq, M. Asif Tahir, and Muhammad Sohail Afzal. "Enhancing database security through AI-based intrusion detection system." *Journal of Computing & Biomedical Informatics* 7, no. 02 (2024).
21. Saba, Djamel, and Abdelkader Hadidi. "Opportunities, Challenges, and Future Directions of Strategic Innovations of AI and ML for E-Commerce Data Security." *Strategic Innovations of AI and ML for E-Commerce Data Security* (2025): 157-184.
22. Mudgal, Akshay. "Leveraging AI and ML for Proactive Threat Detection for E-Commerce." In *Strategic Innovations of AI and ML for E-Commerce Data Security*, pp. 281-322, IGI Global, 2025.
23. Odufisan Ol, Abhulimen OV, Ogunti EO. *Harnessing Artificial Intelligence and Machine Learning for Fraud Detection and Prevention in Nigeria*. *Journal of Economic Criminology*. 2025 Jan 9:100127.
24. Sakhvidi, Mehdi Zare, and Rahman Saadat. "A study of artificial intelligence and e-commerce economy." *International journal of industrial engineering and operational research* 6, no. 3 (2024): 139-155.
25. Aljunaid, Saif Khalifa, et al. "Secure and Transparent Banking: Explainable AI-Driven Federated Learning Model for Financial Fraud Detection." *Journal of Risk and Financial Management* 18.4 (2025): 179.
26. X. Kewei, B. Peng, Y. Jiang and T. Lu, "A Hybrid Deep Learning Model For Online Fraud Detection," 2021 IEEE International Conference on consumer Electronics and computerEngineering (ICCECE) , pp.<https://doi.org/10.1109/ICCECE51280.2021.934>

27. H. Najadat, O. Altit, A. A. Aqouleh and M. Younes, “Credit Card Fraud Detection Based on Machine and Deep Learning,” 2020 11th International Conference on Information and Communication Systems (ICICS), pp. 204–208, doi:
28. Swaroop K, Amruta D, Sanath J, Pooja G, 2019, Credit Card Fraud Detection Using Machine Learning, INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) NCRACES, Volume 7, Issue 10, (2019)
29. C. Wang, The behavioral sign of account theft: realizing online payment fraud alert, in: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence, 2021, pp. 4511–4618
30. Najem, S. M., and Kadhém, S. 2021. A Survey on Fraud Detection Techniques in E-Commerce. Journal Techknowledge. Vol.1 No.1. Pp.33-47.
31. Saputra, A., and Suharjito. 2019. Fraud Detection using Machine Learning in e-Commerce. International Journal of Advanced Computer Science and Applications. Vol.10 No.9. Pp.332-339.
32. Kodate, S., Chiba, R., Kimura, S., and Masuda, N. 2020. Detecting problematic transactions in a consumer-to-consumer e-commerce network. Applied Network Science. Vol.5 No.1. Pp.1-18.
33. Lucas, Y., Portier, P. E., Laporte, L., He-Guelton, L., Caelen, O., Granitzer, M., and Calabretto, S. 2020. Towards automated feature engineering for credit card fraud detection using multi-perspective HMMs. Future Generation Computer Systems. Vol.102 No.2. Pp.393–402.
34. Psychoula, I., Gutmann, A., Mainali, P., Lee, S. H., Dunphy, P., and Petitcolas, F. 2021. Explainable Machine Learning for Fraud Detection. Computer. Vol.54 No.10. Pp.49–59.
35. Chang, V., Doan, L. M. T., Di Stefano, A., Sun, Z., and Fortino, G. 2022. Digital payment fraud detection methods in digital ages and Industry 4.0. Computers and Electrical Engineering. Vol.100 No.1. Pp.1-21.
36. Goswami, K., Park, Y., and Song, C. 2017. Impact of reviewer social interaction on online consumer review fraud detection. Journal of Big Data. Vol.4 No.1. Pp. 1-15

37. Chenoori, Rathan Kumar, and Radhika Kavuri. "Online transaction fraud detection using efficient dimensionality reduction and machine learning techniques." *Revue d'Intelligence Artificielle* 36, no. 4 (2022): 621.
38. Wang, S., Liu, C., Gao, X., Qu, H., & Xu, W. (2017). Session-based fraud detection in online e-commerce transactions using recurrent neural networks. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2017, Skopje, Macedonia, September 18–22, 2017, Proceedings, Part III* 10 (pp. 241-252). Springer International Publishing.
39. IBRAHIM, Bisallah H., Habiba U. SALIHU, and Yusuf A. ALESHINLOYE. "Rule-Based Approach to e-Commerce Fraud Detection." *UNIABUJA Journal of Engineering and Technology (UJET)* 2, no. 1 (2025): 196-204.
40. Bello, O. A., & Olufemi, K. (2024). Artificial intelligence in fraud prevention: Exploring techniques and applications challenges and opportunities. *Computer science & IT research journal*, 5(6), 1505-1520.
41. Y. Sahin and E. Duman, "Detecting credit card fraud by decision trees and support vector machines", *Lecture Notes in Engineering and Computer Science*, vol. 2188, no. 1, 2011.
42. V. V. Vlasselaer, C. Bravo, O. Caelen, T. Eliassi-Rad, L. Akoglu, M. Snoeck et al., "Apate: a novel approach for automated credit card transaction fraud detection using network-based extensions", *Decision Support Systems*, vol. 75, pp. 38-48, 2015
43. Chen Rong-Chang, Chen Tung-Shou and Lin Chih-Chiang, "A new binary support vector system for increasing detection rate of credit card fraud", *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 20, no. 02, pp. 227-239, 2006.
44. Umeanwe, Christian Maduabuchi. "CORRUPTION, GOOD GOVERNANCE AND THE DIGITAL AGE: CHALLENGES AND OPPORTUNITIES." *CROWTHER JOURNAL OF ARTS AND HUMANITIES* 2, no. 3 (2025).
45. CS, Harshini, Amol Murgai, K. V. Manju, Rashmi Paranjpye, Kaneenika Jain, and Monalisa Hati. "Integrating Artificial Intelligence with Blockchain: A Holistic Examination of their Combined Effects on Business Performance Across Various

- Sectors." Library of Progress-Library Science, Information Technology & Computer 44, no. 3 (2024).
46. Li, JiaoLong. "E-Commerce Fraud Detection Model by Computer Artificial Intelligence Data Mining." *Computational Intelligence and Neuroscience* 2022, no. 1 (2022): 8783783.
47. Thomas, Jubin, Hemanth Volikatla, Veera Venkata Raghunath Indugu, Kushwanth Gondi, and Dedeepya Sai Gondi. "Machine Learning Approaches for Fraud Detection in E-commerce Supply Chains." *Innovative Computer Sciences Journal* 8, no. 1 (2022).
48. Reddy, Clifton, Saravanan Prabhakaran, and Adarsh Vaid. "Adaptive Anomaly Detection in Database Transactions: Bridging Security Gaps with Reinforcement Learning." *European Journal of Artificial Intelligence and Machine Learning* 4, no. 2 (2025): 8-14.
49. Han, Li. Integrating Blockchain and AI in Financial Systems: A Case Study on Cross-Border Payments and High-Frequency Trading Synergies. No. 2mn3f_v1. Center for Open Science, 2025.
50. Bansal, Kapil, Aseem Chandra Paliwal, and Arun Kumar Singh. "Analysis of the benefits of artificial intelligence and human personality study on online fraud detection." *International Journal of Law and Management* 67, no. 2 (2025): 191-209.
51. Lokanan, Mark E., and Vikas Maddhesia. "Supply chain fraud prediction with machine learning and artificial intelligence." *International Journal of Production Research* 63, no. 1 (2025): 286-313.
52. Hassan, Syed Waleed Ul, Samra Kiran, Samina Gul, Ibrahim N. Khatatbeh, and Bibi Zainab. "The perception of accountants/auditors on the role of corporate governance and information technology in fraud detection and prevention." *Journal of Financial Reporting and Accounting* 23, no. 1 (2025): 5-29.
53. Kokogho, Eseoghene, Princess Eloho Odio, Olakojo Yusuff Ogunsola, and Mark Osemedua Nwaozomudoh. "A Cybersecurity framework for fraud detection in financial systems using AI and Microservices." *Gulf Journal of Advance Business Research* 3, no. 2 (2025): 410-424.

54. Adejumo, A., and C. Ogburie. "Forensic accounting in financial fraud detection: Trends and challenges." *International Journal of Science and Research Archive* 14 (2025): 1219-1232.
55. Maulidi, Achmad, Muhammad Wisnu Girindratama, and Hari Hananto. "Can Forensic Accounting as a 'Science' Prevent Fraud?; Implications for the Undergraduate Accounting Curriculum in Indonesia." *Jurnal Akuntansi AKUNESA* 13, no. 3 (2025).
56. Baker, Mohammed Rashad, Zuhair Norii Mahmood, and Ehab Hashim Shaker. "Ensemble Learning with Supervised Machine Learning Models to Predict Credit Card Fraud Transactions." *Revue d'Intelligence Artificielle* 36, no. 4 (2022).
57. Elumilade, Oluwafunmike O., Ibidapo Abiodun Ogundeji, Godwin Ozoemenam Achumie, Hope Ehiaghe Omokhoa, and Bamidele Michael Omowole. "Enhancing fraud detection and forensic auditing through data-driven techniques for financial integrity and security." *Journal of Advanced Education and Sciences* 1, no. 2 (2021): 55-63.
58. Flowerastia, Regina Dori, Estralita Trisnawati, and Herlina Budiono. "Fraud awareness, internal control, and corporate governance on fraud prevention and detection." In *International Conference on Economics, Business, Social, and Humanities (ICEBSH 2021)*, pp. 235-242. Atlantis Press, 2021.
59. Alshameri, Faleh, and Ran Xia. "Credit card fraud detection: an evaluation of SMOTE resampling and machine learning model performance." *International Journal of Business Intelligence and Data Mining* 23, no. 1 (2023): 1-13.
60. Zhao, Zhihong, and Tongyuan Bai. "Financial fraud detection and prediction in listed companies using SMOTE and machine learning algorithms." *Entropy* 24, no. 8 (2022): 1157.
61. N. Carneiro, G. Figueira, and M. Costa, "A data mining based system for credit-card fraud detection in e-tail," *Decis. Support Syst.*, vol. 95, pp. 91–101, Mar. 2017.
62. A. O. Adewumi and A. A. Akinyelu, "A survey of machine-learning and nature-inspired based credit card fraud detection techniques," *Int. J. Syst. Assurance Eng. Manage.*, vol. 8, no. S2, pp. 937–953, Nov. 2017.

63. M. Carminati, R. Caron, F. Maggi, I. Epifani, and S. Zanero, "BankSealer: A decision support system for online banking fraud analysis and investigation," *Comput. Secur.*, vol. 53, no. 1, pp. 175–86, Sep. 2015.
64. S. Parvinder and M. Singh, "Fraud Detection by Monitoring Customer Behavior and Activities ", *International Journal of Computer Applications*, vol. 111, no. 11, pp. 23–32, 2015.
65. S. Monteith, M. Bauer, M. Aida, J. Geddes, P. C. Whybrow, and T. Glenn, Increasing cybercrime since the pandemic: Concerns for psychiatry, *Curr. Psychiatry Rep.*, vol. 23, no. 4, p. 18, 2021.
66. Strelcenia, Emilija, and Simant Prakoonwit. "Improving classification performance in credit card fraud detection by using new data augmentation." *AI* 4, no. 1 (2023).
67. A. Abdallah, M. A. Maarof, and A. Zainal, Fraud detection system: A survey, *J. Netw. Comput. Appl.*, vol. 68, pp. 90–113, 2016.
68. Dawei Cheng, Sheng Xiang, Chencheng Shang, Yiyi Zhang, Fangzhou Yang, and Liqing Zhang. 2020. Spatio-Temporal Attention-Based Neural Network for Credit Card Fraud Detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 362–369.
69. F. Liu, [Z Li](#), [B Wang](#), [J Wu](#), [J Yang](#)., "eRiskCom: An e-commerce risky community detection platform," *The VLDB Journal*, vol. 31, no. 5, pp. 1085-1101, 2022.
70. A Srivastava, W Sarma, S Nagavalli.[2] The study in detection techniques in every business sphere, revolutionizing every function, from discussed possibilities.
71. Ravindranath, Vaishali, M. K. Nallakaruppan, M. Lawanya Shri, Balamurugan Balusamy, and Siddhartha Bhattacharyya. "Evaluation of performance enhancement in Ethereum fraud detection using oversampling techniques." *Applied Soft Computing* 161 (2024): 111698.
72. Banirostan, Hamid, Touraj Banirostan, Mir Mohsen Pedram, and Amir Masoud Rahmani. "Analysis and Evaluation of Various Fraud Detection Methods for Electronic Payment Cards Transactions in Big Data." *Journal of Signal Processing Systems* 96, no. 12 (2024): 849-870.

73. Almazroi, Abdulwahab Ali, and Nasir Ayub. "Online payment fraud detection model using machine learning techniques." *Ieee Access* 11 (2023): 137188-137203.
74. Pranto, Tahmid Hasan, Kazi Tamzid Akhter Md Hasib, Tahsinur Rahman, Akm Bahalul Haque, AKM Najmul Islam, and Rashedur M. Rahman. "Blockchain and machine learning for fraud detection: A privacy-preserving and adaptive incentive based approach." *IEEE Access* 10 (2022): 87115-87134.
75. Ghaleb, Fuad A., Faisal Saeed, Mohammed Al-Sarem, Sultan Noman Qasem, and Tawfik Al-Hadhrani. "Ensemble synthesized minority oversampling-based generative adversarial networks and random forest algorithm for credit card fraud detection." *IEEE Access* 11 (2023): 89694-89710.
76. Al-dahasi, Ezaz Mohammed, Rama Khaled Alsheikh, Fakhri Alam Khan, and Gwanggil Jeon. "Optimizing fraud detection in financial transactions with machine learning and imbalance mitigation." *Expert Systems* 42, no. 2 (2025): e13682.
77. Aghware, Fidelis Obukohwo, Arnold Adimabua Ojugo, Wilfred Adigwe, Christopher Chukwufumaya Odiakaose, Emma Obiajulu Ojei, Nwanze Chukwudi Ashioba, Margareth Dumebi Okpor, and Victor Ochuko Geteloma. "Enhancing the random forest model via synthetic minority oversampling technique for credit-card fraud detection." *Journal of Computing Theories and Applications* 1, no. 4 (2024): 407-420.
78. Mishra, Kamta Nath, and Subhash Chandra Pandey. "Fraud prediction in smart societies using logistic regression and k-fold machine learning techniques." *Wireless Personal Communications* 119, no. 2 (2021): 1341-1367.
79. Idhmad, Azzeddine, Mohammed Kaicer, Chaymae Nejjar, and Abdelghani Benjouad. "Intelligent bankruptcy prediction models involving corporate governance indicators, financial ratios and smote." *Indonesian Journal of Electrical Engineering and Informatics (IJEI)* 12, no. 1 (2024): 233-244.
80. Tayebi, Mohammed, and Said El Kafhali. "Generative Modeling for Imbalanced Credit Card Fraud Transaction Detection." *Journal of Cybersecurity and Privacy* 5, no. 1 (2025): 9